



UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

SICUREZZA
DELL'INFRASTRUTTURA AWS IN
UNA STARTUP FINTECH

Relatore: Prof. Claudio Agostino Ardagna

Correlatore: Lorenzo Perotta, Andrea Pasini, Simone Cortese

Tesi di:
Andrea Ferraboli
Matricola: 09985A

Anno Accademico 2024-2025

Prefazione

La crescente adozione delle tecnologie cloud ha rivoluzionato il settore fintech, portando benefici significativi in termini di scalabilità, flessibilità e riduzione dei costi operativi. Dall'altro lato, questa trasformazione digitale introduce nuove sfide di sicurezza derivanti dall'outsourcing dell'infrastruttura IT e dalla gestione di dati finanziari sensibili in ambienti distribuiti. Per le startup fintech, queste problematiche sono amplificate dalla necessità di bilanciare rapidità di sviluppo e conformità normativa con standard di sicurezza elevati.

Amazon Web Services (AWS) rappresenta oggi una delle piattaforme cloud più adottate dalle startup fintech per la sua robustezza e completezza dell'offerta di servizi. Tuttavia, la configurazione sicura di un'infrastruttura AWS richiede competenze specifiche e l'implementazione di strategie di sicurezza avanzate che vanno oltre le configurazioni standard. L'obiettivo di questa tesi è stato analizzare e implementare un'architettura di sicurezza completa su AWS specificamente progettata per le esigenze di una startup fintech.

La ricerca si concentra sull'implementazione pratica di controlli di sicurezza avanzati, tra cui la gestione granulare delle identità e degli accessi (IAM), la progettazione di reti sicure mediante Virtual Private Cloud (VPC), e l'implementazione di sistemi di rilevamento proattivo delle minacce attraverso honeypot. Particolare attenzione è stata posta alla creazione di un ambiente che sia al contempo sicuro, scalabile e conforme ai principali framework di sicurezza internazionali.

L'elaborato è organizzato come segue:

Capitolo 1 – Introduzione In questo capitolo si presenta il contesto delle startup fintech, analizzando le specificità del settore e le principali sfide di sicurezza che caratterizzano queste realtà innovative.

Capitolo 2 – Principi di cybersecurity olistici per un'infrastruttura fintech Il secondo capitolo fornisce le basi teoriche della sicurezza informatica, dalla triade CIA ai principi di difesa in profondità, contestualizzandoli nell'ambito fintech.

Capitolo 3 – Principi dell'Infrastruttura Cloud e Scelta di AWS Il terzo capitolo analizza i paradigmi del cloud computing e presenta AWS come piattaforma di riferimento, descrivendone architettura e caratteristiche di sicurezza.

Capitolo 4 – Progettazione e Implementazione Avanzata della Sicurezza delle Identità e degli Accessi (IAM) in AWS Questo capitolo si concentra sull'implementazione pratica di IAM, analizzando strategie avanzate per la gestione delle identità e l'applicazione del principio del minimo privilegio.

Capitolo 5 – Architettura di Rete Sicura e Protezione dei Servizi Applicativi su AWS per fintech Il quinto capitolo descrive la progettazione di un'architettura di rete sicura mediante VPC, gruppi di sicurezza e controlli di accesso avanzati.

Capitolo 6 – Implementazione di un Honeypot in un'Infrastruttura AWS per Startup fintech L'ultimo capitolo tecnico presenta l'implementazione di un sistema honeypot su AWS per il rilevamento proattivo delle minacce, includendo analisi dei costi e test di efficacia.

Capitolo 7 – Conclusioni Il capitolo finale riassume i risultati raggiunti e presenta considerazioni sulle prospettive future per la sicurezza delle infrastrutture cloud in ambito fintech.

Dedica

*dedicato a chi mi vuole bene, a chi mi stima e ai miei
compagni di viaggio, vi voglio bene*

Ringraziamenti

Vorrei ringraziare soprattutto ...

Indice

Prefazione	1
Dedica	3
Ringraziamenti	4
1 Introduzione	1
1.1 La Cybersecurity nelle Startup fintech: Sfide, Vulnerabilità e Strategie di Protezione in un Ecosistema in Rapida Evoluzione	1
1.1.1 Definizione di fintech	1
1.1.2 Il Contesto delle Startup fintech: Un Ecosistema Dinamico e Sfidante	4
1.1.3 Banche Tradizionali vs. Startup fintech: Divergenze Strategiche in Tecnologia, Regolamentazione e Cybersecurity	5
1.1.4 Analisi delle Sfide di Cybersecurity per le Startup fintech	6
1.1.5 Principali Vettori di Attacco e Minacce Informatiche nel Contesto fintech	8
1.1.6 Conseguenze degli Attacchi e Impatto sulle Startup fintech	11
1.1.7 Importanza di un Approccio Proattivo alla Cybersecurity	11
1.1.8 Approccio Metodologico	12
2 Principi di cybersecurity olistici per un'infrastruttura fintech	13
2.1 Introduzione	13
2.2 Triade CIA: Il Nucleo della Sicurezza delle Informazioni	14
2.3 Difesa in Profondità (Defense in Depth)	16
2.4 Principio del Minimo Privilegio (Principle of Least Privilege - PoLP)	16
2.5 Separazione dei Compiti (Separation of Duties - SoD)	17
2.6 Zero Trust Architecture (ZTA)	17
2.7 Economia del Meccanismo	18
2.8 Impostazioni Sicure per Difetto (Fail-Safe Defaults)	18
2.9 Mediazione Completa	19

2.10	Resilienza Cibernetica (Cyber Resiliency)	20
2.11	Responsabilizzazione e Non-Ripudio (Accountability and Non-Repudiation)	20
2.12	Privacy by Design (PbD) e Privacy by Default	21
2.12.1	Conclusioni Preliminari del Capitolo	21
3	Principi dell'Infrastruttura Cloud e Scelta di AWS	23
3.1	Fondamenti di Cloud Computing	23
3.1.1	Modelli di servizio e distribuzione cloud	24
3.1.2	Cloud Computing vs Infrastrutture On-Premises	27
3.1.3	Perché le Startup Scelgono il Cloud	28
3.1.4	Introduzione ad Amazon Web Services (AWS)	30
3.1.5	Il Caso Specifico: AWS per la Startup fintech	31
3.1.6	Infrastruttura Globale AWS: Fondamenta per la fintech	33
3.1.7	Architettura Virtualizzata e Meccanismi di Scalabilità per la fintech	34
3.1.8	Modello di Responsabilità Condivisa: Implicazioni per la fintech	37
4	Progettazione e Implementazione Avanzata della Sicurezza delle Identità e degli Accessi (IAM) in AWS	39
4.1	Introduzione alla Gestione delle Identità e degli Accessi	39
4.1.1	Configurazione dell'infrastruttura AWS per la piattaforma <i>Finanz</i>	39
4.1.2	Implementazione del Modello Zero Trust e del Principio del Minimo Privilegio	41
4.1.3	Gestione delle Identità e degli Accessi (IAM) come Pilastro di Zero Trust in AWS	44
4.1.4	Valutazione dell'implementazione IAM corrente di <i>Finanz</i>	45
4.2	Implementazione delle Migliorie Proposte alla Gestione IAM	46
4.2.1	Ristrutturazione della Gerarchia degli Accessi	47
4.2.2	Sviluppo di un Modello Ibrido Aggiornato per la Gestione degli Accessi	50
4.2.3	Introduzione di un Break-Glass Account	57
4.2.4	Implementazione di un Workflow di Approvazione a Due Fasi (Opzionale)	60
4.3	Conclusioni sulla Sicurezza IAM	61
5	Architettura di Rete Sicura e Protezione dei Servizi Applicativi su AWS per Finanz	63
5.1	Introduzione alla Sicurezza dell'Infrastruttura	63
5.2	Progettazione di una Rete Sicura con Amazon VPC	63

5.2.1	Subnet Pubbliche e Private: Segmentazione Essenziale	64
5.2.2	Controllo Granulare del Traffico: Gruppi di Sicurezza e Network ACL	64
5.2.3	NAT Gateway per l'Accesso Controllato a Internet	66
5.2.4	VPC Endpoints per Comunicazioni Private con Servizi AWS	66
5.2.5	Connessioni Sicure verso Ambienti Esterni (Opzionale: VPN/Direct Connect)	67
5.3	Gestione Sicura delle Istanze EC2	67
5.3.1	Scelta delle AMI, Patching e Hardening del Sistema Operativo	68
5.3.2	Utilizzo Fondamentale di IAM Roles per le Istanze EC2 . . .	75
5.3.3	Scalabilità e Disponibilità con Auto Scaling Groups	75
5.4	Protezione dei Dati Sensibili: Un Imperativo per le fintech	76
5.4.1	Crittografia dei Dati: a Riposo (At Rest) e in Transito (In Transit)	76
5.4.2	Gestione Centralizzata delle Chiavi Crittografiche con AWS KMS	77
5.4.3	Strategie di Backup e Disaster Recovery (DR)	78
5.4.4	Misure di Sicurezza Specifiche per i Bucket S3	79
5.5	Monitoraggio Continuo, Logging e Alerting: Vedere per Proteggere	80
5.5.1	Abilitazione e Configurazione di AWS CloudTrail e Amazon CloudWatch	80
5.5.2	Configurazione di Allarmi CloudWatch Proattivi	81
5.5.3	Utilizzo di Servizi di Sicurezza Gestiti: AWS Security Hub e Amazon GuardDuty	82
5.6	Automazione e Coerenza con Infrastructure as Code (IaC)	83
5.7	Conclusioni sulla Sicurezza dell'Infrastruttura e dei Servizi	85
6	Implementazione di un Honeypot in un'Infrastruttura AWS per Startup fintech	86
6.1	Definizione e Utilità di un Honeypot	86
6.1.1	Che cos'è un Honeypot	86
6.1.2	Utilità nel Contesto di una Startup fintech	87
6.2	Tipologie di Honeypot e Selezione Progettuale	88
6.3	Valutazione Strategica degli Honeypot nel Contesto della Cybersecurity Contemporanea	90
6.4	Selezione della Piattaforma Honeypot: T-Pot CE 22.04	92
6.5	Implementazione Sicura dell'Architettura su AWS	93
6.5.1	Architettura di Rete Isolata	93
6.5.2	Configurazione dei Controlli di Sicurezza di Rete	94
6.5.3	Provisioning Automatizzato e Sicuro dell'Istanza EC2	94

6.5.4	Sviluppo e Integrazione di un Sensore Fintech Personalizzato	95
6.6	Analisi dei Costi e Valutazione del Valore Strategico	97
6.6.1	Scomposizione Analitica dei Costi (TCO)	97
6.6.2	Impatto dei Programmi di Incentivazione AWS per Startup .	99
6.6.3	Valutazione del Ritorno sull'Investimento (ROI) Strategico .	99
6.7	Validazione Empirica e Analisi dei Risultati	99
6.7.1	Metodologia della Simulazione di Attacco Multi-Fase	100
6.7.2	Correlazione e Analisi Approfondita degli Eventi Rilevati . .	101
6.8	Conclusioni e Sviluppi Futuri	102
7	Conclusioni	104

Elenco delle figure

1	Categorie di Servizi fintech	2
2	Principali vettori di attacco e minacce informatiche nel contesto fintech	9
3	Modelli di Servizio Cloud	25
4	Modello di Responsabilità Condivisa di AWS	37
5	Diagramma semplificato dell'architettura AWS di <i>Finanz.</i>	42

Capitolo 1

Introduzione

1.1 La Cybersecurity nelle Startup fintech: Sfide, Vulnerabilità e Strategie di Protezione in un Ecosistema in Rapida Evoluzione

Il settore fintech rappresenta oggi una delle aree più dinamiche e innovative dell'ecosistema startup, con investimenti globali che hanno raggiunto i 115 miliardi di dollari, in crescita esponenziale rispetto ai 53.2 miliardi del 2018 [1]. Questo rapido sviluppo, caratterizzato dall'implementazione di tecnologie emergenti per i servizi finanziari, porta con sé non solo opportunità senza precedenti, ma anche significative sfide in termini di sicurezza informatica. Le startup fintech, che si trovano all'intersezione tra finanza tradizionale e innovazione tecnologica, gestiscono dati estremamente sensibili, diventando bersagli privilegiati per i cybercriminali. La presente tesi esplora le vulnerabilità specifiche di queste realtà, analizza le principali minacce che esse affrontano e propone strategie di sicurezza efficaci anche in contesti di risorse limitate. Si evidenzia come un approccio proattivo alla cybersecurity non rappresenti un costo, bensì un investimento strategico fondamentale per il successo a lungo termine di una piccola-media impresa nel campo del fintech.

1.1.1 Definizione di fintech

Il termine *fintech* (acronimo di “financial technology”) identifica, nell'ecosistema economico-finanziario contemporaneo, l'intersezione tra servizi finanziari e tecnologie digitali avanzate. Esso è caratterizzato dall'implementazione di soluzioni tecnologiche innovative per la trasformazione digitale del settore bancario e finanziario [2]. Una *startup fintech* si configura come un'entità imprenditoriale emergente che opera nell'ambito della tecnologia finanziaria, sviluppando modelli di

business innovativi basati su architetture tecnologiche all'avanguardia e paradigmi operativi che sfidano le metodologie consolidate delle aziende o delle organizzazioni nel settore finanziario tradizionale [3].

Queste organizzazioni, orientate alla tecnologia, si caratterizzano per l'adozione di approcci centrati sui dati e sul cliente (*data-centric* e *customer-centric*), finalizzati all'ottimizzazione dell'accessibilità, dell'efficienza operativa e della qualità dell'esperienza utente nei servizi finanziari. Esse assumono un ruolo strategico di primo piano nel processo di trasformazione digitale dell'ecosistema finanziario nazionale, contribuendo significativamente alla modernizzazione delle infrastrutture di pagamento e dei servizi bancari digitali [2].

Il panorama dei servizi finanziari sta subendo una profonda trasformazione, guidata dall'emergere delle startup fintech (Financial Technology) e Insurtech (Insurance Technology). Queste entità introducono modelli di business e soluzioni tecnologiche che si pongono in diretta competizione, e talvolta in collaborazione, con gli operatori tradizionali del settore. L'offerta tipica di tali startup si articola in diverse aree chiave, ciascuna caratterizzata da un elevato grado di innovazione e digitalizzazione:

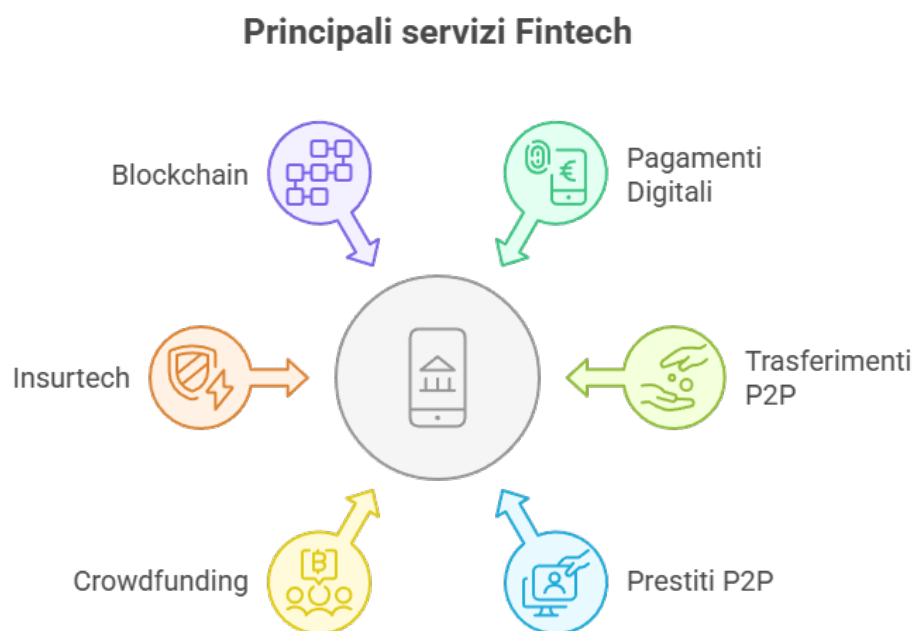


Figura 1: Categorie di Servizi fintech

- **Pagamenti Digitali:** I pagamenti digitali consistono in sistemi che consentono la transazione di denaro in formato elettronico, frequentemente mediante applicazioni mobili, portafogli digitali e tecnologie contactless. Tali soluzioni mirano a migliorare l'efficienza dell'esperienza utente, ridurre i costi di transazione e velocizzare i tempi di elaborazione. Dal punto di vista della sicurezza informatica, l'infrastruttura deve prevedere meccanismi robusti di autenticazione degli utenti, crittografia end-to-end e sistemi antifrode intelligenti, per garantire la protezione dei dati e delle transazioni [4].
- **Trasferimenti di Denaro Peer-to-Peer (P2P):** I sistemi *Peer-to-Peer (P2P)*, che permettono il trasferimento diretto di fondi tra utenti senza l'intermediazione di istituzioni finanziarie tradizionali, consentono la riduzione dei costi e dei tempi di trasferimento. Le piattaforme devono tuttavia implementare processi rigorosi di verifica dell'identità (*Know Your Customer*, KYC) e controllo antiriciclaggio (*Anti-Money Laundering*, AML), oltre a misure per la protezione dei dati personali e per la prevenzione di frodi e attività illecite [4].
- **Prestiti Diretti tra Privati (P2P Lending):** Questo modello di credito consente l'incontro diretto tra richiedenti e finanziatori, riducendo il ruolo delle banche. Le piattaforme si avvalgono di algoritmi di scoring basati su *machine learning* e analisi dei *big data* per valutare il merito creditizio. Le criticità principali riguardano la trasparenza algoritmica, la protezione dei dati finanziari sensibili e la gestione del rischio di insolvenza [5].
- **Finanziamento Partecipativo (Crowdfunding):** Le piattaforme di *crowdfunding* facilitano la raccolta di fondi da un'ampia base di investitori per progetti imprenditoriali, culturali o sociali. Le principali tipologie includono equity-based, lending-based, reward-based e donation-based. Le sfide di sicurezza si concentrano sulla prevenzione delle frodi, sulla protezione degli investitori e sulla trasparenza delle informazioni fornite dai promotori [6].
- **Servizi Assicurativi Innovativi (Insurtech):** L'insurtech introduce tecnologie come l'intelligenza artificiale, l'*Internet of Things (IoT)* e la *big data analytics* nel settore assicurativo, allo scopo di personalizzare i prodotti, migliorare la gestione dei sinistri e ottimizzare la valutazione del rischio. La protezione dei dati sensibili (ad esempio, dati sanitari o comportamentali) è una priorità, richiedendo tecniche avanzate di *data governance* e sicurezza informatica [7].
- **Tecnologie Blockchain e Criptovalute:** L'impiego delle *Distributed Ledger Technologies (DLT)*, come la blockchain, ha rivoluzionato i servizi finanziari

attraverso la decentralizzazione, la tokenizzazione di asset e la nascita della finanza decentralizzata (*Decentralized Finance*, DeFi). Le sfide principali riguardano la sicurezza degli *smart contract*, la custodia delle chiavi private, la prevenzione di attacchi come il 51% attack e la conformità a regolamenti AML e di contrasto al finanziamento del terrorismo (*Combating the Financing of Terrorism*, CFT) [8].

A conferma della crescente rilevanza e della dinamicità di questo settore a livello globale, anche il panorama italiano evidenzia una significativa espansione. Nel 2023, si registravano oltre 600 startup attive nei segmenti fintech e Insurtech [3]. Questo dato sottolinea non solo la vitalità dell’ecosistema innovativo nazionale, ma anche la progressiva adozione di queste nuove soluzioni da parte del mercato. Ciò implica una crescente superficie di attacco potenziale e la necessità di strategie di cybersecurity adeguate a proteggere sia le infrastrutture di questi nuovi attori sia i dati dei loro utenti.

1.1.2 Il Contesto delle Startup fintech: Un Ecosistema Dinamico e Sfidante

Le startup fintech operano in un ambiente caratterizzato da elevata incertezza, risorse limitate e necessità di crescita rapida, fattori che influenzano profondamente le decisioni in ambito IT e sicurezza informatica [9]. A differenza delle istituzioni finanziarie tradizionali, queste realtà innovative non dispongono generalmente di strutture gerarchiche complesse o budget consistenti dedicati alla sicurezza, dovendo invece adottare approcci agili e flessibili.

Il contesto finanziario in cui operano le startup fintech impone pressioni significative sulle decisioni di spesa. Ogni investimento, compreso quello per l’infrastruttura IT e la sicurezza, deve essere attentamente valutato in termini di ritorno immediato e benefici a lungo termine [9]. Questa ottimizzazione dei costi rappresenta una sfida continua, poiché la sicurezza informatica richiede investimenti costanti, spesso non producendo risultati immediatamente visibili; la sua assenza, tuttavia, può comportare conseguenze catastrofiche. In questo equilibrio delicato, le startup fintech devono trovare il giusto compromesso tra la necessità di scalare rapidamente e l’implementazione di solide misure di protezione.

1.1.3 Banche Tradizionali vs. Startup fintech: Divergenze Strategiche in Tecnologia, Regolamentazione e Cybersecurity

Il panorama dei servizi finanziari è caratterizzato da una marcata eterogeneità tra gli operatori tradizionali e le nuove realtà fintech. Queste differenze si manifestano in modo sostanziale negli approcci tecnologici adottati, nei quadri normativi di riferimento e, di conseguenza, nelle strategie implementate per la cybersecurity.

Un aspetto cruciale di distinzione risiede proprio nell'approccio alla cybersecurity. Le istituzioni bancarie tradizionali operano in un contesto altamente strutturato e rigorosamente regolamentato, con precisi obblighi legali in materia di sicurezza e protezione dei dati. Consapevoli che anche il minimo incidente può comportare la perdita di migliaia di clienti e sanzioni finanziarie significative, esse investono ingenti risorse nel testare costantemente le proprie misure di sicurezza. Al contrario, le startup fintech hanno storicamente beneficiato di una maggiore flessibilità normativa [10]. Spesso nate come realtà agili e in rapida espansione, talvolta fungono da "overlay" per le banche, facilitando la fornitura di servizi finanziari innovativi pur operando, almeno inizialmente, con regolamentazioni meno stringenti. Questa disparità normativa sta tuttavia diminuendo, specialmente per quelle fintech che evolvono verso modelli bancari più completi, assoggettandosi così a un crescente scrutinio regolamentare. La sfida per queste startup consiste quindi nel bilanciare l'agilità operativa con l'adozione di standard di sicurezza elevati, anticipando l'evoluzione normativa del settore.

Queste divergenze in ambito cybersecurity sono profondamente radicate in più ampi e distinti approcci alla tecnologia e al contesto regolamentare, che riflettono modelli organizzativi e strategie di innovazione differenti. Le banche tradizionali operano tipicamente con infrastrutture IT complesse e stratificate nel tempo, risultando spesso meno agili e più costose da aggiornare rispetto alle soluzioni moderne. La loro evoluzione tecnologica è stata prevalentemente incrementale, focalizzata sull'automazione dei processi esistenti per migliorare l'efficienza, piuttosto che su innovazioni disruptive. Storicamente, esse hanno perseguito un approccio olistico, integrando internamente un'ampia gamma di servizi, sebbene si osservi una crescente tendenza all'outsourcing e alla specializzazione. I costi IT rappresentano una quota significativa dei loro costi totali (15-20%), a causa della complessità delle infrastrutture legacy. Sul piano regolamentare, sono soggette a un quadro normativo estremamente rigido, intensificatosi dopo la crisi finanziaria del 2008, che comporta elevati costi di compliance e ha storicamente costituito una barriera all'entrata per nuovi competitor. Le startup fintech, d'altra parte, si contraddistinguono per l'adozione di tecnologie moderne quali big data, cloud computing e

blockchain, utilizzate per creare prodotti, servizi e modelli di business innovativi, come piattaforme di crowdfunding o assicurazioni peer-to-peer. La loro innovazione non si limita a miglioramenti incrementali, ma spesso mira a ridefinire radicalmente la catena del valore esistente, come nel caso dei pagamenti peer-to-peer tramite blockchain. Molte fintech si specializzano in nicchie specifiche, bypassando attori tradizionali e promuovendo la disintermediazione; raramente offrono un ventaglio completo di servizi bancari. La tecnologia è anche un driver per modelli di business innovativi, come i robo-advisors, e per l'ingresso di nuovi player nel mercato, come le grandi aziende tecnologiche. Sul fronte regolamentare, esse possono beneficiare di iniziative volte a ridurre le barriere all'entrata, quali i cosiddetti "*fintech sandbox*", che permettono di testare soluzioni in ambienti controllati. Il quadro normativo per le fintech è ancora in evoluzione, creando incertezze ma anche opportunità, con l'emergere di soluzioni "*RegTech*" (Regulatory Technology) per semplificare la compliance. Questo contesto, tendenzialmente più flessibile, consente alle fintech di esercitare una significativa pressione competitiva sulle banche tradizionali [11].

1.1.4 Analisi delle Sfide di Cybersecurity per le Startup fintech

L'ecosistema fintech, pur essendo un catalizzatore di innovazione nel settore finanziario, presenta per le startup che ne fanno parte un panorama di sfide di cybersecurity intrinsecamente complesso. La natura agile, la focalizzazione sulla rapida immissione sul mercato (*time-to-market*) e le risorse spesso limitate di queste nuove realtà imprenditoriali possono portare a una sottovalutazione della postura di sicurezza, rendendole bersagli appetibili per attori malevoli. A differenza delle istituzioni finanziarie consolidate, che dispongono di budget e team dedicati alla mitigazione dei rischi informatici, le startup fintech devono bilanciare l'innovazione con la necessità imprescindibile di proteggere dati sensibili e infrastrutture critiche.

Le principali vulnerabilità e sfide operative per le startup fintech in ambito cybersecurity possono essere così sintetizzate:

- **Allocazione Iniziale delle Risorse e Priorità Strategiche:**

- Nelle fasi embrionali, la priorità delle startup fintech è spesso concentrata sullo sviluppo del prodotto, sull'acquisizione di utenti e sulla generazione di entrate.
- Questa focalizzazione può marginalizzare l'integrazione di solide pratiche di cybersecurity sin dalle prime fasi di progettazione, un approccio noto come *security by design*.

- L'assenza di figure dedicate come il *Chief Information Security Officer (CISO)* e un approccio reattivo – attendere una violazione prima di intervenire – espone l'organizzazione a rischi significativi che potrebbero comprometterne la continuità operativa e la reputazione [12].
- La pressione competitiva per innovare rapidamente, superando gli operatori tradizionali, può incentivare cicli di sviluppo accelerati a scapito della robustezza della sicurezza dei sistemi e delle tecnologie sottostanti [13].

- **Vincoli di Budget e Competenze Specialistiche:**

- Le risorse finanziarie limitate costituiscono una barriera significativa all'adozione di soluzioni di cybersecurity avanzate.
- La capacità di attrarre e trattenere talenti specializzati in sicurezza informatica è limitata.
- Questo divario rispetto alle grandi istituzioni finanziarie, capaci di investimenti ingenti, si traduce in una potenziale disparità nella capacità di difesa contro minacce sofisticate.

- **Gestione di Dati Finanziari Altamente Sensibili:**

- Le startup fintech trattano quotidianamente un volume considerevole di dati finanziari sensibili, inclusi dettagli di conti bancari, informazioni di identificazione personale (*Personally Identifiable Information*, PII) e cronologie delle transazioni.
- La compromissione di tali dati non solo comporta severe sanzioni normative e perdite finanziarie dirette, ma erode irrimediabilmente la fiducia dei clienti, elemento vitale per la sostenibilità del business.

- **Superficie di Attacco Derivante da Tecnologie Emergenti e Interconnessioni:**

- L'adozione spinta di tecnologie innovative (cloud computing, API aperte, intelligenza artificiale, blockchain) è un tratto distintivo delle fintech.
- Sebbene queste tecnologie offrano vantaggi competitivi, esse introducono anche nuove superfici di attacco e vettori di minaccia [9].
- La dipendenza da servizi cloud, pur garantendo scalabilità e accessibilità, può amplificare i rischi se le configurazioni non sono gestite secondo i principi di minima esposizione e sicurezza.

- L’interconnessione con sistemi di terze parti (fornitori, partner) espande la superficie di attacco, rendendo cruciale una rigorosa gestione del rischio della catena di approvvigionamento (*supply chain risk management*).
- **Complessità del Panorama Normativo e degli Standard di Sicurezza:**
 - Gli standard di cybersecurity esistenti sono spesso stati concepiti per istituzioni finanziarie tradizionali o contesti tecnologici più generalisti.
 - Le specificità delle startup fintech – agilità, uso intensivo di cloud e API, modelli decentralizzati – richiedono un adattamento e un’interpretazione sartoriale di tali framework.
 - Questo aggiunge un ulteriore livello di complessità operativa e di compliance.
- **Minacce Interne (Insider Threats):**
 - Particolarmente nelle fasi iniziali, caratterizzate da controlli interni meno formalizzati e da una cultura organizzativa più aperta, il rischio associato a dipendenti negligenti, o in rari casi malintenzionati, non può essere trascurato.
 - La mancanza di segregazione dei compiti (*Segregation of Duties*) e di meccanismi di monitoraggio granulare degli accessi può facilitare incidenti di sicurezza originati dall’interno.

Queste sfide strutturali e operative aumentano la vulnerabilità delle startup fintech a una varietà di attacchi informatici, rendendo essenziale l’adozione di misure di sicurezza proattive e mirate per mitigare tali rischi.

1.1.5 Principali Vettori di Attacco e Minacce Informatiche nel Contesto fintech

Il settore fintech, data la sua natura digitale e la preziosità dei dati trattati, è un obiettivo primario per una vasta gamma di cyberattacchi [14]. Tra le minacce più rilevanti per le startup fintech si annoverano:

- **Attacchi di Social Engineering e Phishing:** Tecniche come il *social engineering* (ingegneria sociale) e il *phishing* mirano a manipolare gli utenti o i dipendenti per indurli a rivelare credenziali di accesso, dati sensibili o a eseguire azioni malevole. E-mail, messaggi e siti web fraudolenti, che replicano comunicazioni legittime, sono strumenti comuni per compromettere la sicurezza a livello umano [14].

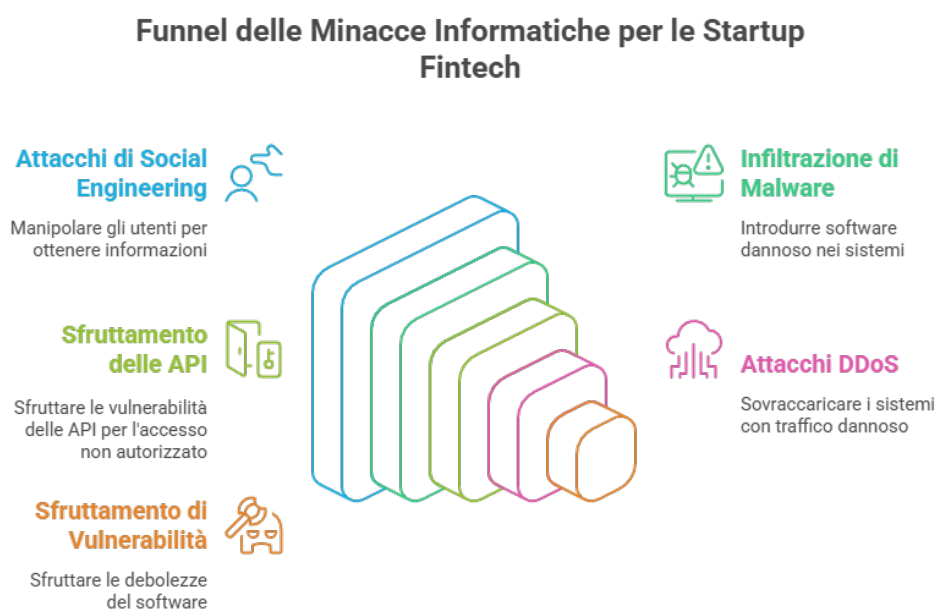


Figura 2: Principali vettori di attacco e minacce informatiche nel contesto fintech

- **Malware e Ransomware:** L'infiltrazione di software malevolo (*malware*), inclusi i *ransomware* che cifrano i dati rendendoli inaccessibili e richiedono un riscatto, rappresenta una minaccia critica. Per una startup, le cui risorse finanziarie sono limitate, l'impatto di un attacco ransomware – in termini di interruzione operativa, costi di ripristino e potenziale pagamento del riscatto – può essere particolarmente devastante [14].
- **Vulnerabilità delle API e Configurazioni Cloud Errate:** Le *Application Programming Interfaces (API)* sono fondamentali per l'ecosistema fintech, abilitando l'integrazione tra servizi diversi. Tuttavia, API insicure o mal configurate possono diventare gateway per accessi non autorizzati a dati e funzionalità critiche [9]. Analogamente, errori nella configurazione dei servizi cloud (*cloud misconfigurations*) possono esporre involontariamente dati sensibili o creare falle di sicurezza sfruttabili.
- **Attacchi Distributed Denial-of-Service (DDoS):** Questi attacchi, noti come *Distributed Denial-of-Service (DDoS)*, mirano a rendere i servizi online inaccessibili sovraccaricando l'infrastruttura target con un volume elevato di traffico malevolo. Sebbene possano apparire meno sofisticati di altri, gli attacchi DDoS possono causare significative interruzioni di servizio, danni reputazionali e perdite economiche; talvolta fungono da diversivo per mascherare intrusioni più mirate [9].
- **Sfruttamento di Vulnerabilità Software e Zero-Day:** Le startup, per la rapidità di sviluppo, potrebbero non avere processi di *patch management* (gestione delle patch) e *vulnerability assessment* (valutazione delle vulnerabilità) altrettanto maturi quanto aziende più strutturate. Questo le espone a rischi derivanti da vulnerabilità note o, nel peggiore dei casi, da quelle non ancora divulgate (definite *zero-day*).

La comprensione approfondita di queste sfide e minacce è il primo passo fondamentale per le startup fintech, al fine di poter sviluppare strategie di cybersecurity proattive ed efficaci, integrando la sicurezza come componente essenziale del proprio modello di business sin dalla sua concezione. Le conseguenze di una violazione, che spaziano da perdite finanziarie dirette a danni reputazionali, impatti normativi e perdita di fiducia da parte degli stakeholder, possono compromettere la sopravvivenza stessa della startup.

1.1.6 Conseguenze degli Attacchi e Impatto sulle Startup fintech

L'impatto di un attacco informatico su una startup fintech può essere multidimensionale e, in molti casi, esistenziale. A livello finanziario, oltre ai costi diretti per il ripristino dei sistemi e la gestione dell'incidente, vanno considerati i potenziali risarcimenti a clienti danneggiati, le sanzioni normative e l'aumento dei premi assicurativi [9]. Tuttavia, è forse l'impatto reputazionale a rappresentare la minaccia più grave: in un settore basato sulla fiducia come quello finanziario, una violazione dei dati può comprometterne irreparabilmente l'immagine, portando alla perdita di clienti attuali e potenziali.

L'interruzione operativa conseguente a un attacco può avere effetti a catena, influenzando non solo i clienti diretti ma anche partner commerciali e fornitori [9]. In un ecosistema interconnesso come quello fintech, l'interdipendenza tra diverse piattaforme e servizi amplifica ulteriormente l'impatto di un incidente di sicurezza, con effetti che possono estendersi ben oltre il perimetro aziendale immediato.

1.1.7 Importanza di un Approccio Proattivo alla Cybersecurity

Implementare una strategia di cybersecurity solida sin dalle prime fasi di sviluppo di una startup fintech non configura un semplice onere, bensì un investimento strategico di primaria importanza [9]. L'adozione del paradigma "security by design" permette infatti di integrare la sicurezza in maniera organica nei processi aziendali e nel ciclo di sviluppo del prodotto, contribuendo alla significativa riduzione dei costi a lungo termine e alla minimizzazione dei rischi potenziali. Al contrario, la mancata attenzione alla sicurezza nelle fasi iniziali comporta l'accumulo di un cosiddetto *security debt*, ovvero un debito tecnico in ambito sicurezza che, analogamente a un mutuo con tassi elevati, diventa progressivamente più oneroso da gestire e da ripagare nel tempo. Infine, la pressione derivante dalla necessità di accelerare lo sviluppo e di raggiungere rapidamente il mercato può portare a trascurare aspetti fondamentali della sicurezza, esacerbando ulteriormente tale debito tecnico.

Un approccio preventivo alla sicurezza risulta sempre più efficace ed economico rispetto a uno reattivo [9]. I costi per implementare misure di sicurezza di base sono generalmente inferiori rispetto a quelli necessari per rispondere a un incidente, che possono includere non solo il ripristino dei sistemi ma anche sanzioni, risarcimenti e danni reputazionali. La cybersecurity deve quindi essere considerata come parte integrante della strategia aziendale, non come un elemento accessorio o un costo da minimizzare.

Le startup fintech devono inoltre considerare che adeguati livelli di sicurezza rappresentano spesso un requisito fondamentale per attrarre investitori e partner commerciali [9]. Durante le fasi di *due diligence*, l'analisi delle misure di sicurezza implementate è diventata una componente standard, e lacune significative in questo ambito possono compromettere opportunità di finanziamento o collaborazioni strategiche.

1.1.8 Approccio Metodologico

Il presente studio si prefigge di analizzare le sfide della cybersecurity nelle startup fintech mediante un approccio metodologico che coniuga rigore strutturale e flessibilità applicativa [9]. Nonostante la focalizzazione su un caso di studio specifico, l'intento è distillare principi e *best practice* di sicurezza trasferibili al più ampio contesto delle startup fintech, a prescindere dalla piattaforma tecnologica da esse adottata. L'approccio metodologico proposto tiene conto delle intrinseche limitazioni di risorse che caratterizzano le startup, offrendo soluzioni scalabili e capaci di evolvere in parallelo alla crescita organizzativa.

La metodologia si articola su tre direttrici fondamentali: l'identificazione delle minacce peculiari al modello di business fintech, la prioritizzazione strategica degli interventi basata sulla valutazione del rapporto rischio/beneficio, e l'implementazione di controlli di sicurezza essenziali e al contempo efficaci [9]. Tale impostazione pragmatica è volta a conseguire un livello di protezione robusto anche in contesti di risorse contenute, concentrando gli sforzi sulle aree di maggiore criticità.

Capitolo 2

Principi di cybersecurity olistici per un'infrastruttura fintech

2.1 Introduzione

Per una startup fintech, la definizione dell'architettura infrastrutturale e del profilo di sicurezza rappresenta una sfida intrinsecamente complessa. Si tratta, infatti, di conciliare la dinamicità imposta dal mercato con l'inderogabile esigenza di tutelare dati finanziari di elevata sensibilità e di conformarsi a un quadro regolatorio in costante mutamento. Ancor prima di esaminare le concrete implementazioni di sicurezza, risulta fondamentale tracciare i pilastri concettuali della cybersecurity. Tali pilastri devono informare ogni scelta progettuale e operativa, con particolare attenzione alle sfide specifiche che le startup fintech, per loro stessa natura, si trovano ad affrontare. Il presente capitolo si propone di analizzare tali principi basilari, mettendo in luce come le specificità operative delle startup – quali risorse contenute, team con competenze trasversali, la spinta verso un rapido ingresso sul mercato e l'esigenza di scalabilità – si intreccino, e non di rado entrino in tensione, con l'applicazione meticolosa di queste pratiche ottimali. La salvaguardia dei dati clientelari, delle operazioni finanziarie e del patrimonio intellettuale, in un contesto operativo ad elevata esposizione, esige una strategia di sicurezza capace di essere al contempo solida, flessibile e concepita nativamente come parte integrante del sistema (Security by Design).

2.2 Triade CIA: Il Nucleo della Sicurezza delle Informazioni

La Triade CIA – Confidentiality (Riservatezza), Integrity (Integrità) e Availability (Disponibilità) – costituisce il framework concettuale riconosciuto a livello globale per la sicurezza delle informazioni [15]. Questi tre pilastri sono interdipendenti e fondamentali per la progettazione di qualsiasi strategia di sicurezza informatica resiliente, specialmente in contesti ad alta sensibilità come quello fintech.

- **Confidentiality (Riservatezza):** Assicura che le informazioni siano accessibili solo a entità autorizzate, proteggendo la privacy individuale e i dati proprietari da divulgazioni indebite [15]. Meccanismi tipici includono la crittografia end-to-end (E2EE) per dati in transito e at-rest, controlli di accesso basati su ruoli (RBAC), autenticazione multi-fattore (MFA) e una gestione rigorosa delle chiavi crittografiche. Nel contesto di una startup fintech, la riservatezza è messa a dura prova. L'accesso a dati finanziari sensibili (ad esempio, Primary Account Number (PAN), Card Verification Value (CVV), dettagli di conti bancari, Personally Identifiable Information (PII)) deve essere blindato. Tuttavia, la fluidità dei ruoli in team snelli spesso conduce a una condivisione pragmatica delle credenziali o all'assegnazione di privilegi sovradimensionati per accelerare i cicli di sviluppo e deployment. La gestione delle chiavi crittografiche, un compito altamente specializzato, può diventare un'area grigia: l'adozione di servizi gestiti (Key Management Service, KMS) offerti dai cloud provider può mitigare parte della complessità, ma richiede comunque una corretta configurazione e una chiara attribuzione di responsabilità per prevenire accessi non autorizzati o perdite di chiavi.
- **Integrity (Integrità):** Garantisce che le informazioni siano protette da modifiche o distruzioni non autorizzate o accidentali, assicurando l'autenticità e la non ripudiabilità dei dati [15]. L'integrità è vitale per la fiducia nelle transazioni finanziarie e nei saldi dei conti. Le tecniche includono:
 - Funzioni di hash crittografiche (ad esempio, SHA-256, SHA-3) per la verifica dell'immutabilità.
 - Firme digitali (basate su Public Key Infrastructure, PKI) per l'autenticazione dell'origine e l'integrità dei messaggi.
 - Sistemi di controllo versione (ad esempio, Git con commit firmati) per codice e configurazioni.
 - Checksum e meccanismi di error detection/correction in storage e trasmissione.

- Log di audit immutabili.

Per una startup fintech, compromettere l'integrità dei dati transazionali o dei saldi dei clienti ha conseguenze catastrofiche, erodendo istantaneamente la fiducia e la brand reputation, oltre a potenziali implicazioni legali e sanzionatorie. La pressione per rilasci software frequenti, attraverso metodologie di Continuous Integration/Continuous Deployment (CI/CD), può portare a cicli di testing affrettati, aumentando il rischio di bug che potrebbero corrompere i dati. Inoltre, l'integrazione con numerose Application Programming Interface (API) di terze parti (ad esempio, per Know Your Customer/Anti-Money Laundering (KYC/AML), open banking, payment gateway) introduce dipendenze la cui affidabilità in termini di integrità dei dati scambiati deve essere costantemente monitorata.

- **Availability (Disponibilità):** Assicura che i sistemi e i dati siano accessibili e utilizzabili su richiesta da parte di un utente autorizzato, in modo tempestivo e affidabile [15]. Interruzioni di servizio, dovute a guasti, errori di configurazione o attacchi (ad esempio, Distributed Denial of Service (DDoS), ransomware), possono paralizzare le operazioni di una fintech, con perdite finanziarie dirette e un danno reputazionale ingente. Le strategie per l'alta disponibilità includono:
 - Architetture ridondanti (N+1, N+M) per server, reti, storage e power.
 - Piani di Disaster Recovery (DR) e Business Continuity (BCP) testati regolarmente.
 - Load balancing e auto-scaling per gestire picchi di traffico.
 - Protezione anti-DDoS a livello di rete e applicativo.
 - Backup frequenti, possibilmente immutabili e off-site.

Le startup fintech, specialmente nelle fasi iniziali, spesso operano con infrastrutture ottimizzate per i costi, talvolta sottodimensionate rispetto a una crescita esplosiva degli utenti o a picchi di carico imprevisti. Questa limitata capacità iniziale può rendere i sistemi particolarmente vulnerabili ad attacchi Denial of Service (DoS) o DDoS volumetrici o applicativi, capaci di saturare le risorse e causare downtime prolungati. La dipendenza da singoli cloud provider o regioni specifiche può anche costituire un single point of failure se non mitigata da strategie multi-region o multi-cloud, spesso considerate troppo costose o complesse all'inizio.

2.3 Difesa in Profondità (Defense in Depth)

Il principio di Difesa in Profondità (DiD) postula la necessità di implementare controlli di sicurezza multipli e stratificati, in modo che il fallimento di un singolo controllo non comprometta l'intero sistema [16]. Questo approccio mira a creare barriere successive per rallentare, rilevare e rispondere agli attacchi. La strategia DiD, come delineata anche dal National Institute of Standards and Technology (NIST), si articola in architetture resistenti alla penetrazione, operazioni di limitazione del danno e progettazione per la cyber resilienza [17]. Nelle startup fintech, l'adozione di una DiD completa è spesso ostacolata da vincoli di budget, expertise e dalla velocità richiesta dal business. Ad esempio, mentre una segmentazione rigorosa della rete (con Demilitarized Zone (DMZ), reti interne dedicate per produzione, staging, sviluppo, e micro-segmentazione a livello di workload) è fondamentale, le startup potrebbero optare per architetture di rete "piatte" per semplificare il deployment iniziale e ridurre l'overhead gestionale, rimandando una segmentazione più granulare. Analogamente, l'implementazione di soluzioni come Web Application Firewall (WAF), Intrusion Detection/Prevention Systems (IDS/IPS), e Security Information and Event Management (SIEM) può essere graduale, partendo da soluzioni più basiche o cloud-native, ma lasciando potenziali lacune nella copertura difensiva nelle prime fasi.

2.4 Principio del Minimo Privilegio (Principle of Least Privilege - PoLP)

Il Principle of Least Privilege (PoLP) stabilisce che a ogni utente, processo o sistema dovrebbero essere concessi solo i privilegi strettamente necessari per svolgere le proprie funzioni autorizzate, e solo per il tempo necessario [18]. Questo limita drasticamente il potenziale danno derivante da un account compromesso, un errore umano o un insider malevolo. Nelle startup fintech, la cultura dell' "execution at all costs" e i team "generalisti" possono portare a una diffusa assegnazione di privilegi amministrativi o accessi ampi. Per esempio, è comune che gli sviluppatori abbiano accesso diretto, talvolta con privilegi di scrittura, ai database di produzione per troubleshooting o rapidi fix, bypassando processi di change management più strutturati. Sebbene ciò possa accelerare la risoluzione di problemi urgenti, espone l'organizzazione a rischi significativi: un singolo account sviluppatore compromesso potrebbe portare alla fuoriuscita o alla manipolazione di ingenti quantità di dati finanziari sensibili. L'implementazione di accessi Just-In-Time (JIT) e di un rigoroso RBAC (già definito) richiede un investimento iniziale in termini di progettazione e configurazione che può essere percepito come un rallentamento.

2.5 Separazione dei Compiti (Separation of Duties - SoD)

La Separation of Duties (SoD) prevede la suddivisione di compiti critici e responsabilità tra individui differenti per prevenire frodi, errori e abusi, assicurando che nessuna singola persona abbia il controllo esclusivo su una transazione o processo end-to-end [19]. Ciò include la separazione tra chi sviluppa, chi effettua i test, chi provvede al rilascio e chi gestisce operativamente i sistemi, nonché tra chi amministra i controlli di accesso e chi ne verifica l'audit. Nelle startup fintech, con team spesso composti da pochi individui altamente versatili, la SoD rappresenta una sfida strutturale. Non è raro che un singolo ingegnere DevOps (Development and Operations) sia responsabile dell'intera pipeline CI/CD (già definita), dalla scrittura del codice infrastrutturale (Infrastructure as Code, IaC) al deployment in produzione e al monitoraggio della sicurezza. In uno scenario di sviluppo di una piattaforma di pagamento, la stessa persona potrebbe definire le logiche di transazione e configurare i controlli di accesso ai dati finanziari sottostanti. Questa concentrazione di potere, sebbene efficiente operativamente, elimina i meccanismi di controllo incrociato, aumentando il rischio che errori, vulnerabilità o attività malevole non vengano rilevati tempestivamente.

2.6 Zero Trust Architecture (ZTA)

Il modello Zero Trust (ZT) rovescia il paradigma tradizionale "trust but verify", assumendo che nessuna entità, interna o esterna al perimetro di rete, sia intrinsecamente affidabile. Ogni richiesta di accesso a una risorsa deve essere autenticata, autorizzata e crittografata dinamicamente, basandosi su policy granulari e sul contesto [20]. La Zero Trust Architecture (ZTA) è una risposta strategica alla dissoluzione dei perimetri tradizionali, caratterizzati da risorse dislocate nel cloud, utenti che operano in remoto e una crescente interconnessione. Per una startup fintech, l'adozione completa di un'architettura Zero Trust è un percorso complesso e oneroso. Le fasi iniziali possono vedere una singola figura tecnica con "chiavi del regno" per l'intera infrastruttura cloud, contravvenendo al principio di verifica continua per ogni accesso. Implementare la micro-segmentazione a livello di rete e applicativo, sistemi di Identity and Access Management (IAM) sofisticati con policy dinamiche, e piattaforme di monitoraggio continuo (ad esempio, Extended Detection and Response, XDR) richiede investimenti significativi in tecnologie e competenze specialistiche, spesso al di là della portata immediata di una startup. La mancanza di SoD (già definita) esacerba ulteriormente il problema, rendendo difficile l'applicazione di policy di accesso granulari e la loro revisione costante.

2.7 Economia del Meccanismo

Formulato da Saltzer e Schroeder, questo principio di progettazione della sicurezza enfatizza la necessità di mantenere i meccanismi di protezione il più semplici e piccoli possibile [21]. Una minore complessità riduce la superficie d'attacco, facilita l'analisi, l'audit, i test e la manutenzione, minimizzando la probabilità di errori di implementazione o configurazione e l'accumulo di debito tecnico con implicazioni per la sicurezza [22]. Strategie includono:

- Architetture a microservizi ben definite, con confini di responsabilità chiari e API (già definite) sicure, per isolare l'impatto di una compromissione.
- Adozione di IaC (già definita) per una gestione dichiarativa, versionata e auditabile dei controlli di sicurezza.
- Preferenza per librerie e componenti con un focus specifico e una base di codice ridotta e ben testata.

Nelle startup fintech, la spinta verso un Minimum Viable Product (MVP) e successivi rapidi cicli di iterazione può favorire l'integrazione di framework complessi o la creazione di monoliti applicativi che, pur accelerando lo sviluppo iniziale, diventano rapidamente difficili da mantenere e securizzare. La tentazione di aggiungere "solo un'altra feature" senza un adeguato refactoring può portare a un aumento della complessità e, di conseguenza, a una maggiore fragilità della sicurezza. Un investimento precoce nella semplicità architetturale e nella modularità, sebbene possa sembrare un rallentamento, si traduce in una riduzione dei costi di remediation e testing nel lungo periodo.

2.8 Impostazioni Sicure per Difetto (Fail-Safe Defaults)

Questo principio, anch'esso di Saltzer e Schroeder, stabilisce che l'accesso a risorse e funzionalità debba essere negato per default, richiedendo un'autorizzazione esplicita (approccio di tipo allow-list) piuttosto che permettere tutto tranne ciò che è esplicitamente negato (approccio di tipo deny-list) [21]. In assenza di un permesso esplicito, l'azione deve essere bloccata. La pubblicazione NIST SP 800-27rA, pur non usando la stessa terminologia, supporta concetti affini enfatizzando la necessità di configurazioni sicure di base [23]. Esempi applicativi:

- Policy Deny All su firewall di rete e WAF (già definito), con regole granulari per il traffico legittimo.

- Policy IAM cloud (ad esempio, AWS IAM, Azure RBAC, Google Cloud IAM) che partono da uno stato di nessun permesso, aggiungendo solo le autorizzazioni minime necessarie.
- Disabilitazione di funzionalità non essenziali e porte non utilizzate su server e servizi.

Nelle startup fintech, la pressione per la velocità può portare a trascurare questo principio. È frequente che i servizi cloud vengano istanziati con le configurazioni di default del provider, che non sempre sono le più restrittive (ad esempio, bucket Simple Storage Service (S3) accessibili pubblicamente in passato, o ruoli IAM con permessi eccessivi). La mentalità del "facciamolo funzionare, poi lo sistemiamo" spesso relega la revisione e l'hardening delle configurazioni a una fase successiva, che potrebbe non arrivare mai o arrivare troppo tardi, esponendo dati sensibili. L'utilizzo di template IaC pre-configurati con standard di sicurezza restrittivi può mitigare questo rischio, ma richiede una disciplina iniziale.

2.9 Mediazione Completa

Ogni accesso a ogni oggetto protetto (file, record, endpoint API) deve essere validato rispetto ai controlli di autorizzazione ogni singola volta, senza fare affidamento su decisioni di accesso memorizzate o cache che potrebbero essere obsolete o compromesse [21]. Questo previene il bypass dei meccanismi di sicurezza dopo un'autenticazione iniziale. Implementazioni pratiche:

- API Gateway che validano token di autenticazione/autorizzazione (ad esempio, JSON Web Token, JWT) per ogni richiesta.
- Applicazione di Row-Level Security (RLS) e Column-Level Security nei database per garantire che le policy siano applicate direttamente a livello di dato.
- Meccanismi di step-up authentication per operazioni ad alto rischio.

In una piattaforma fintech che gestisce pagamenti o dati di investimento, questo principio è cruciale. Ad esempio, l'utilizzo di token di sessione a breve scadenza (short-lived) e la richiesta di ri-autenticazione forte (MFA, già definita) per operazioni critiche come la modifica dei dettagli del beneficiario, l'iniziazione di trasferimenti sopra una certa soglia, o l'accesso a informazioni particolarmente sensibili, sono applicazioni dirette della mediazione completa. Non ci si può fidare di una sessione utente "attiva" per troppo tempo o per operazioni di impatto elevato.

2.10 Resilienza Cibernetica (Cyber Resiliency)

La resilienza cibernetica è la capacità di un sistema di anticipare, resistere, recuperare e adattarsi a condizioni avverse, stress, attacchi o compromissioni, preservando le funzionalità critiche mission-critical [24]. Le strategie includono ridondanza dinamica, degradazione controllata dei servizi (graceful degradation), archiviazione sicura e immutabile dei log per analisi forense e ripristino, e piani di risposta agli incidenti ben definiti e testati. Per le startup fintech, spesso operanti in ambienti cloud con risorse DevOps (già definite) limitate, la resilienza deve essere progettata fin dall'inizio. Questo implica non solo backup e restore, ma anche piani di BCP e DR (già definiti) che considerino scenari di indisponibilità regionale del cloud provider o attacchi ransomware. L'adozione di pratiche di chaos engineering, seppur in forma semplificata, può aiutare a identificare proattivamente le debolezze del sistema e a validare la robustezza dei meccanismi di recovery.

2.11 Responsabilizzazione e Non-Ripudio (Accountability and Non-Repudiation)

L'accountability richiede che ogni azione significativa sia tracciabile univocamente a un'entità (utente o processo) e che esistano prove verificabili di tali azioni [25]. La non-ripudiabilità garantisce che una parte non possa negare di aver eseguito un'azione o inviato/ricevuto dati (ad esempio, una transazione finanziaria) [26]. Firme digitali, marche temporali qualificate e log di audit dettagliati e immutabili sono strumenti chiave. Esempi:

- Log di audit granulari (ad esempio, chi ha fatto cosa, quando, da dove, su quale risorsa), firmati digitalmente, marcati temporalmente (RFC 3161), e archiviati in sistemi Write Once Read Many (WORM) o con Object Lock.
- Uso di firme digitali per transazioni e comunicazioni critiche.

Per le startup fintech, questi principi sono fondamentali per la conformità normativa (ad esempio, Payment Services Directive 2 (PSD2), che impone Strong Customer Authentication (SCA) e tracciabilità delle transazioni) e per la gestione delle dispute. La capacità di dimostrare in modo inconfutabile l'origine e l'integrità di ogni operazione finanziaria è un requisito non negoziabile. La PSD2, ad esempio, attraverso i suoi Regulatory Technical Standards (RTS) su SCA e Common and Secure Communication (CSC), enfatizza la necessità di generare e conservare prove delle transazioni e degli accessi sicuri [27].

2.12 Privacy by Design (PbD) e Privacy by Default

Il framework della Privacy by Design (PbD), concettualizzato da Ann Cavoukian, postula l'integrazione proattiva della protezione dei dati personali fin dalle primissime fasi di progettazione di sistemi, servizi, prodotti e processi di business, rendendo la privacy un'impostazione predefinita (Privacy by Default) [28]. I sette principi fondanti includono: proattività, privacy come impostazione di default, privacy incorporata nel design, piena funzionalità (somma positiva, non somma zero tra privacy e funzionalità), sicurezza end-to-end, visibilità e trasparenza, e rispetto per la privacy dell'utente. Misure concrete:

- **Minimizzazione dei Dati:** Raccolta, trattamento e conservazione dei soli dati personali strettamente necessari e pertinenti alle finalità dichiarate.
- **Tecniche di Anonimizzazione e Pseudonimizzazione (Privacy Enhancing Technologies, PETs):** Applicazione di tecniche come differential privacy, k-anonymity, crittografia omomorfa (homomorphic encryption), o pseudonimizzazione per dataset analitici, sviluppo, test, o condivisione con terze parti.
- **Mappatura e Valutazione d'Impatto sulla Protezione dei Dati (Data Protection Impact Assessment, DPIA):** Mantenimento di registri delle attività di trattamento (Art. 30 del General Data Protection Regulation, GDPR) e conduzione di DPIA per trattamenti a rischio elevato.

Nel settore fintech, dove si trattano enormi volumi di dati personali e finanziari altamente sensibili, l'adozione di PbD è un imperativo legale (GDPR) ed etico. Implementare sistemi di Data Loss Prevention (DLP), classificare i dati fin dall'inizio, e integrare controlli sulla privacy nei cicli di sviluppo agile (ad esempio, "privacy stories" nelle sprint) sono pratiche essenziali. Questo non solo mitiga il rischio di violazioni dei dati, sanzioni e danni reputazionali, ma costruisce anche la fiducia degli utenti, un asset inestimabile per qualsiasi fintech.

2.12.1 Conclusioni Preliminari del Capitolo

I principi di cybersecurity che abbiamo qui delineato – dalla Triade CIA alla Privacy by Design – rappresentano le fondamenta su cui ogni startup fintech deve costruire la propria strategia di sicurezza. Tuttavia, come si è osservato, la loro applicazione rigorosa si scontra frequentemente con le dinamiche operative tipiche delle aziende in fase di avviamento: la scarsità di risorse finanziarie e umane specializzate, la

pressione per un rapido ingresso e affermazione sul mercato, e la necessità di mantenere un'elevata agilità operativa. Questa tensione intrinseca non deve però tradursi in una rinuncia alla sicurezza, bensì in un approccio strategico che integri questi principi in modo pragmatico e progressivo, prioritizzando i controlli in base al rischio e alla criticità dei dati e dei processi coinvolti. Comprendere queste sfide è il primo passo per sviluppare architetture e soluzioni di sicurezza che siano non solo tecnicamente valide, ma anche sostenibili ed efficaci nel peculiare contesto di una startup fintech. Nei capitoli successivi esploreremo come tali principi possano essere tradotti in architetture e controlli tecnologici specifici, tenendo conto di queste sfide.

Capitolo 3

Principi dell'Infrastruttura Cloud e Scelta di AWS

Dopo aver introdotto le sfide di cybersecurity specifiche per le startup fintech, risulta fondamentale comprendere il contesto tecnologico in cui queste operano. Attualmente, la stragrande maggioranza delle nuove imprese, specialmente nel settore tecnologico e finanziario, basa la propria infrastruttura su modelli di **cloud computing**. Questo capitolo esplora i motivi di tale scelta, confrontando l'approccio cloud con quello tradizionale on-premises, e introduce **Amazon Web Services (AWS)**, il provider cloud prescelto per il nostro caso studio, delineandone la struttura e i principi fondamentali.

3.1 Fondamenti di Cloud Computing

Il *cloud computing* (elaborazione tramite nuvola informatica) si configura come un modello di fruizione IT "*on-demand*" che abilita l'accesso ubiquo e conveniente via rete a un pool condiviso di risorse computazionali configurabili (quali reti, server, storage, applicazioni e servizi), le quali possono essere predisposte o rilasciate rapidamente con il minimo sforzo di gestione o intervento da parte del provider [29].

Il modello di cloud computing definito dal *National Institute of Standards and Technology (NIST)* si articola su cinque caratteristiche essenziali che ne delineano la natura e il funzionamento. Queste caratteristiche sono cruciali per comprendere come il cloud si differenzi da altre forme tradizionali di gestione delle risorse IT e perché esso rappresenti un'evoluzione significativa per l'erogazione di servizi digitali. Si analizzano di seguito tali caratteristiche in dettaglio:

- **Autoservizio on-demand (On-demand self-service):** Gli utenti hanno la possibilità, in autonomia e senza l'intervento diretto del fornitore di servizi, di approvvigionarsi delle risorse di cui necessitano (come spazio di archiviazione, potenza di calcolo, ecc.) mediante interfacce self-service, spesso accessibili via web [29].
- **Accesso ampio alla rete (Broad network access):** Le risorse cloud sono accessibili attraverso la rete tramite meccanismi standard (ad esempio, browser web, applicazioni mobili), favorendo così l'utilizzo da dispositivi eterogenei quali smartphone, tablet, laptop e workstation [29].
- **Resource pooling (Multitenancy):** Le risorse del fornitore vengono raggruppate per servire più clienti (tenant) utilizzando un modello multi-tenant, in cui risorse fisiche e virtuali sono assegnate e riassegnate dinamicamente in base alla domanda dei vari utenti. Tale approccio garantisce efficienza e ottimizzazione nell'impiego dell'infrastruttura [29].
- **Rapid elasticity (Scalabilità rapida ed elasticità):** Le capacità possono essere scalate rapidamente, spesso in modo automatico, in base alle necessità dell'utente. Dal punto di vista del cliente, le risorse disponibili appaiono frequentemente illimitate e possono essere acquisite in qualsiasi quantità e in qualsiasi momento [29].
- **Servizio misurato (Measured service):** I sistemi cloud controllano e ottimizzano automaticamente l'uso delle risorse tramite capacità di misurazione appropriate (ad esempio, monitoraggio dell'archiviazione, dell'elaborazione, della larghezza di banda). Questo consente trasparenza sia per il provider sia per il consumatore del servizio [29].

3.1.1 Modelli di servizio e distribuzione cloud

I principali modelli di servizio e distribuzione cloud si rivelano fondamentali per l'innovazione e la crescita delle startup fintech. Essi permettono di accedere a risorse tecnologiche avanzate in modo flessibile e scalabile, spesso con una riduzione dei costi iniziali, accelerando l'innovazione e migliorando l'agilità operativa [30], [31]. La comprensione di questi modelli è cruciale per le startup fintech che mirano a bilanciare efficienza, sicurezza e conformità normativa. Secondo lo standard internazionale ISO/IEC 17788, i modelli di servizio e di distribuzione offrono diverse opzioni per l'adozione del cloud [32].

Modelli di servizio cloud

I modelli di servizio definiscono il livello di controllo e gestione dell'infrastruttura cloud da parte dell'utente. I tre modelli principali, *Infrastructure as a Service (IaaS)*, *Platform as a Service (PaaS)* e *Software as a Service (SaaS)*, offrono diversi gradi di astrazione [33].

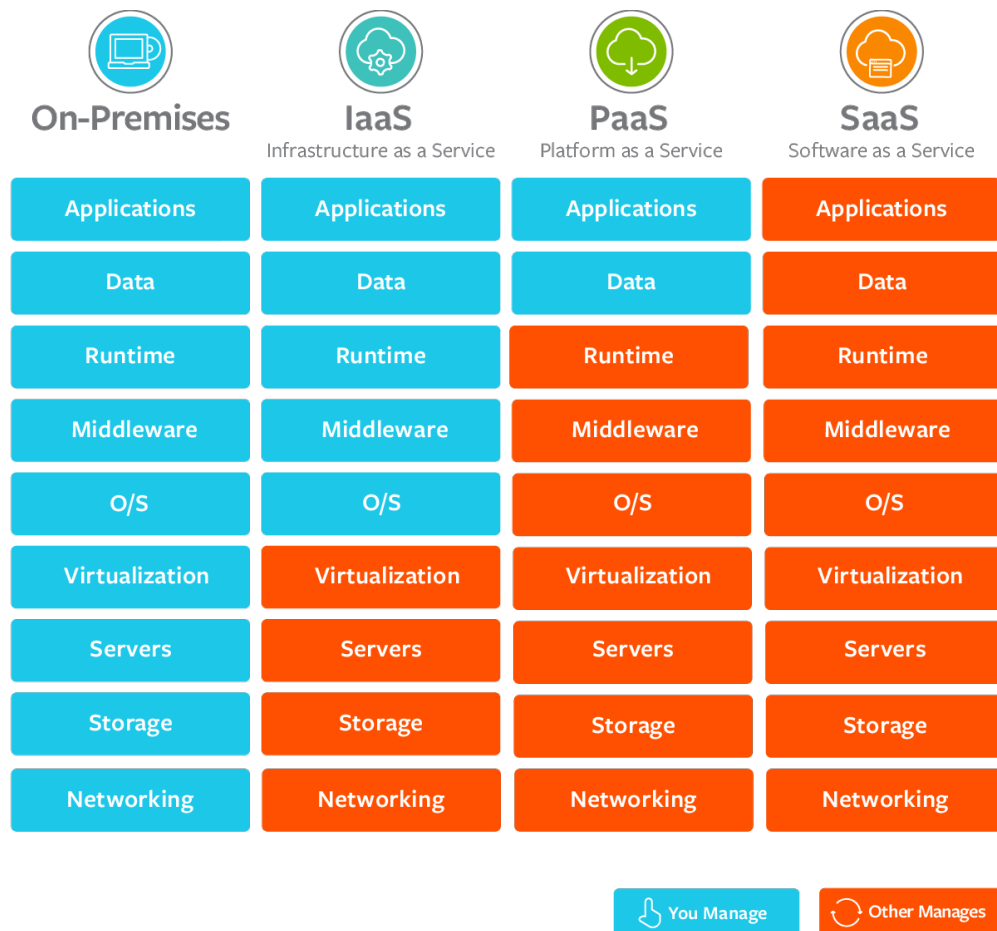


Figura 3: Modelli di Servizio Cloud

- **IaaS (Infrastructure as a Service):** questo modello fornisce infrastruttura IT on-demand, quali server, macchine virtuali, storage e rete, gestita dal provider cloud [33]. Le startup fintech possono così costruire e configurare ambienti IT complessi senza investire in hardware fisico, riducendo i costi di avvio e manutenzione [30]. L'elasticità dell'IaaS consente di scalare rapidamente le risorse in base alla crescita del business o ai picchi di utilizzo, una caratteristica cruciale nel dinamico settore fintech [30], [31].

- **PaaS (Platform as a Service):** questo modello fornisce una piattaforma completa, comprensiva di sistema operativo, middleware e strumenti di sviluppo, pronta per la creazione, l'esecuzione e la gestione di applicazioni [33]. Il provider gestisce l'infrastruttura sottostante, permettendo agli sviluppatori fintech di concentrarsi sull'innovazione del software [30]. Ciò accelera il time-to-market di nuovi servizi finanziari e favorisce l'adozione di tecnologie avanzate come l'intelligenza artificiale e l'analisi dei dati [30].
- **SaaS (Software as a Service):** questo modello offre applicazioni software pronte all'uso accessibili tramite Internet, come sistemi di *Customer Relationship Management (CRM)* o piattaforme di pagamento [33]. Le startup fintech possono integrare rapidamente soluzioni di terze parti nei propri processi, ottimizzando le risorse e focalizzandosi sull'esperienza utente e sull'innovazione dei servizi [30].

Questi modelli di servizio sono stati analizzati in studi sistematici e bibliometrici, i quali hanno evidenziato come la loro adozione, specialmente in contesti agili come le Piccole e Medie Imprese (PMI) e le startup fintech, apporti benefici in termini di flessibilità, riduzione dei costi e scalabilità [30], [31].

Modelli di distribuzione del cloud

I modelli di distribuzione definiscono come l'infrastruttura cloud è resa disponibile e a chi è dedicata. I principali modelli includono il *cloud pubblico (public cloud)*, il *cloud privato (private cloud)*, il *cloud ibrido (hybrid cloud)* e il *cloud comunitario (community cloud)* [32], [34].

- **Public Cloud:** l'infrastruttura è di proprietà di un provider terzo (ad esempio, AWS, Azure, Google Cloud) e condivisa tra più clienti via Internet [34]. Per le startup fintech, il public cloud offre alta scalabilità e costi operativi proporzionali all'uso, risultando ideale per lanciare servizi senza investimenti iniziali significativi [34]. Tuttavia, la condivisione delle risorse può sollevare preoccupazioni per la sicurezza e il controllo diretto, aspetti critici per i dati finanziari [30], [34].
- **Private Cloud:** l'infrastruttura è dedicata a una singola organizzazione, garantendo livelli superiori di controllo, sicurezza e conformità normativa [34]. Questo aspetto è essenziale per le fintech che trattano dati sensibili. Lo svantaggio principale risiede nei costi più elevati e nella minore scalabilità immediata rispetto al cloud pubblico [34].

- **Hybrid Cloud:** questo modello combina ambienti pubblici e privati, permettendo di spostare carichi di lavoro in base alle necessità [34]. Una startup fintech può utilizzare il public cloud per servizi meno critici e il private cloud per dati regolamentati, bilanciando flessibilità, scalabilità, sicurezza e compliance [34]. Tuttavia, l'adozione del cloud ibrido nel settore fintech presenta sfide di sicurezza specifiche, che possono essere mitigate, ad esempio, attraverso l'uso della tecnologia *blockchain* (catena di blocchi) per garantire la compliance e la protezione dei dati sensibili [30], [35]. La gestione efficace di un ambiente ibrido è cruciale per ottimizzare i costi e mantenere la coerenza con i requisiti normativi [34].

3.1.2 Cloud Computing vs Infrastrutture On-Premises

Tradizionalmente, le aziende gestivano la propria infrastruttura IT internamente, in data center di proprietà o in affitto. Questo modello è noto come **on-premises** (infrastruttura locale). Esso richiede l'acquisto di hardware (server, storage, apparati di rete), software (sistemi operativi, licenze) e l'impiego di personale specializzato per la gestione, la manutenzione, gli aggiornamenti e la sicurezza fisica ed operativa.

Il **cloud computing**, al contrario, si basa sull'erogazione di risorse informatiche (come potenza di calcolo, storage, database, reti, software, analytics, intelligenza artificiale) tramite Internet, secondo un modello *pay-as-you-go* (pagamento basato sul consumo effettivo). I fornitori di servizi cloud, denominati *Cloud Service Provider (CSP)*, come AWS, Microsoft Azure o Google Cloud Platform, gestiscono l'infrastruttura fisica sottostante, permettendo ai clienti di accedere alle risorse di cui hanno bisogno in modo flessibile e scalabile.

Le differenze principali risiedono nei seguenti aspetti:

- **Costi:** L'approccio on-premises richiede un ingente investimento iniziale, noto come *Capex (Capital Expenditure)*, per l'acquisto dell'hardware, mentre il cloud trasforma questo costo in una spesa operativa variabile, definita *Opex (Operational Expenditure)*, basata sul consumo effettivo.
- **Scalabilità:** Il cloud offre una scalabilità *elastica*, permettendo di aumentare o diminuire le risorse quasi istantaneamente in base alla domanda. L'infrastruttura on-premises presenta una scalabilità limitata e richiede pianificazione e acquisti anticipati per gestire picchi di carico.
- **Manutenzione:** Nel cloud, la manutenzione dell'hardware e dell'infrastruttura di base è responsabilità del provider, liberando il team IT del cliente da tali incombenze.

- **Agilità e Velocità:** Il cloud permette di predisporre nuove risorse in pochi minuti, accelerando notevolmente i cicli di sviluppo e il *time-to-market* (tempo di immissione sul mercato) di nuovi prodotti o servizi.
- **Affidabilità e Portata Globale:** I principali CSP dispongono di data center ridondati in diverse regioni geografiche, offrendo alta disponibilità e la possibilità di distribuire applicazioni a livello globale con bassa latenza.

3.1.3 Perché le Startup Scelgono il Cloud

Per le startup, e in modo particolarmente accentuato per quelle operanti nel dinamico settore fintech che richiedono elevata agilità e una gestione efficiente delle risorse, il modello cloud computing offre vantaggi strategici e operativi decisivi rispetto alle tradizionali infrastrutture on-premises. Tali vantaggi includono:

- **Riduzione delle Barriere all'Ingresso e Ottimizzazione dei Costi:** L'eliminazione di ingenti investimenti iniziali in hardware e software (Capex) rende accessibili tecnologie avanzate anche a entità con budget limitati. Il modello di pagamento basato sul consumo effettivo (Opex) permette di allineare i costi direttamente alla crescita e all'utilizzo reale delle risorse.
- **Agilità e Adattabilità delle Risorse:** Le startup possono avviare le proprie operazioni con un dimensionamento infrastrutturale minimo, per poi espandere o contrarre rapidamente le risorse in risposta alle dinamiche del mercato o alla crescita della base utenti e del volume transazionale. Questa capacità è fondamentale nel fintech, dove la domanda può essere volatile e imprevedibile. Le modalità tecniche attraverso cui si realizza tale agilità, ovvero scalabilità ed elasticità, saranno analizzate in dettaglio nel prosieguo della sezione.
- **Focalizzazione Strategica sul Core Business:** Delegando la gestione, la manutenzione e l'aggiornamento dell'infrastruttura IT al Cloud Service Provider (CSP), la startup può concentrare le proprie risorse umane e finanziarie, spesso limitate, sullo sviluppo del prodotto, sull'innovazione di servizio e sull'acquisizione di quote di mercato, piuttosto che sulla complessa gestione dei sistemi.
- **Accelerazione del Time-to-Market:** La capacità di predisporre e de-predisporre rapidamente ambienti di sviluppo, test e produzione consente di abbreviare significativamente i cicli di rilascio di nuove funzionalità e servizi, un fattore competitivo critico in settori ad alta innovazione come il fintech.

- **Accesso a Tecnologie Avanzate e Servizi Gestiti:** I CSP mettono a disposizione un vasto portafoglio di servizi all'avanguardia, pronti all'uso e gestiti, che includono database performanti, piattaforme di machine learning, strumenti di big data analytics e soluzioni di sicurezza. L'implementazione e la gestione autonoma di tali tecnologie on-premises comporterebbero costi e complessità proibitivi per la maggior parte delle startup.
- **Elevati Livelli di Resilienza, Disponibilità e Sicurezza Infrastrutturale:** I CSP investono massicciamente in infrastrutture ridondanti e geograficamente distribuite, offrendo elevati livelli di disponibilità (tipicamente garantiti da *Service Level Agreement* - *SLA*, accordi sul livello di servizio) e resilienza operativa, spesso superiori a quanto una startup potrebbe garantire autonomamente. Questo si combina con solide fondamenta di sicurezza fisica e operativa dei data center (sebbene la sicurezza nel cloud, relativa ai dati e alle applicazioni, rimanga una responsabilità condivisa e primariamente del cliente).

Questi vantaggi traggono origine, a livello infrastrutturale, da caratteristiche intrinseche dei modelli cloud che risultano particolarmente sinergiche con le esigenze del fintech. Tra queste, spiccano la scalabilità, la flessibilità e l'elasticità, le cui specificità verranno ora analizzate.

Scalabilità (Scalability): Si riferisce alla capacità intrinseca di un sistema di incrementare la propria capacità elaborativa e di gestione dei dati per far fronte a un aumento del carico di lavoro. Tale incremento può essere pianificato (ad esempio, in previsione di una crescita organica del numero di utenti o di transazioni) e si attua mediante l'aggiunta di risorse (*scale-out*, aggiungendo più nodi) o l'aumento della potenza dei nodi esistenti (*scale-up*, incrementando le risorse di un singolo nodo). Per una startup fintech, la scalabilità assicura che l'infrastruttura possa supportare l'espansione del business e gestire volumi di domanda crescenti, anche se caratterizzati da ciclicità prevedibile (ad esempio, picchi di fine mese per elaborazione stipendi o rendicontazioni).

Flessibilità (Flexibility): Concerne la possibilità di scegliere, configurare e modificare un'ampia gamma di risorse, servizi e modelli operativi offerti dalla piattaforma cloud. Include la facoltà di selezionare diverse tipologie di istanze computazionali, opzioni di storage, database, servizi di intelligenza artificiale, strumenti di networking e sicurezza. Per una startup fintech, la flessibilità si traduce nella capacità di sperimentare rapidamente nuove soluzioni, integrare servizi di terze parti, adattarsi a requisiti normativi mutevoli o modificare l'architettura applicativa senza vincoli infrastrutturali stringenti. Oltre alla vasta gamma di opzioni tecnologiche, la flessibilità si estende ai modelli contrattuali e di pricing (ad esempio, istanze on-demand, reserved instances, spot instances), consentendo

un'ulteriore ottimizzazione dei costi in base a profili di utilizzo specifici e alla strategia finanziaria della startup.

Elasticità (Elasticity): Rappresenta una forma dinamica e, idealmente, automatizzata di scalabilità. L'elasticità permette al sistema di allocare e deallocare risorse computazionali (come capacità di calcolo, memoria o banda) in maniera automatica e in tempo reale, in risposta a fluttuazioni immediate e spesso imprevedibili del carico di lavoro [36]. A differenza della scalabilità, che può implicare interventi pianificati, l'elasticità reagisce autonomamente a picchi improvvisi (*burst*) o a cali repentini della domanda, ottimizzando sia le prestazioni che i costi.

La sinergia di queste tre caratteristiche – scalabilità, flessibilità ed elasticità – consente alle startup fintech di ottimizzare l'allocazione dei costi infrastrutturali, aderendo pienamente al principio del "pay-per-use" (pagamento basato sull'effettivo consumo), e di garantire livelli prestazionali adeguati e resilienti. Questa sinergia è particolarmente vitale per le startup fintech, che operano in un mercato caratterizzato da un'intensa competizione, una rapida evoluzione delle aspettative dei consumatori e una continua innovazione tecnologica, richiedendo la massima reattività e capacità di adattamento per mantenere e incrementare la propria competitività.

3.1.4 Introduzione ad Amazon Web Services (AWS)

Tra i principali fornitori di servizi cloud, **Amazon Web Services (AWS)** si posiziona come leader di mercato e rappresenta la scelta infrastrutturale per moltissime startup a livello globale, incluse quelle operanti nel settore fintech, come nel caso studio di questa tesi. Lanciato nel 2006, AWS offre un portafoglio estremamente ampio e maturo di servizi cloud.

La struttura di AWS si basa su alcuni concetti chiave:

- **Infrastruttura Globale:** AWS opera attraverso una rete mondiale di **Regioni**. Ogni Regione è un'area geografica fisica separata (ad esempio, Irlanda, Francoforte, Nord Virginia). All'interno di ciascuna Regione, esistono multiple **Zone di Disponibilità (Availability Zones - AZ)**. Una AZ è costituita da uno o più data center discreti, con alimentazione, raffreddamento e rete ridondati. Le AZ all'interno di una Regione sono interconnesse con reti a bassa latenza ma sono fisicamente separate per garantire l'isolamento in caso di guasti (come incendi o allagamenti). Questa architettura permette di costruire applicazioni altamente disponibili e tolleranti ai guasti, distribuendole su più AZ.
- **Servizi Fondamentali:** AWS offre centinaia di servizi, ma alcuni sono considerati fondamentali:

- **Compute:** Servizi per l'esecuzione di codice, come *Amazon Elastic Compute Cloud (Amazon EC2)* per macchine virtuali scalabili, *AWS Lambda* per l'esecuzione di codice serverless (senza la gestione di server), e servizi container come *Amazon Elastic Container Service (ECS)* ed *Amazon Elastic Kubernetes Service (EKS)*.
 - **Storage:** Servizi per l'archiviazione dei dati, come *Amazon Simple Storage Service (Amazon S3)* per lo storage a oggetti altamente duraturo e scalabile, *Amazon Elastic Block Store (Amazon EBS)* per volumi a blocchi destinati alle istanze EC2, e *Amazon Elastic File System (Amazon EFS)* per file system condivisi.
 - **Database:** Una vasta gamma di database gestiti, inclusi database relazionali (*Amazon Relational Database Service - Amazon RDS*), NoSQL (*Amazon DynamoDB*), data warehouse (*Amazon Redshift*), ecc.
 - **Networking:** Servizi per definire e controllare la rete virtuale, come *Amazon Virtual Private Cloud (Amazon VPC)* per creare reti isolate, *Elastic Load Balancing (ELB)* per distribuire il traffico, e *AWS Direct Connect* per connessioni dedicate.
 - **Security, Identity, & Compliance:** Servizi per gestire accessi, sicurezza e conformità, come *AWS Identity and Access Management (IAM)*, *AWS Key Management Service (KMS)*, *AWS Web Application Firewall (WAF)*, *Amazon GuardDuty* (per il rilevamento delle minacce).
- **Modello Pay-as-you-go:** Come accennato, si paga solo per le risorse effettivamente consumate, senza contratti a lungo termine o costi iniziali (per la maggior parte dei servizi).
 - **Modello di Responsabilità Condivisa (Shared Responsibility Model):** È cruciale comprendere che la sicurezza su AWS è una responsabilità condivisa. AWS è responsabile della sicurezza **del** cloud (l'infrastruttura fisica, la rete, l'hypervisor), mentre il cliente è responsabile della sicurezza **nel** cloud (la configurazione dei servizi, la gestione degli accessi, la protezione dei dati, la sicurezza del sistema operativo e delle applicazioni).

3.1.5 Il Caso Specifico: AWS per la Startup fintech

La selezione di AWS quale infrastruttura cloud per la startup fintech oggetto di questa tesi è il risultato di una valutazione strategica, basata su fattori tecnici e di business specifici. Oltre ai benefici generali offerti dal cloud computing, AWS presenta caratteristiche particolarmente determinanti per un operatore del settore finanziario:

- **Maturità e Affidabilità:** Essendo il provider cloud più longevo e con la più vasta quota di mercato, AWS vanta una comprovata esperienza nella gestione di carichi di lavoro *mission-critical* (critici per la missione aziendale). Tale affidabilità è essenziale per un settore, come quello finanziario, che richiede massima operatività e fiducia da parte degli utenti.
- **Ampiezza dei Servizi:** Il vasto portafoglio di servizi AWS permette di costruire architetture complesse, resilienti e moderne. Esso consente di integrare nativamente servizi per l'analisi avanzata dei dati, il machine learning (fondamentale, ad esempio, per sistemi antifrode, credit scoring o personalizzazione dei servizi finanziari), e la gestione sicura delle transazioni.
- **Supporto alla Compliance:** AWS offre documentazione dettagliata, strumenti e servizi specifici (come AWS Artifact per l'accesso ai report di compliance) che aiutano le organizzazioni a soddisfare i rigorosi standard di conformità richiesti nel settore finanziario, tra cui PCI DSS, GDPR, ISO 27001 e normative locali specifiche. AWS stessa mantiene numerose certificazioni internazionali per la propria infrastruttura, fornendo una base solida su cui costruire applicazioni conformi.
- **Scalabilità e Performance Elevate:** La capacità di scalare le risorse in modo elastico e di garantire alte prestazioni è fondamentale per gestire picchi transazionali improvvisi (ad esempio, durante l'apertura dei mercati o in seguito a campagne promozionali) e assicurare tempi di risposta rapidi, critici per l'esperienza utente nei servizi finanziari.
- **Ecosistema di Partner Specializzati:** Esiste un vasto ecosistema di partner tecnologici e di consulenza con competenze specifiche su AWS, inclusi numerosi attori con expertise verticale nel settore fintech, in grado di supportare la startup in fasi di progettazione, implementazione e gestione.
- **Servizi di Sicurezza Avanzati e Stratificati:** AWS offre un set robusto e profondo di strumenti nativi per implementare controlli di sicurezza a molteplici livelli (rete, identità, dati, rilevamento delle minacce, crittografia), aspetti che verranno analizzati in dettaglio nei capitoli successivi.

Nei capitoli successivi, si analizzerà in dettaglio come l'infrastruttura di questa specifica startup fintech sia stata progettata e messa in sicurezza sfruttando i servizi e le *best practice* (migliori pratiche) offerte da AWS, con un focus sulle implicazioni e le scelte architetturali per un operatore del settore finanziario.

3.1.6 Infrastruttura Globale AWS: Fondamenta per la fin-tech

Amazon Web Services (AWS) dispone di una infrastruttura globale altamente distribuita: nel 2025 il cloud AWS è esteso su 36 Regioni geografiche (ciascuna costituita da più Availability Zone) per un totale di 114 Availability Zones lanciate [37]. Ogni Regione AWS rappresenta un'area geografica distinta, fisicamente isolata dalle altre per garantire *fault tolerance* (tolleranza ai guasti) e permettere il rispetto dei requisiti regolamentari sulla sovranità dei dati [37]. All'interno di ogni Regione sono presenti almeno tre *Availability Zone (AZ)*, concepite come data center indipendenti dal punto di vista dell'alimentazione, del raffreddamento e della connettività di rete fisica, pur essendo interconnesse tramite collegamenti privati ridondanti ad alta velocità e bassa latenza [37].

Questo design multi-AZ è particolarmente critico per una startup fintech: la capacità di resistere a guasti localizzati (come un'interruzione di corrente in un data center o un evento catastrofico limitato a una singola AZ) senza interruzione del servizio è fondamentale per mantenere la continuità operativa nelle transazioni finanziarie, preservare la fiducia degli utenti e rispettare potenziali requisiti normativi sulla disponibilità dei servizi. Secondo AWS, ciò garantisce un'elevata disponibilità dell'infrastruttura e contiene l'impatto di eventuali interruzioni all'interno della Regione interessata [37].

Per supportare applicazioni globali a bassa latenza, cruciali per l'esperienza utente nei servizi finanziari digitali, AWS integra inoltre le *Edge Location* (punti di presenza perimetrali) e le *Local Zone* (zone locali). Le Edge Location (oltre 700 nel mondo) sono data center che ospitano servizi come Amazon CloudFront, una *Content Delivery Network (CDN)* (rete per la consegna di contenuti) che permette la consegna rapida di contenuti statici e dinamici agli utenti finali. CloudFront instrada le richieste al *punto di presenza (PoP)* geograficamente più vicino all'utente, minimizzando la latenza [38]. Le Local Zone sono estensioni dell'infrastruttura AWS posizionate strategicamente vicino a grandi centri urbani o industriali, progettate per offrire latenze nell'ordine dei millisecondi a singola cifra per scenari applicativi specifici (ad esempio, sistemi di trading ad alta frequenza, elaborazione di pagamenti in tempo reale o applicazioni di realtà aumentata per servizi finanziari).

Il backbone di rete globale AWS, basato su una dorsale in fibra ottica ridondata che interconnette le Regioni con capacità fino a 400 Gb/s [39], è un altro asset fondamentale. Tutti i dati che transitano su questa rete globale tra i data center e le Regioni AWS vengono crittografati automaticamente a livello fisico prima di lasciare le strutture protette [39]. Il cliente, inoltre, mantiene il pieno controllo sui propri dati, inclusa la facoltà di applicare ulteriori livelli di crittografia utilizzando

servizi dedicati. Questa solida infrastruttura di rete non solo garantisce performance elevate, ma la crittografia automatica dei dati in transito aggiunge un livello di sicurezza fondamentale per i dati finanziari sensibili.

Dal punto di vista di una startup fintech, una siffatta infrastruttura globale distribuita offre vantaggi tangibili. La possibilità di collocare geograficamente le risorse permette di posizionare le applicazioni e i dati vicino agli utenti finali o in specifiche giurisdizioni per soddisfare requisiti di bassa latenza (cruciali per l'esperienza utente in app finanziarie) e di sovranità dei dati (come il GDPR). L'ampia rete backbone e la sua crittografia intrinseca proteggono le comunicazioni inter-regionali, essenziali ad esempio per strategie di disaster recovery. Le Edge Location, tramite CloudFront, possono accelerare significativamente l'erogazione di interfacce web o API rivolte ai clienti, migliorando la reattività delle piattaforme fintech, un fattore competitivo chiave.

3.1.7 Architettura Virtualizzata e Meccanismi di Scalabilità per la fintech

Le risorse computazionali su AWS sono erogate principalmente attraverso tecnologie di *virtualizzazione*. Su ogni server fisico (host) viene eseguito un *hypervisor* (monitor di macchine virtuali), un software che astrae l'hardware sottostante e crea molteplici istanze virtuali isolate tra loro. In AWS, la virtualizzazione, storicamente basata su hypervisor Xen e più recentemente sul sistema AWS Nitro (che include un hypervisor leggero basato su KVM, Kernel-based Virtual Machine) [40], consente di eseguire su un singolo server fisico decine di *macchine virtuali (VM)* indipendenti, ciascuna con il proprio sistema operativo e le proprie applicazioni [41]. L'hypervisor assegna a ogni VM una porzione dedicata di CPU, memoria e storage, garantendo l'isolamento delle risorse e della sicurezza tra le istanze [41].

Grazie alla virtualizzazione, più tenant (clienti AWS) possono condividere in modo sicuro lo stesso hardware fisico sottostante; questo è il concetto di *multitenancy* (multi-locazione), definito dal NIST come un'architettura in cui "una singola istanza di un software gira su un server ed è usata da più tenant" [29]. In pratica, anche in un modello multitenant come quello di AWS, ogni cliente opera all'interno del proprio ambiente virtuale logicamente isolato, con i propri dati segregati, grazie a meccanismi di isolamento di rete (ad esempio, *Virtual Private Cloud - VPC*) e alle garanzie fornite dal software di virtualizzazione. Per una startup fintech, il modello multitenant, pur offrendo significativi vantaggi in termini di costi e agilità, richiede un'attenta valutazione e l'implementazione di robuste strategie di isolamento a livello applicativo e di rete, complementari a quelle fornite da AWS, per garantire la segregazione e la protezione dei dati finanziari sensibili.

La virtualizzazione è il fulcro dell'offerta IaaS di AWS. Come delineato nel modello di responsabilità condivisa (descritto in seguito), AWS gestisce l'infrastruttura fisica sottostante (hardware, networking, data center, patching dell'hypervisor), mentre il cliente mantiene il controllo sul sistema operativo guest, sugli aggiornamenti software, sulle configurazioni di sicurezza dell'ambiente virtuale e sulle applicazioni [42]. Ad esempio, quando si avvia un'istanza Amazon EC2 (il servizio IaaS di AWS), AWS fornisce la macchina virtuale, ma è responsabilità del cliente gestirne il sistema operativo, le patch di sicurezza e il software applicativo. Questa astrazione permette di orchestrare e scalare migliaia di istanze virtuali con un elevato grado di automazione.

Per gestire la crescita del carico di lavoro e le fluttuazioni della domanda, tipiche del settore fintech, AWS offre due principali meccanismi di scalabilità:

- La **scalabilità verticale** (scale-up) consiste nell'aumentare le risorse di una singola istanza (ad esempio, incrementando CPU, RAM o capacità di I/O del disco) [43]. Questa strategia può essere utile per carichi di lavoro monolitici o database, ma presenta limiti fisici intrinseci e può introdurre un *single point of failure* (singolo punto di guasto) se non gestita con ridondanza. Per una startup, può essere una soluzione iniziale, ma la sua limitata resilienza la rende meno adatta per carichi di lavoro critici nel settore fintech a lungo termine.
- La **scalabilità orizzontale** (scale-out) implica la distribuzione del carico di lavoro su più istanze o nodi, replicando l'applicazione [43]. Ad esempio, si possono avviare più istanze EC2 identiche dietro un bilanciatore di carico (come Elastic Load Balancing). In questo modo, il traffico utente viene distribuito tra le VM disponibili, e il sistema può tollerare il guasto di singole istanze senza interrompere il servizio. La scalabilità orizzontale è particolarmente vantaggiosa per le applicazioni fintech, che spesso sperimentano fluttuazioni significative nel carico di lavoro (ad esempio, durante l'apertura dei mercati, eventi promozionali, o picchi di fine mese per elaborazioni contabili). La capacità di aggiungere o rimuovere istanze dinamicamente, spesso gestita da servizi come AWS Auto Scaling [43], permette alla startup di mantenere performance ottimali e controllare i costi in modo efficiente, pagando solo per le risorse effettivamente utilizzate.

Per massimizzare la disponibilità delle applicazioni, un requisito non negoziabile per i servizi finanziari, in AWS si progetta attivamente per la resilienza utilizzando repliche *multi-AZ* (distribuite su più Zone di Disponibilità). Ad esempio, il servizio Amazon RDS consente di creare istanze database in configurazione Multi-AZ: quando questa opzione è abilitata, AWS provvede automaticamente a creare e

mantenere una replica sincrona (standby) del database primario in una Availability Zone differente all'interno della stessa Regione [44]. Tutte le modifiche al database primario vengono replicate in tempo reale alla standby. In caso di guasto dell'istanza primaria o dell'intera AZ in cui risiede, RDS gestisce un *failover* (passaggio automatico alla replica) trasparente alla replica standby, minimizzando i tempi di indisponibilità, noti come *Recovery Time Objective (RTO)*. Questa funzionalità è di importanza critica per una startup fintech, poiché la perdita di accesso al database transazionale principale potrebbe comportare l'interruzione dei servizi, perdite finanziarie e danni reputazionali significativi. Analogamente, servizi come Elastic Load Balancing possono e devono essere configurati per distribuire il traffico su istanze dislocate in multiple AZ, garantendo che un'interruzione locale sia compensata dagli altri nodi attivi. Le scelte architetturali per l'alta disponibilità includono quindi l'uso sistematico di configurazioni multi-AZ, il bilanciamento del carico e la replica dei dati (eventualmente su più Regioni per il *disaster recovery*, recupero da disastro).

Nei casi estremi di disastro che potrebbero compromettere un'intera Regione AWS, è necessario implementare strategie di Disaster Recovery (DR) più complesse, come delineato nel Well-Architected Framework di AWS [45]. Tra queste strategie si annoverano:

- **Backup and Restore (Backup e Ripristino):** Utilizzo di snapshot periodici dei dati (ad esempio, archiviati su Amazon S3 e Amazon S3 Glacier) e template infrastrutturali (ad esempio, AWS CloudFormation) per ricreare l'infrastruttura e ripristinare i dati in una Regione secondaria. È la strategia con RTO e *Recovery Point Objective (RPO)* (obiettivo del punto di ripristino, ovvero la massima perdita di dati accettabile) più elevati.
- **Pilot Light (Fiamma Pilota):** Mantenimento di una copia minima dell'infrastruttura (core infrastrutturale) e dei dati critici costantemente replicati nella Regione di DR. Le risorse applicative principali vengono attivate solo in caso di disastro.
- **Warm Standby (Standby Tiepido):** Mantenimento di una versione scalata ridotta, ma pienamente funzionante, dell'applicazione nella Regione di DR, pronta a gestire il traffico in caso di failover.
- **Multi-site Active/Active (o Hot Standby, Standby Caldo):** L'applicazione è dispiegata e attivamente serve traffico contemporaneamente in due o più Regioni, con meccanismi di bilanciamento del carico geografico. È la strategia più complessa e costosa, ma offre l'RTO e l'RPO più bassi.

La scelta della strategia di DR per una startup fintech dipenderà da una valutazione del rischio, dai requisiti normativi (spesso stringenti nel settore finanziario per

quanto riguarda la continuità operativa e i tempi di ripristino) e dal budget disponibile. AWS fornisce strumenti come AWS Resilience Hub [46] per aiutare a definire, testare e monitorare la postura di resilienza delle applicazioni, assicurando che i tempi di recovery (RTO) e la perdita massima di dati accettabile (RPO) siano allineati con le esigenze di business e i requisiti di compliance.

3.1.8 Modello di Responsabilità Condivisa: Implicazioni per la fintech

La sicurezza nel cloud AWS opera secondo il *modello di responsabilità condivisa* (*Shared Responsibility Model*) [47]. Questo modello distingue nettamente le responsabilità di AWS da quelle del cliente. In sintesi, AWS è responsabile della *sicurezza "del" cloud* (*"security of the cloud"*). Ciò include la protezione dell'infrastruttura fisica e globale che esegue tutti i servizi AWS: l'hardware, il software di base (hypervisor, sistemi operativi dei servizi gestiti), la rete e le strutture fisiche (data center), compresi i controlli fisici e ambientali. AWS investe significativamente in misure quali sorveglianza 24/7, rigorosi controlli degli accessi fisici alle strutture, ridondanza dei sistemi e patching dell'infrastruttura sottostante [47].

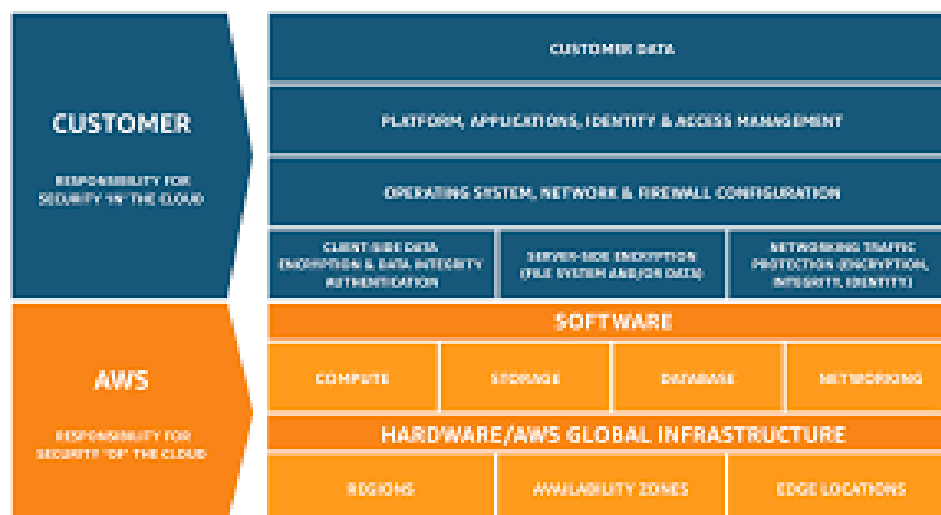


Figura 4: Modello di Responsabilità Condivisa di AWS

D'altro canto, il cliente è responsabile della *sicurezza "nel" cloud* (*"security in the cloud"*). La natura e l'estensione di questa responsabilità dipendono dai servizi AWS scelti e dalla loro astrazione. Ad esempio, per un servizio IaaS come Amazon EC2, il cliente deve gestire la sicurezza del sistema operativo guest (installazione di

patch, hardening), di tutte le applicazioni e software installati, e la configurazione dei controlli di rete a livello di istanza (come i Security Group, che agiscono da firewall stateful) e di sottorete (Network ACLs, liste di controllo accessi di rete) [47]. Per servizi più astratti o gestiti, come Amazon S3 (storage) o Amazon DynamoDB (database NoSQL), AWS gestisce l'infrastruttura sottostante, il sistema operativo e la piattaforma applicativa. Tuttavia, spetta al cliente proteggere i dati che carica e le applicazioni che vi accedono: ciò include la corretta configurazione delle policy di accesso (tramite AWS Identity and Access Management - IAM), la cifratura dei dati sensibili (a riposo e in transito, se non gestita nativamente dal servizio per specifici casi d'uso), la gestione delle chiavi di crittografia e l'applicazione di criteri di rete appropriati [47]. In pratica, AWS fornisce gli strumenti e i meccanismi di sicurezza (crittografia, isolamento di rete, logging e auditing, ecc.), ma la loro corretta implementazione, configurazione e la gestione operativa della sicurezza applicativa e dei dati rimangono a carico del cliente.

Per una startup fintech, una profonda e continua comprensione di questo modello è fondamentale: essa guida l'allocazione delle risorse interne dedicate alla sicurezza, la definizione dei processi di compliance (ad esempio, per PCI DSS o GDPR, dove la startup deve dimostrare di aver adempiuto alle proprie responsabilità), la progettazione di controlli di sicurezza specifici per i dati finanziari e le transazioni dei propri clienti, e la corretta configurazione dei servizi AWS per costruire un ambiente sicuro e resiliente. La mancata comprensione di queste responsabilità può portare a vulnerabilità significative e a non conformità normative.

Capitolo 4

Progettazione e Implementazione Avanzata della Sicurezza delle Identità e degli Accessi (IAM) in AWS

4.1 Introduzione alla Gestione delle Identità e degli Accessi

La gestione delle identità e degli accessi (IAM) rappresenta un passo fondamentale per sviluppare qualsiasi strategia di sicurezza robusta in un ambiente cloud come Amazon Web Services (AWS). Per una startup fintech, dove la struttura gerarchica è ancora poco definita, stabilire chi può accedere alle risorse e con quali privilegi non è solo una best practice, ma una necessità imprescindibile per garantire una gestione sicura e affidabile dei dati. Questo capitolo esplora i principi cardine della sicurezza IAM, analizza la configurazione attuale della startup "Finanz" e propone una serie di miglioramenti concreti per rafforzare la postura di sicurezza, ispirandosi ai modelli Zero Trust e al Principio del Minimo Privilegio. L'obiettivo è creare un framework IAM che sia non solo sicuro, ma anche flessibile e gestibile, per supportare la crescita dinamica della startup.

4.1.1 Configurazione dell'infrastruttura AWS per la piattaforma *Finanz*

Panoramica architetturale L'infrastruttura tecnologica che supporta la piattaforma *Finanz* è interamente provisionata all'interno della regione AWS 'eu-south-1'.

(Milano) e consolidata in un singolo account. Per garantire una netta separazione degli ambienti, sono stati definiti due contesti logici distinti, ‘Finanz-Dev’ per lo sviluppo e ‘Finanz-Prod’ per la produzione. La segregazione tra questi non avviene a livello di account, bensì è implementata attraverso una strategia granulare di tagging delle risorse, l’impiego di ruoli IAM (Identity and Access Management) con permessi specifici e l’adozione di pipeline di rilascio completamente separate e indipendenti.

Layer applicativo Il layer applicativo, che ospita il servizio core ‘Finanz-Backend’, è orchestrato tramite il servizio gestito **AWS Elastic Beanstalk**. Tale servizio astrae la gestione dell’infrastruttura sottostante, automatizzando il provisioning e la configurazione delle risorse. La configurazione prevede l’impiego di istanze EC2 di tipo **burstable** (‘t3a.small’), definite tramite un Launch Template. La scalabilità orizzontale è gestita da un Auto Scaling Group, le cui politiche sono basate su metriche di carico della CPU e sul numero di richieste al secondo, con soglie differenziate tra l’ambiente di sviluppo (da 1 a 2 istanze) e quello di produzione (da 3 a 5 istanze). Un Application Load Balancer (ALB) funge da unico entry-point per il traffico HTTPS, instradandolo verso le istanze applicative. Queste ultime sono protette da un Security Group che ne permette l’accesso esclusivamente dall’ALB, isolandole da accessi diretti. Infine, i log applicativi e di sistema vengono inoltrati in tempo reale verso Amazon CloudWatch Logs per il monitoraggio e archiviati su Amazon S3 con rotazione oraria per l’analisi a lungo termine.

Layer dati Il layer di persistenza dei dati transazionali si affida al servizio gestito **Amazon RDS for PostgreSQL**. La sicurezza dei dati **at-rest** è garantita dalla cifratura, attuata tramite una chiave gestita dal cliente (Customer-Managed Key) in AWS Key Management Service (KMS). Per rispondere a diverse esigenze di disponibilità e costo, l’ambiente di produzione impiega un’istanza in configurazione Multi-AZ, che assicura alta disponibilità e failover automatico, con una retention dei backup di sette giorni e la protezione contro la cancellazione accidentale. Al contrario, l’ambiente di sviluppo utilizza un’istanza Single-AZ, con backup disabilitati. Per entrambe le configurazioni, l’accesso pubblico è negato e i log di database, insieme alle metriche di Performance Insights, vengono aggregati in Amazon CloudWatch per un monitoraggio centralizzato delle prestazioni.

Topologia di rete La rete è incapsulata in un Virtual Private Cloud (VPC) con un ampio spazio di indirizzamento privato (‘10.0.0.0/16’). La segmentazione è ottenuta attraverso una topologia multi-tier che prevede subnet pubbliche e private. Una subnet pubblica ospita le risorse che necessitano di esposizione diretta

a Internet, ovvero l'Application Load Balancer e un NAT Gateway. Le risorse computazionali, cioè le istanze EC2, risiedono invece in subnet private dedicate, una per l'ambiente di produzione e una per quello di sviluppo. Questo disegno architetturale assicura che le istanze applicative non siano mai direttamente raggiungibili dall'esterno. Il traffico in ingresso (**ingress**) è sempre mediato dall'ALB, mentre il traffico in uscita (**egress**), necessario per le chiamate a servizi esterni, è canalizzato attraverso il NAT Gateway. Le tabelle di rotta sono configurate di conseguenza, impedendo alle subnet private qualsiasi rotta diretta verso l'Internet Gateway.

Storage e processi CI/CD Per lo storage degli artefatti e dei log si utilizzano bucket Amazon S3, protetti da policy restrittive. Il bucket impiegato da Elastic Beanstalk per le versioni applicative è protetto contro la cancellazione accidentale, mentre quello associato ad **AWS CodePipeline**, che contiene gli artefatti di build, impone la cifratura lato server e la comunicazione tramite TLS. Il processo di Continuous Delivery è automatizzato da una pipeline in CodePipeline, articolata nelle fasi canoniche di Source, Build e Deploy. La fase di Source è innescata da commit su branch specifici per lo sviluppo e da tag semantici per la produzione. La fase di Build, eseguita da **AWS CodeBuild**, include passaggi di sicurezza aggiuntivi per l'ambiente produttivo, come l'analisi di vulnerabilità (SCA/CVE) e la firma digitale degli artefatti. Infine, la fase di Deploy avviene con una strategia di **rolling update** per minimizzare i disservizi, e richiede un'approvazione manuale per ogni rilascio in produzione. Gli eventi salienti della pipeline vengono notificati tramite Amazon SNS.

4.1.2 Implementazione del Modello Zero Trust e del Principio del Minimo Privilegio

Come già analizzato [2], il modello **Zero Trust** rappresenta un superamento del paradigma di sicurezza tradizionale basato sul perimetro. L'adozione di questo principio risulta cruciale nel contesto delle startup, i cui ambienti operativi sono per natura dinamici e flessibili. Le caratteristiche intrinseche delle startup non solo giustificano, ma impongono la necessità di un framework di sicurezza ispirato a tale modello. Approfondiamo ora gli aspetti chiave a sostegno di questa visione:

- **Instabilità relazionale:** Le relazioni professionali nelle startup possono deteriorarsi rapidamente a causa della pressione elevata, degli obiettivi spesso poco definiti e della mancanza di esperienza nella gestione dei conflitti, sia a livello dirigenziale che operativo. Questi fattori creano tensioni che possono sfociare in incomprensioni e scontri personali. Secondo un'analisi di CB

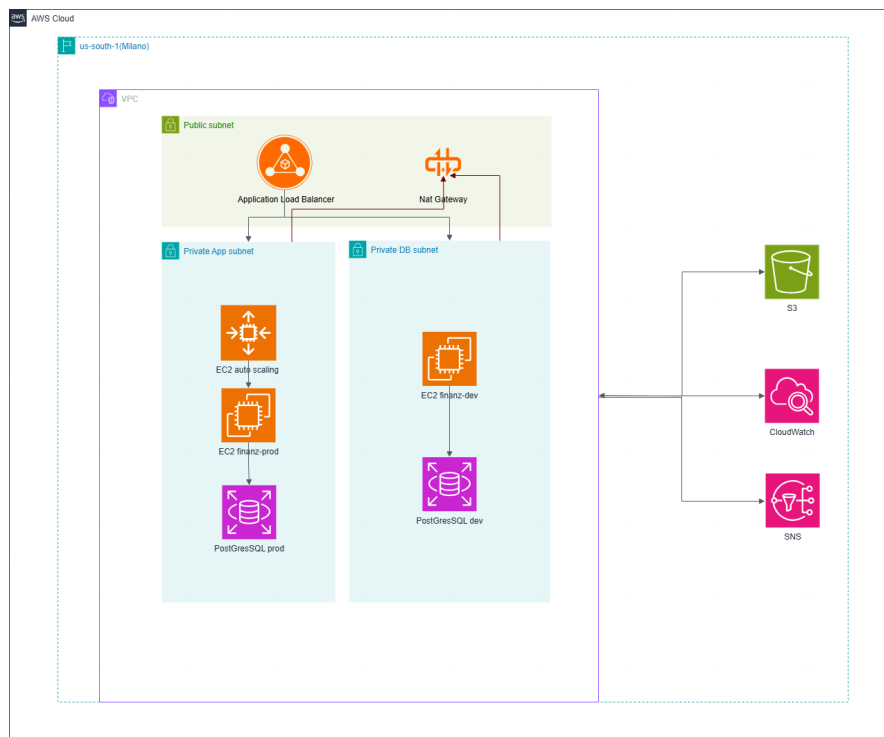


Figura 5: Diagramma semplificato dell'architettura AWS di *Finanz*.

Insights, i conflitti interni tra fondatori rappresentano una delle principali cause di fallimento delle startup, incidendo per circa il 13% dei casi esaminati [48].

- **Rischio di attacchi interni:** La fragilità dei rapporti interni, unita a dinamiche di potere e insoddisfazioni personali, aumenta la probabilità che ex-collaboratori con accessi privilegiati possano compiere azioni vendicative o dannose. Questo rischio è amplificato dalla scarsa attenzione alle politiche di controllo degli accessi e alla gestione delle risorse umane. Il "2023 Data Breach Investigations Report" di Verizon evidenzia che circa il 20% delle violazioni di dati coinvolge insider, sottolineando l'importanza di monitorare e limitare gli accessi privilegiati [49].
- **Infrastrutture di sicurezza inadeguate:** Le startup spesso destinano la maggior parte delle risorse allo sviluppo del prodotto e alla crescita del mercato, trascurando gli investimenti in infrastrutture di sicurezza. Inoltre, la mancanza di personale specializzato e di processi consolidati rende difficile implementare misure efficaci. Un rapporto del Ponemon Institute mostra che le piccole organizzazioni hanno una probabilità tre volte maggiore di subire attacchi informatici rispetto alle grandi imprese, proprio a causa di investimenti insufficienti e di una cultura della sicurezza ancora poco sviluppata [50].

Questa sezione illustra come i principi Zero Trust possano essere tradotti in misure di sicurezza concrete all'interno dell'infrastruttura cloud di una startup, con specifico riferimento all'ambiente AWS. Ci concentreremo in particolare sulla gestione delle identità e degli accessi, un pilastro fondamentale per qualsiasi architettura Zero Trust, e sulla sua stretta interconnessione con il **Principio del Minimo Privilegio (Principle of Least Privilege - PoLP)**.

Sinergia tra Principio del Minimo Privilegio (PoLP) e Zero Trust

Il Principio del Minimo Privilegio non è solo una buona pratica di sicurezza a sé stante, ma è intrinsecamente legato e **fondamentale per il successo di un'architettura Zero Trust**. La loro sinergia si manifesta in diversi modi:

- **Riduzione della Superficie d'Attacco:** Limitando strettamente le azioni consentite a ciascuna identità, PoLP riduce l'insieme delle operazioni che un attaccante potrebbe eseguire anche riuscendo a compromettere le credenziali di quell'identità. La verifica dell'identità (Zero Trust) è necessaria ma non sufficiente; i privilegi limitati (PoLP) ne circoscrivono le capacità.

- **Limitazione del Raggio d’Esplosione (*Blast Radius*):** In caso di compromissione o errore, i danni potenziali sono confinati. Un utente o servizio con privilegi minimi non può accedere o modificare risorse al di fuori del suo ambito operativo ristretto, limitando il movimento laterale dell’attaccante e l’impatto dell’incidente.
- **Applicazione della Verifica Esplicita:** Implementare PoLP costringe a definire policy di accesso granulari e intenzionali, basate sulle reali necessità operative. Questo si allinea perfettamente con la richiesta di Zero Trust di basare ogni decisione di accesso su policy esplicite e dinamiche, piuttosto che su autorizzazioni ampie o ereditate implicitamente.
- **Miglioramento del Controllo e dell’Auditabilità:** Policy di accesso minimali e specifiche sono più facili da comprendere, gestire e verificare. Ciò semplifica l’audit della postura di sicurezza e la dimostrazione della conformità, permettendo di attestare che gli accessi sono effettivamente limitati come richiesto dal modello Zero Trust.

4.1.3 Gestione delle Identità e degli Accessi (IAM) come Pilastro di Zero Trust in AWS

L’infrastruttura ospitata su un Cloud Service Provider (CSP) come AWS è un asset critico per una startup fintech. Essa contiene dati sensibili degli utenti e ospita i servizi essenziali (endpoint API, istanze EC2 per server applicativi, networking VPC, ecc.) che ne garantiscono l’operatività. La protezione di queste risorse inizia dalla gestione rigorosa di chi può accedervi e cosa può fare. **AWS Identity and Access Management (IAM)** è il servizio centrale per implementare questi controlli e costituisce una base imprescindibile per un modello Zero Trust.

Una delle prime e più critiche aree di intervento riguarda l’**account root di AWS**. Questo account possiede privilegi illimitati sull’intero ambiente AWS e rappresenta, di conseguenza, un obiettivo di altissimo valore per gli attaccanti e una fonte significativa di rischio operativo se usato impropriamente. Un’implementazione Zero Trust richiede misure stringenti per l’account root:

- **Limitazione Estrema dell’Uso:** L’accesso come utente root deve essere evitato per le operazioni quotidiane e riservato esclusivamente a quelle poche attività che lo richiedono obbligatoriamente (es. modifica delle informazioni di fatturazione, chiusura dell’account, modifica dei piani di supporto).
- **Protezione Robusta delle Credenziali:** La password deve essere estremamente complessa e, soprattutto, l’**Autenticazione a Più Fattori (MFA)** deve essere *sempre* abilitata e richiesta per l’accesso root.

- **Monitoraggio Continuo:** Ogni azione eseguita tramite l'account *root* deve essere tracciata e monitorata tramite servizi come AWS CloudTrail, generando allarmi per qualsiasi utilizzo.

4.1.4 Valutazione dell'implementazione IAM corrente di *Finanz*

Analisi degli accessi e delle policy

L'analisi dell'infrastruttura di *Finanz* (marzo 2025) ha evidenziato che il Chief Technology Officer (CTO) opera direttamente con l'account *root* dell'ambiente AWS. Oltre a tale account, sono stati identificati due utenti IAM distinti che dispongono di privilegi amministrativi completi tramite l'assegnazione della policy `AdministratorAccess`.

Si è riscontrato che, nei 90 giorni precedenti l'analisi, l'account *root* è stato utilizzato 76 volte per eseguire attività che potevano essere delegate a ruoli con privilegi limitati, in palese violazione del principio del minimo privilegio.

Inoltre, tutte le policy di autorizzazione assegnate sono di tipo *AWS managed*, ovvero predefinite dal provider. Si rileva la totale assenza di condizioni contestuali (*Condition*), di controlli basati su attributi (*Attribute-Based Access Control*, ABAC) e di *permission boundary* per circoscrivere i permessi. Ad esempio, l'utente di servizio `finanz-backend` dispone della policy `AmazonS3FullAccess`, sebbene le sue mansioni richiedano unicamente l'accesso in sola lettura (*read-only*) a tre specifici bucket di produzione.

Criticità rilevate e violazioni delle best practice

Dall'analisi emergono diverse criticità significative che espongono l'organizzazione a rischi concreti. Tali configurazioni violano direttamente le raccomandazioni definite nelle *AWS Foundational Security Best Practices* [51], come dettagliato di seguito:

1. **Protezione e uso improprio dell'account *root*:** L'account principale non è protetto da un dispositivo di *Multi-Factor Authentication* (MFA) di tipo hardware e viene impiegato per attività operative ordinarie. Questa pratica contravviene alla raccomandazione **IAM-6**, che ne prescrive l'uso esclusivo per attività di emergenza.
2. **Privilegi eccessivi per gli utenti amministrativi:** L'assegnazione della policy `AdministratorAccess` a entrambi gli utenti IAM concede permessi equivalenti a quelli dell'account *root*. Tale configurazione, che consente pieni poteri su tutte le risorse, costituisce una violazione diretta della best practice **IAM-1**.

3. **Autenticazione debole per gli utenti IAM:** Per gli account amministrativi *Ferraboli* e *Giuntoni* non è obbligatorio l'uso della MFA per accedere alla console. Questo espone gli account al rischio di compromissione tramite furto di credenziali e viola la raccomandazione **IAM-5**.
4. **Mancanza di una policy per le password:** Non è stata definita una policy che imponga requisiti di complessità, rotazione e lunghezza minima per le password degli utenti. L'assenza di tale politica di sicurezza viola la direttiva **IAM-7**.
5. **Assenza di restrizioni a livello di organizzazione:** A livello di AWS Organization non sono state implementate *Service Control Policies* (SCP) restrittive. Il mantenimento della policy predefinita **FullAWSAccess** non impone alcun *guardrail* preventivo agli account membri, in contrasto con la raccomandazione **ORG-1**.
6. **Gestione delle credenziali non sicura:** Sebbene sia disponibile un'istanza di *Single Sign-On* (SSO) tramite IAM Identity Center, questa non viene utilizzata. L'accesso avviene tramite credenziali IAM statiche e a lungo termine, una pratica obsoleta e rischiosa rispetto all'uso di ruoli con credenziali temporanee.
7. **Mancanza di account di emergenza (*break-glass*):** Non sono stati predisposti account dedicati e sicuri per garantire l'accesso in scenari critici di *lock-out* amministrativo, lasciando l'organizzazione senza un piano di contingenza affidabile.

4.2 Implementazione delle Migliorie Proposte alla Gestione IAM

A partire dalle criticità evidenziate nell'analisi presentata nella Sezione 4.1.4, abbiamo progettato e implementato un piano strategico per il rafforzamento della gestione degli accessi e delle identità (IAM) all'interno dell'ambiente AWS di *Finanz*. Gli obiettivi principali di questo intervento consistono nell'applicazione rigorosa del principio del least-privilege (minimo privilegio), nella riduzione del cosiddetto blast radius in caso di compromissione di un'identità e nel pieno soddisfacimento dei nuovi requisiti normativi. Tra questi figurano le direttive AWS previste per il 2025 sull'autenticazione a più fattori (Multi-Factor Authentication, MFA) e gli standard di settore come NIST SP 800-63 e PCI DSS.

4.2.1 Ristrutturazione della Gerarchia degli Accessi

Il primo passo ha riguardato una profonda revisione della gerarchia degli accessi, con un'attenzione particolare all'account `root` e all'introduzione di meccanismi di controllo preventivo.

Revisione dell'Account `root` e delle Policy di Sicurezza

La gestione dell'account `root` è stata interamente rivista secondo il principio di utilizzo per la sola emergenza (break-glass only). Per massimizzarne la sicurezza, sono state revocate tutte le access key programmatiche preesistenti e sono stati registrati due dispositivi Multi-Factor Authentication (MFA) hardware conformi allo standard FIDO2 (es. YubiKey 5 C NFC). Tali dispositivi sono custoditi in un luogo fisico sicuro, come una cassetta di sicurezza, e il loro accesso è limitato a figure apicali designate, come il CEO, secondo una procedura formale di emergenza [52], [53]. Per le attività amministrative ordinarie, è stato invece predisposto un utente IAM dedicato al CTO, `andrea.pasini.admin`, associato al ruolo `CTO-AdminRole`. Come descritto nella Sezione 4.2.2, questo ruolo non possiede privilegi elevati diretti ma delega le operazioni potenzialmente distruttive a ruoli temporanei con permessi specifici, in linea con il principio del privilegio minimo. A ulteriore protezione degli ambienti critici, è stata implementata una policy di negazione esplicita, `DenyProdResourceDeletion`, che funge da barriera di sicurezza aggiuntiva. Come mostrato nel Listato 4.1, la policy impedisce categoricamente la cancellazione di risorse nell'ambiente di produzione (identificate tramite il tag `Environment=prod`) e la rimozione accidentale o malevola di utenti e ruoli IAM.

```

1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Sid": "DenyProdTaggableResourceDeletion",
6              "Effect": "Deny",
7              "Action": [
8                  "ec2:TerminateInstances",
9                  "rds>DeleteDBInstance",
10                 "s3>DeleteBucket",
11                 "vpc>DeleteVpc"
12             ],
13             "Resource": "*",
14             "Condition": {
15                 "StringEquals": {
16                     "aws:ResourceTag/Environment": "prod"
17                 }
18             }
19         }
20     ]
21 }
```

```

18     }
19   },
20   {
21     "Sid": "DenyCriticalIAMPrincipalDeletion",
22     "Effect": "Deny",
23     "Action": [
24       "iam:DeleteUser",
25       "iam:DeleteRole",
26       "iam:DeleteRolePolicy",
27       "iam:DetachRolePolicy"
28     ],
29     "Resource": [
30
31       "arn:aws:iam::538927841179:role/CTO-AdminRole",
32
33       "arn:aws:iam::538927841179:role/incident-responder",
34
35       "arn:aws:iam::538927841179:user/andrea.pasini.admin",
36
37       "arn:aws:iam::538927841179:user/matteo.giuntoni",
38
39       "arn:aws:iam::538927841179:user/andrea.ferraboli"
40     ]
41   }
42 ]
43 }

```

Listing 4.1: Policy IAM per negare eliminazioni in produzione

Segmentazione dei Ruoli tramite Permission Boundaries

Per garantire che i permessi non possano superare una soglia di sicurezza predefinita, abbiamo reso obbligatoria l'applicazione di Permission Boundaries a tutti i nuovi ruoli IAM. Un permission boundary definisce il perimetro massimo di autorizzazioni che un'entità può possedere, anche se la sua policy di identità ne concedesse di più. Il Listato 4.2 illustra un esempio di tale confine, **FinanzDeveloperBoundary**, che limita le azioni degli sviluppatori ai servizi necessari (EC2, S3, RDS), impedendo al contempo qualsiasi operazione su risorse di produzione e negando esplicitamente l'accesso ai servizi di gestione IAM e AWS Organizations. L'applicazione di questi confini è stata automatizzata tramite una funzione Lambda, **enforce-boundaries-lambda**, che viene attivata dagli eventi **CreateRole** e **PutRolePolicy** registrati da AWS CloudTrail.

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "AllowDevServicesAndActions",
6       "Effect": "Allow",
7       "Action": [
8         "ec2:Describe*",
9         "ec2:RunInstances",
10        "ec2:StartInstances",
11        "ec2:StopInstances",
12        "ec2:TerminateInstances",
13        "s3:ListBucket",
14        "s3:GetObject",
15        "s3:PutObject",
16        "rds:Describe*",
17        "logs:CreateLogGroup",
18        "logs:CreateLogStream",
19        "logs:PutLogEvents"
20      ],
21      "Resource": "*",
22      "Condition": {
23        "StringNotEqualsIfExists": { "aws:ResourceTag/
Environment": "prod" }
24      }
25    },
26    {
27      "Sid": "DenyIAMModificationAndProdAccess",
28      "Effect": "Deny",
29      "Action": [
30        "iam:*",
31        "organizations:*",
32        "ec2:TerminateInstances",
33        "rds>DeleteDBInstance"
34      ],
35      "Resource": "*",
36      "Condition": {
37        "StringEqualsIfExists": { "aws:ResourceTag/
Environment": "prod" }
38      }
39    },
40    {

```

```

41     "Sid": "DenyIAMModificationOutsideBoundary",
42     "Effect": "Deny",
43     "Action": [
44         "iam:AttachUserPolicy",
45         "iam:AttachRolePolicy",
46         "iam:PutUserPolicy",
47         "iam:PutRolePolicy",
48         "iam:CreatePolicy",
49         "iam:CreatePolicyVersion",
50         "iam:SetDefaultPolicyVersion",
51         "iam>DeletePolicy",
52         "iam>DeletePolicyVersion",
53         "iam:DetachUserPolicy",
54         "iam:DetachRolePolicy",
55         "iam>DeletePermissionsBoundary"
56     ],
57     "Resource": "*",
58     "Condition": {
59         "StringNotLike": {
60             "iam:PermissionsBoundary": "arn:aws:iam
::538927841179:policy/FinanzDeveloperBoundary"
61         }
62     }
63 }
64 ]
65 }

```

Listing 4.2: Esempio di FinanzDeveloperBoundary

4.2.2 Sviluppo di un Modello Ibrido Aggiornato per la Gestione degli Accessi

Per rispondere alle esigenze di una startup fintech come Finanz, che richiede agilità e, al contempo, un elevato livello di sicurezza, in questa sezione si propone un modello ibrido di *Identity and Access Management* (IAM). Questo approccio si articola su *tre gruppi baseline*—*dev*, *backend-dev* e *admin*—ai quali vengono assegnati i permessi necessari per le attività ordinarie. A questi si affiancano *quattro ruoli operativi circoscritti*, assumibili *on-demand* tramite il servizio AWS STS (Security Token Service), che richiedono sistematicamente l'autenticazione a più fattori (MFA).

L'architettura è concepita per ridurre il cosiddetto *blast-radius* (raggio d'esplosione) in caso di compromissione delle credenziali e per agevolare gli audit di

conformità (come PCI DSS o SOC-2). Il modello si allinea infatti ai principi di *least privilege* (minimo privilegio) e *zero-trust*, come delineato da standard e best practice di settore [54], [55], [56], [57].

Gruppi baseline

dev Questo gruppo è destinato agli sviluppatori front-end e full-stack.

- **EC2:** È concesso il permesso di avviare, interrompere e terminare *esclusivamente* le istanze EC2 cui è associato il tag `Environment=dev`. Non sono concessi diritti sulle istanze di produzione [58].
- **Elastic Beanstalk:** Viene garantita la facoltà di eseguire deploy (ad esempio, tramite il comando `eb deploy`) negli ambienti di sviluppo, identificati dal tag `dev`. Tale autorizzazione può essere implementata utilizzando la policy gestita `AWSElasticBeanstalkFullAccess`, rigorosamente vincolata da una clausola `Condition` basata sul tag `aws:ResourceTag/Environment=dev` [59].
- **S3:** Sono concessi permessi di lettura e scrittura nei bucket S3 designati per lo sviluppo (ad esempio, bucket con suffisso `-dev` o tag specifici), mentre l'accesso ai bucket di produzione è esplicitamente negato [60].
- **Load Balancer:** È prevista la possibilità di descrivere (API `Describe*`) i load balancer associati agli ambienti di sviluppo, ma senza alcuna facoltà di modifica [61].
- **RDS:** Viene fornito un accesso di tipo *data-reader* (sola lettura dei dati) sui cluster Aurora/RDS di sviluppo. Le operazioni modificative, come `ModifyDBInstance` o `DeleteDBInstance`, devono essere proibite [62].

backend-dev Questo gruppo è concepito per gli sviluppatori back-end con responsabilità specifiche sull'integrazione dei dati.

- Oltre a ereditare tutti i permessi del gruppo `dev`, dispone delle seguenti autorizzazioni aggiuntive.
- **RDS:** Sono aggiunti i permessi di *data-writer* (scrittura dati) sui database di sviluppo. Per l'accesso ai database di produzione (ad esempio, tramite `QueryEditor`), si raccomanda di concedere il permesso `rds-db:connect` condizionandolo tramite tag di richiesta (`aws:RequestTag/ChangeId`), il che implica un processo di approvazione formale per modifiche o query dirette.

- **SQS/SNS:** È garantita la capacità di gestire code (SQS) e topic (SNS) negli ambienti non di produzione, funzionalità essenziale per le pipeline di dati event-driven.
- **Secrets Manager:** È concesso il permesso di leggere segreti il cui ambito è limitato all'ambiente di sviluppo, ad esempio tramite l'uso di tag specifici associati al segreto [63].

admin Gruppo riservato ai Cloud Engineer o al personale DevOps responsabile del controllo e della manutenzione continua dell'infrastruttura.

- **EC2 e Auto Scaling:** Dispone della piena gestione delle istanze e dei gruppi di auto-scaling, ad eccezione di azioni altamente distruttive come l'eliminazione di VPC di produzione, che dovrebbero essere impediti da Service Control Policies (SCP) o da un *permission boundary*.
- **S3:** Ha la facoltà di modificare le *lifecycle rules* dei bucket e le policy di replica cross-region, operazioni essenziali per le strategie di backup e disaster recovery.
- **Elastic Load Balancing:** Può creare e aggiornare listener e target group in tutti gli ambienti, previa validazione formale per le modifiche in produzione.
- **RDS:** Può eseguire operazioni di manutenzione come il patching, la creazione di snapshot e la gestione del *failover* dei database.
- **IAM:** La capacità di creare o aggiornare policy IAM deve essere strettamente confinata da un *permissions-boundary* globale. Tale boundary deve impedire azioni critiche come `iam:DeleteRolePolicy` su ruoli sensibili, `organizations:DeleteOrganization` o la modifica del boundary stesso [64].

Ruoli Operativi Specifici (Assumibili On-Demand)

Questi ruoli sono concepiti per un'assunzione temporanea e strettamente necessaria, con una durata della sessione limitata (ad esempio, 1 ora) e con l'obbligo di MFA per l'attivazione. Si raccomanda che i log di CloudTrail relativi all'assunzione e all'utilizzo di tali ruoli siano archiviati in un bucket S3 immutabile, possibilmente con replica cross-region per una maggiore resilienza.

- **dev-privileged:** Estende i permessi del gruppo `dev` per consentire operazioni di manutenzione specifiche su ambienti non di produzione, come la migrazione di uno schema di database di sviluppo o l'ottimizzazione dei CPU credit su istanze dev. Le azioni devono essere rigorosamente limitate a risorse con tag `Environment=dev`.

- **db-migration**: Fornisce accesso a *AWS Database Migration Service* (DMS) e permessi critici come `rds:ModifyDBInstance` in produzione. L'uso di questo ruolo deve essere consentito solo durante finestre di manutenzione programmate e approvate, tracciate attraverso un sistema di ticketing.
- **incident-responder**: Abilita azioni rapide in caso di incidente di sicurezza, come lo scaling immediato di risorse, la modifica di security group, l'attivazione di *AWS Shield Advanced* o la modifica di regole *AWS WAFv2*. L'assunzione di questo ruolo, riservata ai membri del gruppo `admin`, deve generare notifiche di allarme immediate.
- **breakglass-admin**: Si tratta di un ruolo con privilegi amministrativi molto ampi (potenzialmente `AdministratorAccess`), il cui accesso è protetto da un processo di attivazione estremamente rigoroso (si veda la sezione 4.2.3). Il suo utilizzo è strettamente limitato a scenari di *disaster recovery* estremi. Il processo di assunzione deve essere monitorato da *AWS Config Rules* e da allarmi *CloudWatch* dedicati [65].

Mappatura dei Permessi per Servizio

Di seguito si presenta una sintesi esemplificativa di come i permessi vengono distribuiti tra i gruppi e i ruoli.

EC2 dev: permesso di `Start/Stop/Terminate` per le istanze di sviluppo. **backend-dev**: stessi permessi del gruppo `dev`, con l'aggiunta di `DescribeImages`. **admin**: controllo completo, ad eccezione di azioni critiche come `DeleteVpc` in produzione (limitate tramite `boundary` o `SCP`).

Elastic Beanstalk dev: autorizzazione al `deploy` negli ambienti di sviluppo. **backend-dev**: `deploy` e salvataggio delle configurazioni (`eb config save`) negli ambienti di sviluppo. **admin**: gestione dei template e delle versioni delle applicazioni, anche in produzione, previa adozione di specifiche cautele [59].

S3 dev: lettura/scrittura sui bucket con suffisso `-dev`. **backend-dev**: dispone inoltre dei permessi `PutObjectAcl` su bucket di log specifici. **admin**: facoltà di modificare policy di bucket (`PutBucketPolicy`) e configurazioni di replica (`PutReplicationConfiguration`) [60].

Load Balancer dev: sola descrizione (`Describe*`). **backend-dev**: registrazione di target nei target group di sviluppo (`RegisterTargets`). **admin**: creazione e modifica degli attributi dei load balancer su tutti gli ambienti, subordinando le modifiche in produzione a processi di approvazione formali [61].

RDS dev: connessione in sola lettura (`rds-db:connect`) ai database di sviluppo. **backend-dev:** esecuzione di statement tramite Data API sugli ambienti di sviluppo. **admin:** creazione di snapshot (`CreateDBSnapshot`), avvio di task di esportazione (`StartExportTask`) e gestione del failover (`FailoverDBCluster`) [62].

L'adozione di un approccio basato sul controllo degli accessi tramite tag, noto come *Tag-Based Access Control* (ABAC), può ridurre significativamente la complessità delle policy puntuali. Questo metodo consente una gestione più scalabile degli accessi man mano che gli ambienti (sviluppo, staging, produzione) crescono o si moltiplicano [58], [61].

Applicazione delle Service Control Policies (SCP) a Livello di Organizzazione

Le *Service Control Policies* (SCP) stabiliscono i confini massimi dei permessi per l'intera AWS Organization di Finanzi o per specifiche *Organizational Unit* (OU), agendo come un meccanismo di controllo centralizzato. Tali restrizioni non possono essere superate dagli amministratori degli account membri. A differenza degli altri account, quello di management dell'organizzazione non è soggetto a questi vincoli. In questo contesto, sono state implementate le seguenti policy strategiche:

- **Prevenzione della Disattivazione di Controlli di Sicurezza Critici:**

Per salvaguardare l'integrità dei meccanismi di sicurezza e l'immutabilità dell'audit trail, viene applicata una SCP ad ampio spettro. Tale policy è progettata per negare un insieme di azioni ad alto rischio, tra cui la manipolazione dei servizi di monitoraggio (AWS CloudTrail, Amazon GuardDuty, AWS Config), la modifica o l'eliminazione dei bucket S3 designati per l'archiviazione dei log, e la compromissione di ruoli IAM critici. Inoltre, essa vieta la creazione di chiavi di accesso per l'utente root, una pratica fortemente sconsigliata che introduce un rischio significativo per l'account.

```

1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Sid": "DenyCriticalSecurityChanges",
6              "Effect": "Deny",
7              "Action": [
8                  // Prevenzione della manomissione di
CloudTrail
9                  "cloudtrail:DeleteTrail",

```

```

10         "cloudtrail:StopLogging",
11         "cloudtrail:UpdateTrail",
12
13         // Prevenzione della disattivazione di
GuardDuty
14         "guardduty:DeleteDetector",
15         "guardduty:DisableOrganizationAdminAccount",
16         "guardduty:DisassociateFromMasterAccount",
17
18         // Prevenzione della disattivazione di AWS
Config
19         "config:DeleteConfigurationRecorder",
20         "config:StopConfigurationRecorder",
21
22         // Protezione dei bucket di log (da usare con
una Condition sul Resource ARN)
23         "s3:DeleteBucket",
24         "s3:PutBucketPolicy",
25         "s3:DeleteBucketPolicy",
26
27         // Protezione di ruoli IAM critici
28         "iam:DeleteRole",
29         "iam:DeleteRolePolicy",
30
31         // Divieto di creare chiavi di accesso per l'
utente root
32         "iam:CreateAccessKey",
33         "iam:UpdateAccessKey",
34         "iam:DeleteAccessKey"
35     ],
36     "Resource": "*",
37     "Condition": {
38         "StringLike": {
39             "aws:PrincipalArn": "arn:aws:iam::*:root"
40         }
41     }
42 }
43 ]
44 }
45

```

Listing 4.3: SCP per la protezione dei controlli di sicurezza fondamentali

La verifica di una policy analoga in un ambiente di sviluppo ha confermato la sua efficacia nel bloccare i tentativi di disabilitazione di tali servizi, anche quando eseguiti da utenti con privilegi amministrativi.

- **Restrizione Geografica delle Regioni AWS:** Per ragioni di conformità normativa (ad esempio, il GDPR) e per ridurre la superficie d'attacco, si raccomanda di limitare l'operatività alle sole regioni AWS approvate (come eu-central-1, eu-south-1 e eu-west-1). Una SCP può essere impiegata per impedire il provisioning di risorse in regioni non autorizzate [66].

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "DenyNonApprovedRegions",
6       "Effect": "Deny",
7       "NotAction": [
8         "iam:*", "organizations:*", "route53:*", "
9         cloudfront:*",
10        "support:*", "health:*", "budgets:*", "waf-
11        regional:*"
12      ],
13      "Resource": "*",
14      "Condition": {
15        "StringNotEquals": {
16          "aws:RequestedRegion": [
17            "eu-central-1", "eu-south-1", "eu-west-1",
18            "us-east-1"
19          ]
20        },
21        "ArnNotLike": {
22          "aws:PrincipalARN": "arn:aws:iam::*:role/
23          OrganizationAccountAccessRole"
24        }
25      }
26    }
27  ]
28 }

```

Listing 4.4: SCP per limitare le regioni AWS utilizzabili

L'utilizzo della clausola `NotAction` è cruciale per escludere i servizi globali (come IAM e Route 53) che non sono legati a una regione specifica. La

regione `us-east-1` viene inclusa in via eccezionale, poiché ospita gli endpoint di alcuni di questi servizi.

- **Obbligo di Autenticazione a Più Fattori (MFA):** In linea con le più recenti direttive di sicurezza, è stata introdotta una SCP che rende obbligatoria l'autenticazione a più fattori (MFA). Questa policy nega qualsiasi operazione se non eseguita all'interno di una sessione autenticata tramite un secondo fattore, anticipando i requisiti che AWS renderà mandatori.

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "BlockUnlessMFAPresent",
6       "Effect": "Deny",
7       "Action": "*",
8       "Resource": "*",
9       "Condition": {
10        "BoolIfExists": { "aws:MultiFactorAuthPresent":
11          "false" }
12      }
13    ]
14  }
15
```

Listing 4.5: SCP per richiedere l'uso obbligatorio di MFA (RequireMFA)

4.2.3 Introduzione di un Break-Glass Account

Per scenari d'emergenza estrema, come un attacco ransomware che compromette l'IdP o errori di configurazione IAM catastrofici, è fondamentale disporre di un meccanismo di “rottura del vetro” (Break Glass). Di seguito viene delineata la soluzione proposta, con ID, nomenclatura e servizi aggiornati per coerenza.

1. **Account AWS dedicato e isolato:** Viene creato un nuovo account AWS all'interno dell'Organization (`o-4g2j3d5e61`), posizionato nella OU «Security». Questo account, con ID riservato `538927841179`, è mantenuto operativamente isolato, senza risorse operative e con un ciclo di fatturazione separato per garantirne l'integrità.
2. **Utente di emergenza BreakGlassEmergencyUser:** All'interno dell'account Break-Glass, viene configurato un singolo utente IAM, il cui ARN

è `arn:aws:iam::538927841179:user/BreakGlassEmergencyUser`. La sua sicurezza è affidata a una password di 32 caratteri generata casualmente, custodita in una busta fisica sigillata, e a un dispositivo MFA hardware FIDO2 dedicato (es. YubiKey, Seriale: YK-87654321). Per ridurre la superficie d'attacco, l'utente non dispone di access key permanenti e la sua unica abilità è quella di poter assumere il ruolo `BreakGlassAdminRole`.

3. **Ruolo amministrativo `BreakGlassAdminRole`:** Questo ruolo, definito nell'account Break-Glass, è il fulcro operativo del meccanismo. È associato alla policy gestita `AdministratorAccess` e la sua *trust-policy* è configurata per consentire l'assunzione del ruolo esclusivamente da parte di `BreakGlassEmergencyUser`. A sua volta, il ruolo è autorizzato, tramite `sts:AssumeRole`, ad assumere il ruolo `OrganizationAccountAccessRole` presente in tutti gli altri account operativi, garantendo così la capacità di intervento a livello di intera organizzazione.
4. **Procedura di attivazione rigorosa:** L'utilizzo del Break-Glass Account è un evento eccezionale, la cui attivazione richiede un'autorizzazione congiunta e tracciata del CEO e del CTO, con la registrazione formale dell'evento in ServiceNow. Ogni sessione attivata ha una durata massima di 8 ore e deve essere seguita da una revisione post-incidente obbligatoria per analizzare le cause e le azioni intraprese.
5. **Monitoraggio e lockdown automatico:** È implementato un sistema di monitoraggio intensivo. Una regola EventBridge rileva ogni evento di login (`aws.signin`) e invia una notifica immediata al topic SNS `security-alerts-breakglass`. Inoltre, una funzione Lambda (`breakglass-auto-restrict`) viene attivata per applicare automaticamente una *permissions-boundary* restrittiva al ruolo qualora la sessione superi la durata di 8 ore, limitando così la finestra di esposizione al rischio.

Lambda di auto-restrizione Viene di seguito mostrata l'implementazione della Lambda per il lockdown automatico del ruolo.

```

1 import os, boto3, json
2 from datetime import datetime, timedelta, timezone
3
4 iam = boto3.client("iam")
5 sns = boto3.client("sns")
6 ROLE_ARN = os.environ["ROLE_TO_ASSUME"] #
7     arn:aws:iam::538927841179:role/BreakGlassAdminRole
8 BOUNDARY_ARN = os.environ["RESTRICTIVE_POLICY_ARN"]

```

```

8 SNS_TOPIC_ARN = os.environ["SNS_TOPIC_ARN"]
9
10 def lambda_handler(event, _):
11     # l'evento trigger contiene il timestamp di inizio
12     sessione
13     session_start = datetime.now(timezone.utc)
14     expiry = session_start + timedelta(hours=8)
15
16     # --- Logica di business: applica il boundary al ruolo ---
17     iam.put_role_permissions_boundary(
18         RoleName = ROLE_ARN.split('/')[1],
19         PermissionsBoundary = BOUNDARY_ARN
20     )
21
22     msg = (f"Break-Glass session restricted at
23           {session_start.isoformat()}. "
24           f"Boundary {BOUNDARY_ARN} applicato; scade alle
25           {expiry.isoformat()}")
26     sns.publish(TopicArn=SNS_TOPIC_ARN, Subject="Break-Glass
27               lockdown", Message=msg)
28     return {"statusCode": 200, "body": json.dumps({"message":
29           "Boundary applied"})}

```

Listing 4.6: Lambda semplificata di auto-lock del ruolo

Utilizzo sistematico di credenziali temporanee (STS)

Per mitigare il rischio associato a credenziali statiche a lunga durata, è cruciale imporre l'uso di credenziali temporanee ottenute tramite il servizio AWS Security Token Service (STS).

- **Accesso umano:** Gli operatori devono accedere tramite Identity Center, che gestisce l'assunzione di ruoli specifici e fornisce credenziali temporanee con una durata limitata (es. 1 ora), valide sia per la console che per la CLI.
- **Workload applicativi:** I servizi di calcolo (EC2, ECS, Lambda) devono essere associati a un ruolo IAM. Le credenziali temporanee vengono fornite e rinnovate automaticamente dall'ambiente di esecuzione (es. tramite IMDSv2), eliminando la necessità di gestire chiavi statiche nel codice.
- **Script e pipeline CI/CD:** Gli script e i processi automatizzati devono utilizzare il comando `aws sts assume-role` per ottenere credenziali a breve

termine (durata consigliata < 1 ora) legate a ruoli "runner" dedicati, creati con il minimo privilegio necessario.

```

1
2 #Ottiene credenziali temporanee e le scrive in un file
3
4 aws sts assume-role
5 --role-arn
6     arn:aws:iam::538927841179:role/S3ReadOnlyForFinanzScript
7 --role-session-name $(date +%Y%m%d_%H%M%S)
8 --duration-seconds 3600
9 --query
10     'Credentials.[AccessKeyId,SecretAccessKey,SessionToken]'
11 --output text > /tmp/aws-creds
12
13 #Esporta le credenziali come variabili d'ambiente
14
15 source /tmp/aws-creds
16
17 #Ora i comandi AWS CLI usano le credenziali temporanee
18
19 aws s3 ls s3://finanz-data-dev/
20
21 #unset delle variabili al termine dell'esecuzione
22
23 unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY
24     AWS_SESSION_TOKEN

```

Listing 4.7: Esempio di Assume-Role in uno script CI

4.2.4 Implementazione di un Workflow di Approvazione a Due Fasi (Opzionale)

Per operazioni ad altissimo impatto (es. eliminazione di un bucket S3 in produzione, disattivazione del logging), si può introdurre un workflow di approvazione multi-persona, orchestrato tramite AWS Step Functions.

1. **Trigger dell'operazione:** Un utente, pur non avendo il permesso diretto, avvia l'operazione critica invocando un endpoint di API Gateway che, a sua volta, attiva la state machine di Step Functions `critical-op-approval`.

2. **Notifica agli approvatori:** La state machine invia una richiesta di approvazione tramite SNS a un canale Slack dedicato, notificando le figure responsabili (es. CTO, Responsabile Compliance).
3. **Raggiungimento del quorum:** Il workflow attende che tutte le approvazioni richieste vengano concesse entro un tempo limite predefinito (es. 2 ore).
4. **Esecuzione controllata:** Solo a seguito del raggiungimento del quorum, la state machine procede invocando una funzione Lambda che assume un ruolo ad-hoc (`CriticalOpsRole`) per eseguire l'azione richiesta.
5. **Audit completo:** Ogni stato del processo (richiesta, approvazione, esecuzione, esito) viene meticolosamente registrato in una tabella DynamoDB e tracciato in CloudTrail, garantendo una piena auditabilità.

Tale flusso, pur introducendo un leggero overhead, riduce drasticamente il rischio di azioni irreversibili accidentali o malevole, senza ostacolare in modo significativo la velocità operativa.

4.3 Conclusioni sulla Sicurezza IAM

L'intervento strategico sulla gestione delle identità e degli accessi, descritto in questo capitolo, ha trasformato la postura di sicurezza dell'infrastruttura AWS di Finanz, spostandola da un modello permissivo e ad alto rischio a un'architettura allineata ai principi di Zero Trust e del Minimo Privilegio (PoLP). L'implementazione delle misure proposte ha risolto in modo sistematico le criticità identificate nell'analisi iniziale (Sezione 4.1.4), raggiungendo risultati concreti e misurabili. Tra questi, spiccano la messa in sicurezza dell'account root, l'eliminazione dei privilegi amministrativi permanenti per gli utenti IAM, l'imposizione dell'autenticazione a più fattori (MFA) e, soprattutto, l'introduzione di un modello granulare basato su gruppi e ruoli temporanei. L'applicazione di guardrail preventivi tramite Permission Boundaries e Service Control Policies (SCP) ha ridotto drasticamente la superficie d'attacco e il potenziale raggio d'esplosione (blast radius) in caso di compromissione di un'identità.

Tuttavia, è fondamentale riconoscere che tale rafforzamento della sicurezza introduce nuove considerazioni operative e non è esente da limitazioni. La principale contropartita è un leggero aumento dell'overhead operativo, in particolare per il team di sviluppo, che deve ora operare tramite l'assunzione di ruoli specifici (`sts:AssumeRole`) anziché con permessi diretti e persistenti. Questo modello impone anche un onere di manutenzione continua: le policy IAM devono essere

periodicamente revisionate e aggiornate per adattarsi all'introduzione di nuovi servizi AWS o all'evoluzione delle responsabilità dei team. Inoltre, il passaggio a un regime di controllo così rigoroso richiede una necessaria evoluzione culturale all'interno della startup, supportata da formazione continua per garantire che i nuovi processi di accesso vengano compresi e adottati correttamente, senza che diventino un ostacolo alla produttività.

In conclusione, la sicurezza IAM non deve essere concepita come un traguardo, ma come un processo dinamico e iterativo. Le fondamenta gettate in questo capitolo trascendono la mera mitigazione del rischio, trasformando la gestione degli accessi in un asset strategico per Finanz. Stabilire un framework sicuro, granulare e verificabile fin dalle prime fasi non solo protegge i dati sensibili, ma costruisce la fiducia necessaria per supportare una crescita aziendale ambiziosa e sostenibile nel competitivo settore fintech.

Capitolo 5

Architettura di Rete Sicura e Protezione dei Servizi Applicativi su AWS per Finanz

5.1 Introduzione alla Sicurezza dell'Infrastruttura

Una volta stabilite le fondamenta per una gestione sicura delle identità e degli accessi, come discusso nel Capitolo 4, il passo successivo è garantire la sicurezza dell'infrastruttura di rete e dei servizi applicativi ospitati su AWS. Per una startup fintech come "Finanz", la resilienza, la disponibilità e la confidenzialità dei dati e dei servizi sono altrettanto cruciali. Questo capitolo si concentra sulla progettazione di una rete virtuale sicura tramite Amazon VPC, sulla protezione delle istanze EC2, sulla salvaguardia dei dati sensibili, sull'importanza del monitoraggio continuo e sull'adozione dell'Infrastructure as Code (IaC) per una gestione coerente e sicura. L'obiettivo è delineare un'architettura che non solo supporti le operazioni attuali di Finanz, ma che sia anche scalabile e in grado di adattarsi alla crescita futura, mantenendo elevati standard di sicurezza.

5.2 Progettazione di una Rete Sicura con Amazon VPC

La base di ogni infrastruttura sicura su AWS è rappresentata da una rete virtuale ben progettata tramite **Amazon Virtual Private Cloud (VPC)**. Il VPC permette di creare un ambiente di rete logicamente isolato all'interno del cloud

AWS, sul quale si ha pieno controllo. Una progettazione VPC sicura costituisce il primo e fondamentale livello di difesa perimetrale per le risorse.

5.2.1 Subnet Pubbliche e Private: Segmentazione Essenziale

Una pratica fondamentale, la cui implementazione è stata verificata in Finanz, consiste nella suddivisione del VPC in **subnet pubbliche** e **subnet private**, distribuite su diverse Availability Zones per garantire alta disponibilità. Nel caso specifico dell'ambiente di Finanz:

- Le **subnet pubbliche** (come `subnet-0a1b2c3d` in eu-south-1a e `subnet-4e5f6789` in eu-south-1b) sono configurate con una rotta diretta verso l'Internet Gateway (IGW) del VPC. Queste subnet ospitano risorse che necessitano di essere direttamente esposte a Internet, come il NAT Gateway (es. `nat-0123456789abcdef0`) e, tipicamente, gli Application Load Balancer. Si stima che il traffico in uscita da queste subnet pubbliche (escludendo quello verso i client finali veicolato dall'ALB) ammonti a circa 50 GB/mese, principalmente per aggiornamenti e chiamate a servizi esterni da parte del NAT Gateway.
- Le **subnet private** (come `subnet-0x1y2z3w` per gli application server e `subnet-0m1n2o3p` per i database) non hanno una rotta diretta verso l'IGW. Le istanze applicative EC2 gestite da Elastic Beanstalk e le istanze database RDS risiedono, correttamente, esclusivamente in queste subnet. Il traffico interno tra queste subnet private (es. comunicazione tra server applicativi e database) è stimato intorno ai 120 GB/mese.

Questa separazione è cruciale: le risorse backend non sono direttamente raggiungibili da Internet, riducendo significativamente la loro superficie d'attacco.

5.2.2 Controllo Granulare del Traffico: Gruppi di Sicurezza e Network ACL

Il controllo del traffico di rete all'interno del VPC è affidato a due meccanismi principali, entrambi impiegati da Finanz. La loro configurazione è stata analizzata per verificarne la correttezza:

- **Gruppi di Sicurezza (Security Groups - SG):** Agiscono come firewall stateful a livello di istanza (o più precisamente, a livello di interfaccia di rete). Permettono di definire regole di traffico in entrata (ingress) e in uscita

(egress). Nella configurazione di Finanz, si identificano circa 7 Security Group principali, specializzati per i diversi tier dell'applicazione:

- **sg-web-tier** (ID es.: `sg-0a1b2c3d4e5f67890`): Associato all'ALB, permette traffico HTTPS (porta 443) e HTTP (porta 80, idealmente da reindirizzare a HTTPS) da qualsiasi sorgente (0.0.0.0/0).
- **sg-app-tier** (ID es.: `sg-1b2c3d4e5f678901`): Associato alle istanze EC2 dell'applicazione, permette traffico sulla porta dell'applicazione (es. 8080) solo dal Security Group **sg-web-tier**. Questo è un ottimo esempio di referenziazione tra SG, che limita il traffico solo alla sorgente attesa (l'ALB).
- **sg-db-tier** (ID es.: `sg-2c3d4e5f67890123`): Associato alle istanze RDS, permette traffico sulla porta del database (PostgreSQL, porta 5432) solo dal Security Group **sg-app-tier**.
- **sg-mgmt** (ID es.: `sg-3d4e5f6789012345`): Utilizzato per accessi di gestione (es. SSH sulla porta 22 o RDP sulla 3389, sebbene l'accesso diretto alle istanze dovrebbe essere evitato in favore di Systems Manager Session Manager) da un range di IP limitato, come quello dell'ufficio (es. 203.0.113.25/32).

Dall'analisi dei VPC Flow Logs dell'ultima settimana, emerge che circa il 97% del traffico campionato rispettava queste regole, mentre il 3% veniva bloccato. Quest'ultimo dato, relativo principalmente a tentativi di scansione di porte dall'esterno o a traffico interno non previsto, meriterebbe ulteriore indagine.

- **Network Access Control Lists (Network ACLs)**: Agiscono come firewall stateless a livello di subnet e rappresentano un ulteriore, fondamentale strato di difesa. A differenza dei Security Group, le NACL non sono state lasciate nella loro configurazione di default (permetti tutto), ma sono state configurate con regole restrittive secondo il principio del minimo privilegio, per rafforzare la segmentazione di rete. L'analisi ha confermato l'adozione di NACL specifiche per le subnet pubbliche e private:
 - **NACL per le Subnet Private**: Questa è la configurazione più critica. Le regole sono state definite per permettere solo il traffico strettamente necessario, bloccando di default ogni altra comunicazione.
 - * **Regole in Entrata (Ingress)**: Permettono traffico solo dalla CIDR del VPC (es. 10.0.0.0/16) sulle porte delle applicazioni (es. 8080) e del database (5432), e il traffico di ritorno (porte effimere 1024-65535) per le connessioni iniziate dall'interno.

- * **Regole in Uscita (Egress):** Permettono traffico verso la CIDR del VPC sulle porte necessarie e traffico verso Internet (0.0.0.0/0) solo sulla porta 443 (per aggiornamenti e chiamate API via NAT Gateway). Tutto il resto è negato. Questo impedisce a un'eventuale istanza compromessa di comunicare con server di comando e controllo su porte non standard.
- **NACL per le Subnet Pubbliche:** Le regole qui sono leggermente più permissive per consentire il traffico da e verso Internet, ma sempre in modo controllato.
- * **Regole in Entrata (Ingress):** Permettono traffico da Internet (0.0.0.0/0) solo sulle porte 80 e 443 (per l'ALB) e il traffico di ritorno (porte effimere) dalle subnet private.
- * **Regole in Uscita (Egress):** Permettono traffico verso Internet su tutte le porte e traffico verso le subnet private sulle porte applicative.

Questa configurazione restrittiva delle NACL implementa un robusto secondo livello di difesa perimetrale a livello di subnet, rafforzando significativamente il concetto di difesa in profondità e rendendo molto più difficile per un attaccante muoversi lateralmente all'interno del VPC.

5.2.3 NAT Gateway per l'Accesso Controllato a Internet

Come accennato, le istanze nelle subnet private, pur non essendo direttamente raggiungibili da Internet, necessitano spesso di avviare connessioni verso l'esterno (es. per scaricare aggiornamenti software o per interagire con API di terze parti). A tale scopo, Finanz utilizza un **NAT Gateway** (es. `nat-0123456789abcdef0`) posizionato in una subnet pubblica (es. in `eu-south-1a`). Le route table delle subnet private indirizzano il traffico destinato a Internet (0.0.0.0/0) verso questo NAT Gateway. Dall'analisi condotta, si è osservato che il NAT Gateway gestisce un throughput medio di circa 50 Mbps, con picchi che possono arrivare fino a 200 Mbps, specialmente durante le fasi di deployment automatizzato. I costi mensili associati a questo servizio si aggirano tipicamente sui 45-60 EUR. Per aumentare la resilienza, si potrebbe considerare di deployare NAT Gateway in più Availability Zones.

5.2.4 VPC Endpoints per Comunicazioni Private con Servizi AWS

Un'eccellente pratica di sicurezza, già in uso presso Finanz, è l'impiego di **VPC Endpoints**. Si è rilevata la presenza di un VPC Endpoint di tipo Gateway per S3

(vpce-1a2b3c4d5e6f7g8h9). Questo permette alle risorse all'interno del VPC di comunicare con Amazon S3 utilizzando la rete privata di AWS, senza che il traffico debba attraversare l'Internet Gateway o il NAT Gateway. Ciò migliora la sicurezza, riduce la latenza e può portare a risparmi sui costi di trasferimento dati. Sarebbe opportuno verificare se vengano utilizzati altri servizi AWS che supportano VPC Endpoints di tipo Interface (es. KMS, SNS, SQS) e, in caso affermativo, valutare la creazione di endpoint dedicati anche per essi.

5.2.5 Connessioni Sicure verso Ambienti Esterni (Opzionale: VPN/Direct Connect)

Allo stato attuale, l'analisi non ha evidenziato per Finanz la necessità di connettere in modo persistente la propria infrastruttura AWS a data center on-premises. Tuttavia, qualora questa esigenza dovesse emergere, AWS offre soluzioni robuste come:

- **AWS Site-to-Site VPN:** Permette di creare tunnel IPsec crittografati tra il VPC di Finanz e una rete esterna, utilizzando Internet come mezzo di trasporto. È una soluzione relativamente rapida da implementare e con costi contenuti.
- **AWS Direct Connect:** Offre una connessione di rete fisica dedicata e privata tra un data center on-premises (o una colocation facility) e AWS. Garantisce una larghezza di banda più consistente e una latenza inferiore rispetto a una VPN su Internet, ma richiede un investimento iniziale e costi operativi maggiori.

Per una startup Per una startup cloud-native come Finanz, queste opzioni sono generalmente meno prioritarie nelle fasi iniziali, ma è bene conoscerle per future evoluzioni.

5.3 Gestione Sicura delle Istanze EC2

Le istanze **Amazon EC2** costituiscono il nucleo computazionale per molte applicazioni, inclusa quella di Finanz. Attualmente, l'azienda gestisce circa 8 istanze nell'ambiente di produzione (es. i-0a1b2c3d4e5f67890) e 3 in quello di sviluppo. La loro sicurezza è, quindi, di importanza cruciale.

5.3.1 Scelta delle AMI, Patching e Hardening del Sistema Operativo

La sicurezza di un'istanza EC2 inizia dalla scelta dell'Amazon Machine Image (AMI) e dalla sua corretta configurazione.

- **Utilizzo di AMI Affidabili e Aggiornate:** Si è appurato che Finanz impiega esclusivamente AMI ufficiali fornite da AWS, come Amazon Linux 2 e Ubuntu Server 20.04 LTS. È buona norma aggiornare regolarmente le AMI di base. Si suggerisce di implementare un processo per rivedere e aggiornare le AMI almeno ogni 3 mesi, o più frequentemente se vengono rilasciate patch critiche. AWS Systems Manager Patch Manager può aiutare ad automatizzare il patching delle istanze esistenti
- **Hardening del Sistema Operativo:** È stato sviluppato e testato uno script di hardening basato sui CIS Benchmarks, eseguibile tramite user-data al primo boot. Questo script si occupa di disabilitare servizi non necessari (circa 23 servizi identificati come non essenziali per le applicazioni di Finanz), configurare 'fail2ban' per una protezione base contro attacchi brute-force SSH (se SSH è esposto, anche se idealmente non dovrebbe esserlo pubblicamente), e rafforzare la configurazione di SSHD (es. disabilitare login root, autenticazione via password).
- **Minimizzazione del Software Installato:** È fondamentale installare sulle istanze solo il software strettamente necessario. Ogni pacchetto aggiuntivo rappresenta una potenziale vulnerabilità.

Di seguito, si presenta una versione esemplificativa e commentata dello script di hardening preparato, adattabile sia ad Amazon Linux 2 che a Ubuntu.

```

1      #!/binbash
2      # Script di hardening del sistema operativo (adatto
      per Amazon Linux 2 e Ubuntu 20.04)
3      set -euo pipefail # Esce in caso di errore, variabile
      non definita o errore in una pipe
4      # set -x # Decomenta per debugging dettagliato
      durante i test
5
6      # --- Configurazione iniziale e Logging ---
7      LOG_FILE="/var/log/finanz-hardening-script.log"
8      exec > >(tee -a "${LOG_FILE}") 2>&1 # Logga stdout e
      stderr su file e console
9      echo "INFO: Inizio script di hardening del sistema
      operativo per Finanz - $(date)"

```

```

10
11  # Rileva il sistema operativo per adattare i comandi
12  OS_ID="unknown"
13  if [ -f /etc/os-release ]; then
14      . /etc/os-release
15      OS_ID=$ID
16      echo "INFO: Sistema operativo rilevato: $OS_ID"
17  else
18      echo "WARN: Impossibile determinare il sistema
19  operativo. Alcuni comandi potrebbero fallire."
20  fi
21
22  # --- Aggiornamento pacchetti e installazione utility
23  di sicurezza base ---
24  echo "INFO: Aggiornamento lista pacchetti e
25  installazione utility base (ufw/firewalld, fail2ban,
26  auditd)..."
27  if [[ "$OS_ID" == "ubuntu" ]]; then
28      apt-get update -y
29      DEBIAN_FRONTEND=noninteractive apt-get install -y
30  ufw fail2ban auditd
31  elif [[ "$OS_ID" == "amzn" ]]; then
32      yum update -y
33      yum install -y firewalld fail2ban auditd
34  fi
35  echo "INFO: Utility di sicurezza
36  installate/aggiornate."
37
38  # --- Disabilitazione Servizi Non Necessari ---
39  echo "INFO: Tentativo di disabilitazione servizi non
40  strettamente necessari..."
41  SERVICES_TO_DISABLE=(
42      "cups" "avahi-daemon" "bluetooth" "ModemManager"
43      "apport" "whoopsie" # Comuni
44      "nfs-server" "rpcbind" "xinetd" "telnet.socket" #
45  Servizi di rete datati o specifici
46      # Per Ubuntu, potremmo aggiungere:
47      "saned" "snapd.socket" "bolt" "smartmontools"
48      "anacron" "lxcfs" "speech-dispatcher"
49      # Per Amazon Linux, alcuni potrebbero non
50  esistere o avere nomi diversi
51  )

```

```

42     for service in "${SERVICES_TO_DISABLE[@]"; do
43         # Verifica se il servizio esiste prima di tentare
di disabilitarlo
44         if systemctl list-units --full --all | grep -qF
"$service.service"; then
45             echo "INFO: Disabilitazione e stop di
$service..."
46             systemctl stop "$service" &>/dev/null || echo
"WARN: Impossibile stoppare $service (potrebbe essere
gia\' stoppato)"
47             systemctl disable "$service" &>/dev/null ||
echo "WARN: Impossibile disabilitare $service (potrebbe
non esistere o essere statico)"
48             elif systemctl list-unit-files | grep -qF
"$service.service"; then # Prova a disabilitare anche
se non attivo
49                 echo "INFO: Servizio $service trovato ma non
attivo, tentativo di disabilitazione..."
50                 systemctl disable "$service" &>/dev/null ||
echo "WARN: Impossibile disabilitare $service"
51             else
52                 echo "INFO: Servizio $service non trovato,
skippato."
53             fi
54         done
55         echo "INFO: Disabilitazione servizi non necessari
completata."

56     # --- Configurazione Firewall di Base (Host-based) ---
57     echo "INFO: Configurazione firewall di base (ufw per
Ubuntu, firewalld per Amazon Linux)..."
58     if [[ "$OS_ID" == "ubuntu" ]]; then
59         ufw default deny incoming
60         ufw default allow outgoing
61         ufw allow ssh # Assicurarsi che la porta SSH sia
permessa!
62         ufw allow http # Se necessario direttamente
sull\'istanza (non comune se dietro ALB)
63         ufw allow https # Se necessario direttamente
sull\'istanza
64         # ufw allow from <IP_ALB_o_SG_ALB> to any port
<PORTA_APPLICAZIONE>
65

```

```

66     sed -i 's/ENABLED=no/ENABLED=yes/'
    /etc/ufw/ufw.conf # Assicura che ufw sia abilitato al
    boot
67     echo "y" | ufw enable || ufw reload # Abilita o
    ricarica ufw
68     echo "INFO: Firewall UFW configurato per Ubuntu."
69     elif [[ "$OS_ID" == "amzn" ]]; then
70         systemctl enable --now firewalld
71         firewall-cmd --set-default-zone=drop # Blocca
    tutto il traffico in entrata di default
72         firewall-cmd --permanent --add-service=ssh #
    Permetti SSH
73         # firewall-cmd --permanent
    --add-port=<PORTA_APPLICAZIONE>/tcp # Permetti la porta
    dell'applicazione
74         firewall-cmd --reload
75         echo "INFO: Firewall firewalld configurato per
    Amazon Linux."
76     fi
77
78     # --- Rafforzamento Configurazione SSHD ---
79     echo "INFO: Rafforzamento configurazione SSHD
    (/etc/ssh/sshd_config)..."
80     SSHD_CONFIG_FILE="/etc/ssh/sshd_config"
81     if [ -f "$SSHD_CONFIG_FILE" ]; then
82         sed -i 's/^#*PermitRootLogin .*/PermitRootLogin
    no/' "$SSHD_CONFIG_FILE"
83         sed -i 's/^#*PasswordAuthentication
    .*/PasswordAuthentication no/' "$SSHD_CONFIG_FILE" #
    Richiede key-based auth
84         sed -i 's/^#*X11Forwarding .*/X11Forwarding no/'
    "$SSHD_CONFIG_FILE"
85         sed -i 's/^#*ClientAliveInterval
    .*/ClientAliveInterval 300/' "$SSHD_CONFIG_FILE"
86         sed -i 's/^#*ClientAliveCountMax
    .*/ClientAliveCountMax 0/' "$SSHD_CONFIG_FILE"
87         grep -qxF 'Protocol 2' "$SSHD_CONFIG_FILE" ||
    echo 'Protocol 2' >> "$SSHD_CONFIG_FILE"
88         # Valutare aggiunta di: AllowUsers, AllowGroups,
    DenyUsers, DenyGroups
89         # Valutare cambio porta SSH di default (con
    attenzione ai Security Groups)

```

```

90     systemctl restart sshd || systemctl restart ssh #
    Riavvia sshd per applicare le modifiche
91     echo "INFO: Configurazione SSHD rafforzata."
92     else
93         echo "WARN: File $SSHD_CONFIG_FILE non trovato.
    Impossibile rafforzare SSHD."
94     fi
95
96     # --- Configurazione Auditd (Regole base CIS
Benchmark) ---
97     echo "INFO: Configurazione regole auditd di base..."
98
    AUDIT_RULES_FILE="/etc/audit/rules.d/99-finanz-hardening.rules"
99     cat <<EOF > "$AUDIT_RULES_FILE"
100     # Monitoraggio modifiche a file di identità\`
101     -w /etc/passwd -p war -k identity_passwd
102     -w /etc/shadow -p war -k identity_shadow
103     -w /etc/group -p war -k identity_group
104     -w /etc/gshadow -p war -k identity_gshadow
105
106     # Monitoraggio modifiche a configurazioni di sistema
critiche
107     -w /etc/sudoers -p war -k sudoers_change
108     -w /etc/sudoers.d/ -p war -k sudoers_d_change
109     -w /etc/ssh/ssh_config -p war -k sshd_config_change
110
111     # Monitoraggio uso di comandi privilegiati (esempi)
112     -a always,exit -F arch=b64 -S execve -F euid=0 -k
    privileged_exec_b64
113     -a always,exit -F arch=b32 -S execve -F euid=0 -k
    privileged_exec_b32
114
115     # Monitoraggio tentativi di modifica delle regole di
audit
116     -w /etc/audit/auditd.conf -p war -k audit_conf_change
117     -w /etc/audit/rules.d/ -p war -k audit_rules_change
118     EOF
119     augenrules --load # Carica le nuove regole
120     # systemctl restart auditd # Potrebbe essere
necessario su alcuni sistemi
121     echo "INFO: Regole auditd di base configurate in
    $AUDIT_RULES_FILE."
122

```

```

123     # --- Configurazione Fail2Ban (Esempio base per SSH)
124     ---
125     # Fail2Ban dovrebbe essere gia' installato. Creiamo
126     un jail locale per SSH.
127     JAIL_LOCAL_FILE="/etc/fail2ban/jail.local"
128     if [ -f /etc/fail2ban/jail.conf ] && [ ! -f
129     "$JAIL_LOCAL_FILE" ]; then
130         echo "INFO: Creazione configurazione locale per
131         Fail2Ban ($JAIL_LOCAL_FILE)..."
132         cat <<EOF > "$JAIL_LOCAL_FILE"
133         [DEFAULT]
134         # Escludi IP locali o fidati
135         ignoreip = 127.0.0.1/8 ::1
136
137         # Tempo di ban (in secondi)
138         bantime   = 1h
139
140         # Numero di tentativi prima del ban
141         maxretry  = 5
142
143         # Finestra temporale per il conteggio dei tentativi
144         findtime  = 10m
145
146         [sshd]
147         enabled = true
148         # Si possono specificare porte non standard se
149         necessario: port = ssh,tuaporta
150         # logpath = %(sshd_log)s # Default solitamente
151         corretto
152         # backend = %(sshd_backend)s # Default solitamente
153         corretto
154         EOF
155         systemctl enable fail2ban
156         systemctl restart fail2ban
157         echo "INFO: Configurazione locale di Fail2Ban per
158         SSH creata e servizio riavviato."
159     elif [ -f "$JAIL_LOCAL_FILE" ]; then
160         echo "INFO: File $JAIL_LOCAL_FILE gia'
161         esistente. Skippata creazione jail.local per Fail2Ban."
162     else
163         echo "WARN: Fail2Ban non sembra installato
164         correttamente. Impossibile configurare jail.local."
165     fi

```

```

156      # --- Pulizia finale (pacchetti non necessari, cache)
157      ---
158      echo "INFO: Pulizia pacchetti non necessari e
cache..."
159      if [[ "$OS_ID" == "ubuntu" ]]; then
160          apt-get autoremove -y
161          apt-get clean -y
162      elif [[ "$OS_ID" == "amzn" ]]; then
163          yum autoremove -y # Su Amazon Linux 2, yum e\` un
link a dnf o yum stesso
164          yum clean all
165      fi
166
167      echo "INFO: Script di hardening per Finanz completato
con successo - $(date)"
168      exit 0
169

```

Listing 5.1: Script di Hardening del Sistema Operativo (hardening_script.sh)

Verifica e Mantenimento Continuo della Conformità

L'esecuzione di uno script di hardening al momento del boot è un primo passo fondamentale, ma non garantisce che la configurazione rimanga sicura nel tempo. La configuration drift (modifiche non intenzionali alla configurazione) e la scoperta di nuove vulnerabilità richiedono un processo di verifica continuo. Per questo motivo, sono state integrate due pratiche di assessment periodico:

- **AWS Inspector:** Questo servizio nativo di AWS è stato configurato per eseguire scansioni automatizzate e continue delle istanze EC2. AWS Inspector non solo identifica vulnerabilità software a livello di pacchetti (CVEs), ma verifica anche la configurazione di rete e la conformità rispetto a benchmark di sicurezza, come il **CIS Benchmarks**. I risultati vengono aggregati in AWS Security Hub, fornendo una visione centralizzata delle vulnerabilità da correggere.
- **Scansioni con Strumenti Open-Source (Lynis):** In aggiunta ad AWS Inspector, vengono eseguite scansioni periodiche (trimestrali) utilizzando **Lynis**, uno strumento di auditing della sicurezza open-source. Lynis esegue un'analisi approfondita della configurazione del sistema operativo direttamente sull'istanza, fornendo un report dettagliato con un punteggio di hardening

e suggerimenti pratici. Questo approccio a doppio strumento garantisce una copertura completa, combinando la facilità di un servizio gestito con la profondità di uno strumento specializzato.

Questo processo di verifica assicura che le policy di hardening siano non solo applicate, ma mantenute nel tempo, fornendo un feedback continuo per migliorare la sicurezza delle AMI di base e degli script di configurazione.

5.3.2 Utilizzo Fondamentale di IAM Roles per le Istanze EC2

Questa si configura come una delle pratiche di sicurezza più importanti, la cui corretta implementazione è stata verificata nell'ambiente di Finanz. **Non bisogna mai salvare credenziali AWS statiche direttamente su un'istanza EC2.** Si deve, invece, associare un **IAM Role** all'istanza. L'applicazione può quindi ottenere credenziali temporanee tramite il servizio metadati dell'istanza (IMDS), assumendo i permessi definiti nel ruolo. È stato osservato che tutte le istanze EC2 di Finanz utilizzano il ruolo IAM `FinanzEC2AppRole` (ARN: `arn:aws:iam::538927841179:role/FinanzEC2AppRole`). Tale ruolo, in linea con il principio del minimo privilegio, concede permessi specifici, tra cui lettura da bucket S3, scrittura su CloudWatch Logs e accesso a parametri in AWS Systems Manager Parameter Store. L'applicazione ottiene le credenziali interrogando l'endpoint IMDS locale (`http://169.254.169.254/latest/meta-data/iam/security-credentials/FinanzEC2AppRole`), le quali vengono ruotate automaticamente da AWS ogni 6 ore. È importante anche configurare IMDSv2 (Instance Metadata Service Version 2) per una maggiore protezione contro attacchi SSRF (Server-Side Request Forgery), richiedendo una session token per accedere ai metadati.

5.3.3 Scalabilità e Disponibilità con Auto Scaling Groups

Per garantire la disponibilità dell'applicazione, Finanz impiega **Auto Scaling Groups (ASG)**, gestiti prevalentemente tramite Elastic Beanstalk. Sono stati identificati gli ASG `finanz-prod-asg` e `finanz-dev-asg`. L'ASG di produzione è configurato per mantenere 3 istanze (desired), con un minimo di 2 e un massimo di 8. Le policy di scaling sono basate su metriche CloudWatch, come l'utilizzo medio della CPU del gruppo. Ad esempio:

- Utilizzo medio della CPU del gruppo $> 70\%$ per almeno 2 minuti consecutivi
→ Azione di Scale Out (aggiunta di istanze).
- Utilizzo medio della CPU del gruppo $< 30\%$ per almeno 10 minuti consecutivi
→ Azione di Scale In (rimozione di istanze).

- Talvolta vengono usate anche metriche di rete, come Network In > 50 MB/min per istanza, come trigger aggiuntivo per lo Scale Out.

Il tempo medio osservato per il provisioning di una nuova istanza è di circa 4 minuti e 30 secondi, garantendo una buona reattività ai cambiamenti di carico.

5.4 Protezione dei Dati Sensibili: Un Imperativo per le fintech

In una startup fintech, la protezione dei dati è di massima priorità. AWS offre una suite completa di strumenti per implementare una solida strategia in merito.

5.4.1 Crittografia dei Dati: a Riposo (At Rest) e in Transito (In Transit)

La crittografia è un pilastro della protezione dati. L'analisi ha confermato che Finanz adotta misure di crittografia per entrambe le modalità.

- **Crittografia a Riposo (At Rest):** È essenziale crittografare i dati memorizzati.
 - **Amazon S3:** Tutti i bucket critici sono configurati per la crittografia lato server con chiavi gestite da AWS KMS (SSE-KMS), utilizzando una Customer Managed Key (CMK) specifica. Il bucket dei log (`finanz-logs-538927841179`) ha anche S3 Object Lock abilitato in modalità Governance con un periodo di retention di 7 anni, per garantire l'immutabilità dei log a fini di compliance e audit.
 - **Amazon EBS:** Tutti i volumi EBS sono configurati per la crittografia di default a livello di account, utilizzando una CMK dedicata.
 - **Amazon RDS:** Entrambe le istanze PostgreSQL utilizzano la crittografia at-rest integrata di RDS, anch'essa basata su AWS KMS con una CMK dedicata. Dai benchmark consultati, l'impatto sulle performance di questa crittografia è minimo, solitamente inferiore al 2%.
- **Crittografia in Transito (In Transit):** È altrettanto cruciale proteggere i dati mentre viaggiano sulla rete, sia esternamente (tra i client e AWS) sia internamente (tra i servizi AWS).
 - **Comunicazioni Esterne:** L'Application Load Balancer (ALB) termina le connessioni TLS/SSL utilizzando certificati gestiti tramite AWS

Certificate Manager (ACM). È stata verificata la policy di sicurezza TLS sull'ALB, configurata per accettare solo protocolli moderni (es. TLS 1.2 o superiore), e circa il 99.7% del traffico osservato utilizza queste versioni. Le connessioni HTTP sulla porta 80 vengono automaticamente reindirizzate a HTTPS sulla porta 443.

- **Comunicazioni Interne:** Per le comunicazioni all'interno del VPC, ci si affida all'isolamento fornito dal VPC stesso. Per una sicurezza rafforzata, si potrebbe valutare l'implementazione di TLS anche per queste comunicazioni interne, sebbene possa introdurre complessità nella gestione dei certificati.

5.4.2 Gestione Centralizzata delle Chiavi Crittografiche con AWS KMS

Come evidenziato, **AWS Key Management Service (KMS)** gioca un ruolo centrale. Permette di creare e controllare le chiavi crittografiche (Customer Managed Keys - CMKs). Nell'account di Finanz, si identificano circa 8 CMK attive per diversi servizi, tra cui:

- **finanz-s3-encryption-key:** Utilizzata per la crittografia SSE-KMS dei bucket S3. Registra un utilizzo di circa 1000 operazioni di crittografia/decriptografia al giorno.
- **finanz-rds-encryption-key:** Utilizzata per la crittografia at-rest delle istanze RDS PostgreSQL. Utilizzo meno frequente, circa 50 operazioni al giorno (principalmente durante backup e restore).
- **finanz-ebs-encryption-key:** Utilizzata per la crittografia dei volumi EBS aggiuntivi o per sovrascrivere la chiave di default. Circa 20 operazioni al giorno.
- **finanz-secrets-key:** Utilizzata da AWS Secrets Manager per crittografare i segreti memorizzati (es. password, chiavi API). Circa 200 operazioni al giorno, in base agli accessi ai segreti da parte delle applicazioni.
- Altre chiavi potrebbero essere usate per CloudTrail log encryption, CodePipeline artifacts, ecc.

. L'uso di CMK offre un controllo granulare sui permessi, la rotazione automatica e l'auditing del loro utilizzo tramite CloudTrail. I costi mensili per KMS si aggirano tipicamente sui 15-20 EUR, dovuti principalmente al costo di ogni CMK (circa 1 EUR/mese ciascuna) e al volume di operazioni API che richiedono l'uso della chiave.

Per requisiti ancora più elevati, si potrebbe considerare **AWS CloudHSM**, ma KMS offre un eccellente equilibrio tra sicurezza, gestibilità e costi per la maggior parte dei casi d'uso.

5.4.3 Strategie di Backup e Disaster Recovery (DR)

Backup regolari e testati sono essenziali per la business continuity. Finanz ha implementato una strategia di backup utilizzando **AWS Backup**.

- **Piano di Backup Centralizzato:** Un piano denominato `FinanzDailyBackupPlan` orchestra i backup per:
 - **Istanze RDS:** Backup giornalieri automatici delle istanze RDS (sia produzione che sviluppo) vengono eseguiti durante la finestra di manutenzione notturna, con una retention dei backup di 30 giorni. Questi snapshot permettono il Point-In-Time Recovery (PITR) fino a 5 minuti prima dell'ultimo backup.
 - **Volumi EBS:** Backup settimanali (snapshot) di tutti i volumi EBS critici (associati alle istanze di produzione) vengono eseguiti ogni domenica, con una retention di 12 settimane.
 - **Bucket S3:** La versioning è abilitata su tutti i bucket critici, fornendo una forma di recupero da eliminazioni o sovrascritture accidentali. Per un DR più robusto, si sta valutando S3 Cross-Region Replication (CRR) per i bucket più importanti.
 - **Cross-Region Backup:** Per scopi di Disaster Recovery, i backup più importanti (es. snapshot RDS di produzione e snapshot EBS settimanali) vengono copiati mensilmente in una regione AWS secondaria (es. `eu-central-1`, Francoforte).
- **Vault di Backup Sicuro:** I backup sono archiviati in un vault dedicato (`finanz-backup-vault`), protetto da policy restrittive e crittografia.
- **Obiettivi di Ripristino (RTO/RPO):** Gli obiettivi target sono un RTO (Recovery Time Objective) massimo di 4 ore per i servizi critici di produzione e un RPO (Recovery Point Objective) massimo di 24 ore (idealmente molto meno per i dati transazionali, grazie al PITR di RDS). È fondamentale testare periodicamente le procedure di ripristino (almeno una o due volte l'anno) per assicurarsi che questi obiettivi siano raggiungibili e che le procedure siano efficaci..

5.4.4 Misure di Sicurezza Specifiche per i Bucket S3

Una configurazione errata di Amazon S3 può portare a gravi data breach. Si è constatato che Finanz adotta diverse misure di protezione:

- **Block Public Access (BPA):** La funzionalità S3 Block Public Access è abilitata a livello di account AWS e ulteriormente verificata a livello di singolo bucket per tutti i bucket che non devono essere pubblici. Questa impostazione previene la concessione accidentale di accesso pubblico ai dati. La sua corretta applicazione è monitorata trimestralmente tramite una regola AWS Config custom (`s3-bucket-public-access-prohibited-check`) che verifica che BPA sia attivo su tutti i bucket non esplicitamente approvati per l'accesso pubblico (come quello per gli asset statici serviti da CloudFront).
- **Bucket Policies Granulari:** Vengono utilizzate bucket policy per definire permessi di accesso specifici. Ad esempio, il bucket dei log di CloudTrail (`finanz-cloudtrail-logs-538927841179`, che è un bucket diverso da quello generico dei log applicativi) ha una policy che permette la scrittura solo al servizio CloudTrail e la lettura solo a un ruolo IAM dedicato alla sicurezza e all'audit (es. `SecurityAuditRole`). Per il bucket `finanz-static-assets`, la policy permette l'accesso in lettura solo all'Origin Access Identity (OAI) di CloudFront, garantendo che gli asset siano serviti solo tramite la CDN.
- **S3 Access Points:** Per semplificare la gestione degli accessi a dati condivisi su larga scala o per applicazioni specifiche, Finanz sta iniziando ad utilizzare gli S3 Access Points. Attualmente sono configurati 3 access point:
 - **dev-team-access:** Fornisce accesso specifico ai bucket di sviluppo per il team di sviluppo (ARN es.: `arn:aws:s3:eu-south-1:538927841179:accesspoint/dev-team-access`).
 - **prod-read-only-data:** Fornisce accesso in sola lettura a specifici prefissi all'interno dei bucket di produzione per applicazioni o utenti che necessitano solo di leggere dati.
 - **backup-operator-access:** Utilizzato da ruoli IAM specifici per le operazioni di backup e restore, limitando l'accesso solo ai bucket e alle azioni necessarie.
- **Amazon Macie per la Scoperta di Dati Sensibili:** Finanz ha configurato Amazon Macie per eseguire scansioni settimanali dei bucket S3 (specialmente quelli che potrebbero contenere dati dei clienti o PII). Negli ultimi tre mesi, Macie ha analizzato e classificato oltre 25.000 oggetti, identificando 12 istanze di dati che potenzialmente contenevano PII (es. numeri di telefono o indirizzi email in file di log non correttamente anonimizzati). Queste identificazioni

sono state investigate e le problematiche risolte (es. migliorando i processi di logging o spostando i file in bucket con controlli più stringenti).

- **S3 Storage Lens:** Per una visibilità operativa sull'utilizzo dello storage, sulle tendenze di crescita e sulle metriche di sicurezza, Finanz utilizza S3 Storage Lens. Il dashboard di default fornisce già informazioni utili, e si sta valutando l'attivazione di metriche avanzate per analisi più approfondite.

5.5 Monitoraggio Continuo, Logging e Alerting: Vedere per Proteggere

Il principio fondamentale secondo cui non è possibile proteggere ciò che non si può osservare sottolinea l'importanza di un sistema robusto di monitoraggio, logging e alerting per rilevare attività sospette e rispondere tempestivamente agli incidenti.

5.5.1 Abilitazione e Configurazione di AWS CloudTrail e Amazon CloudWatch

Questi due servizi sono i pilastri del monitoraggio in AWS.

- **AWS CloudTrail:** È stato verificato che CloudTrail risulta abilitato in tutte le regioni e a livello di Organization. Sono stati configurati due trail principali:
 - **finanz-audit-trail** (ARN es.: `arn:aws:cloudtrail:eu-south-1:538927841179:`)
Questo è il trail principale che cattura quasi tutte le chiamate API effettuate nell'account AWS, fornendo una traccia di audit fondamentale ("chi ha fatto cosa, quando, da dove e su quale risorsa"). Include eventi di gestione e, opzionalmente, eventi dati per S3 (per bucket specifici) e Lambda.
 - **finanz-security-trail:** Un secondo trail, più specifico, potrebbe essere configurato per catturare solo eventi di sicurezza critici, magari con filtri su azioni IAM, modifiche a Security Group, terminazione di istanze RDS, ecc., per facilitare analisi mirate. (Nota: se non esiste, suggerisco di crearlo).

I log di CloudTrail vengono inviati a un bucket S3 dedicato e protetto (**finanz-cloudtrail-logs-538927841179**), con la crittografia lato server (SSE-S3 o SSE-KMS) e la convalida dell'integrità dei file di log abilitate. Questi log vengono anche inviati a CloudWatch Logs per facilitare query e

allarmi in tempo reale. Finanz analizza, in media, circa 15.000-20.000 eventi CloudTrail al giorno solo nell'ambiente di produzione.

- **Amazon CloudWatch:** È utilizzato estensivamente per raccogliere:
 - **Metriche:** metriche di performance e utilizzo da oltre 45 risorse AWS (EC2, RDS, ALB, S3, Lambda, ecc.).
 - **Log Applicativi e di Sistema:** L'agente CloudWatch unificato è installato su tutte le istanze EC2 (sia quelle gestite da Elastic Beanstalk che eventuali altre) per inviare i log delle applicazioni, i log di sistema (es. `/var/log/messages`, `/var/log/secure`) e metriche personalizzate a CloudWatch Logs. Attualmente, sono gestiti circa 23 gruppi di log principali. L'agente invia le metriche con una granularità di 60 secondi.
 - **Eventi:** CloudWatch Events (ora parte di Amazon EventBridge) viene utilizzato per reagire a cambiamenti di stato nelle risorse AWS o per schedare azioni.

Il costo mensile per CloudWatch si aggira sugli 85-95 EUR, principalmente per l'ingestion e l'archiviazione dei log e per le metriche personalizzate o quelle a risoluzione dettagliata.

5.5.2 Configurazione di Allarmi CloudWatch Proattivi

Raccogliere dati è solo il primo passo; è cruciale agire su di essi. Finanz ha configurato 28 allarmi CloudWatch principali per notifiche proattive. Alcuni esempi significativi includono:

- **HighCPUUtilization-Prod-EC2:** Triggera se l'utilizzo medio della CPU su una qualsiasi istanza EC2 di produzione supera l'80% per più di 5 minuti consecutivi.
- **DatabaseConnections-Prod-RDS-Critical:** Si attiva se il numero di connessioni al database RDS di produzione supera 400 (circa l'80% del limite massimo configurato per l'istanza `db.t4g.small`).
- **HTTP5xxErrors-Prod-ALB:** Notifica se il numero di errori HTTP 5xx (errori lato server) sull'Application Load Balancer di produzione supera 10 in un intervallo di 5 minuti.
- **UnauthorizedAPICalls-CloudTrail:** Utilizza un filtro metrico sui log di CloudTrail per rilevare e allertare su un numero anomalo di chiamate API che risultano in `AccessDenied` o `UnauthorizedOperation`.

- **RootAccountUsage-Alert:** Un allarme ad altissima priorità che triggera immediatamente per qualsiasi utilizzo (login o chiamata API) dell'account root AWS.
- **SecurityGroupChanges-Alert:** Notifica qualsiasi modifica (creazione, eliminazione, modifica di regole) ai Security Group critici.
- **NATGatewayErrorPortAllocation-Alert:** Monitora la metrica **ErrorPortAllocation** del NAT Gateway per prevenire l'esaurimento delle porte.

Questi allarmi inviano notifiche tramite SNS a canali dedicati. Da questo topic, le notifiche vengono poi inoltrate via email a un gruppo di distribuzione del team tecnico/DevOps e, per gli allarmi più critici, anche via SMS o tramite integrazione con sistemi di PagerDuty/Opsgenie (attualmente in valutazione). Negli ultimi 30 giorni, si è registrata la ricezione di 47 notifiche, di cui 3 classificate come critiche e risolte entro un tempo medio di 2 ore.

5.5.3 Utilizzo di Servizi di Sicurezza Gestiti: AWS Security Hub e Amazon GuardDuty

Per un rilevamento avanzato delle minacce, Finanz si avvale di:

- **Amazon GuardDuty:** Questo servizio è abilitato per analizzare continuamente log di VPC Flow, CloudTrail e DNS per identificare attività malevole. I finding più comuni includevano:
 - **Recon:EC2/PortProbeUnprotectedPort:** 8 occorrenze, indicanti tentativi di scansione di porte su istanze EC2 da indirizzi IP esterni. Queste sono state verificate e i Security Group sono stati confermati essere restrittivi.
 - **UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration:** 2 tentativi sospetti rilevati, che sono stati investigati approfonditamente. In questi casi specifici, si è trattato di falsi positivi dovuti a test interni, ma l'allerta è stata comunque preziosa.
 - Altri finding di tipo **Trojan:EC2/DGADomainCall** o **Stealth:IAMUser/LoggingConfiguration** (questi ultimi molto rari e da investigare con massima priorità).

Negli ultimi 90 giorni, GuardDuty ha generato 23 "finding", la maggior parte di severità bassa o media. Il costo mensile per GuardDuty è relativamente contenuto, circa 12-15 EUR, e varia in base al volume di log analizzati. Si ritiene che il valore fornito da GuardDuty superi ampiamente il suo costo contenuto.

- **AWS Security Hub:** Fornisce una vista aggregata degli avvisi di sicurezza provenienti da vari servizi AWS (GuardDuty, Inspector, Macie, ecc.). Aiuta a prioritizzare i risultati e a verificare la conformità rispetto a standard come il **CIS AWS Foundations Benchmark**. Attualmente, Security Hub in Finanz mostra:
 - 127 finding totali attivi negli ultimi 30 giorni (aggregati da tutte le fonti).
 - L'89% di questi sono classificati come di severità BASSA.
 - L'8% sono di severità MEDIA.
 - Il 3% sono di severità ALTA (tutti quelli storici di alta severità sono stati risolti entro 24 ore dalla loro identificazione).

Il punteggio di conformità attuale di Finanz si attesta intorno all'87%, e Security Hub evidenzia i controlli non conformi per pianificare le azioni correttive.

5.6 Automazione e Coerenza con Infrastructure as Code (IaC)

Per garantire coerenza, ridurre errori manuali e facilitare le revisioni, è fortemente raccomandato gestire l'infrastruttura tramite un approccio di **Infrastructure as Code (IaC)**. Si è constatato che Finanz ha adottato questa pratica in modo estensivo.

- **Strumento Utilizzato: Terraform:** Finanz utilizza Terraform per definire e provisionare la quasi totalità della sua infrastruttura AWS. I file di configurazione sono scritti in HCL (HashiCorp Configuration Language).
- **Gestione del Codice e dello Stato:**
 - **Repository Git:** Il codice Terraform è conservato in un repository Git privato, permettendo modifiche all'infrastruttura, collaborazione tra i membri del team e la possibilità di effettuare rollback a versioni precedenti in caso di problemi. Si registrano circa 147 commit negli ultimi 6 mesi.
 - **Moduli Terraform Riutilizzabili:** Il codice Terraform è organizzato in 8 moduli principali riutilizzabili ('vpc', 'security-groups', 'ec2-instance', 'rds-cluster', 's3-bucket', 'iam-role', 'cloudwatch-monitoring',

‘backup-plan’). Questo promuove la standardizzazione e riduce la duplicazione del codice.

- **State Management Remoto e Sicuro:** Lo stato di Terraform è conservato in un bucket S3 dedicato e protetto, con un sistema di lock basato su DynamoDB.
- **Integrazione con Pipeline CI/CD e Sicurezza "Shift-Left":** Le modifiche all'infrastruttura seguono un processo CI/CD gestito tramite GitHub Actions, che integra la sicurezza fin dalle primissime fasi ("shift-left"):
 - Su ogni Pull Request (PR) che modifica il codice Terraform, la pipeline esegue una sequenza automatizzata di controlli:
 1. **Formattazione e Sintassi:** Vengono eseguiti i comandi `terraform fmt` e `terraform validate` per garantire la coerenza e la correttezza del codice.
 2. **Analisi di Sicurezza Statica (SAST):** Prima di ogni altra cosa, il codice viene analizzato da strumenti di Static Analysis Security Testing (SAST) specifici per IaC, come `tfsec` o `Checkov`. Questi tool confrontano il codice Terraform con centinaia di policy di sicurezza predefinite, identificando configurazioni rischiose come Security Group troppo permissivi, bucket S3 senza crittografia o ruoli IAM con privilegi eccessivi. Se viene rilevata una vulnerabilità di gravità alta, la pipeline fallisce, impedendo che codice insicuro possa anche solo essere proposto per il deployment.
 3. **Piano di Esecuzione:** Solo se l'analisi di sicurezza ha successo, viene eseguito il comando `terraform plan` per generare e pubblicare il piano delle modifiche, che sarà oggetto di revisione manuale.
 - L'applicazione delle modifiche (`terraform apply`) avviene solo dopo una revisione manuale della PR e del piano di Terraform da parte di un membro qualificato del team (es. il CTO o un senior cloud engineer) e dopo il merge della PR nel branch principale.
 - Negli ultimi 3 mesi, sono stati eseguiti 34 deployment di modifiche infrastrutturali tramite questa pipeline, con un tasso di successo del 98%.
- **Benefici Osservati:** L'adozione di IaC ha portato a benefici tangibili:
 - **Riduzione degli Errori:** Stima interna di una riduzione del 90% degli errori di configurazione manuale.

- **Velocità di Provisioning:** Un nuovo ambiente completo può essere creato in circa 45-60 minuti, contro i diversi giorni necessari manualmente.
- **Auditabilità e Compliance:** L'infrastruttura come codice semplifica gli audit e consente l'uso di strumenti di analisi statica per identificare problemi di sicurezza prima del deployment.
- **Ripetibilità e Coerenza:** L'infrastruttura può essere deployata in modo identico tra gli ambienti, facilitando anche il disaster recovery.

L'adozione di IaC sin dalle prime fasi si è rivelata una scelta strategica vincente, contribuendo a costruire un'infrastruttura robusta, sicura e gestibile nel tempo.

5.7 Conclusioni sulla Sicurezza dell'Infrastruttura e dei Servizi

Questo capitolo ha delineato le principali misure e pratiche per la messa in sicurezza dell'infrastruttura della startup Finanz su AWS. Dalla progettazione del VPC alla protezione delle istanze EC2, dalla crittografia dei dati al monitoraggio proattivo e all'automazione tramite IaC, emerge un approccio multi-livello. È importante sottolineare che la sicurezza non è uno stato statico, ma un processo continuo di valutazione, adattamento e miglioramento. Le minacce evolvono e i requisiti cambiano. Pertanto, per Finanz sarà cruciale mantenere un impegno costante verso la revisione periodica delle configurazioni, l'aggiornamento delle competenze e l'esecuzione di test. Solo così potrà garantire che la sua infrastruttura AWS rimanga un ambiente sicuro e affidabile per le sue operazioni fintech.

Capitolo 6

Implementazione di un Honeypot in un'Infrastruttura AWS per Startup fintech

Nell'odierno panorama della cybersecurity, le istituzioni finanziarie e le startup fintech come "Finanz" costituiscono bersagli primari per attacchi informatici la cui sofisticazione e frequenza sono in costante aumento. La gestione di dati sensibili e di transazioni economiche impone un approccio alla sicurezza che sia non solo reattivo, ma intrinsecamente proattivo. Il presente capitolo documenta la progettazione, l'implementazione e la validazione di un sistema honeypot di produzione, basato sulla suite T-Pot CE (Community Edition) versione 22.04, all'interno dell'infrastruttura cloud AWS (Amazon Web Services) di Finanz. A differenza di un mero esercizio accademico, questo progetto è stato concepito come un'iniziativa strategica, con l'obiettivo di fornire a Finanz un'intelligence sulle minacce (Threat Intelligence) specifica e azionabile. Saranno analizzate in dettaglio le scelte architetturali, le procedure tecniche di implementazione sicura su AWS, i costi operativi associati e, infine, i risultati di una simulazione di attacco controllata, condotta per validare l'efficacia del sistema dispiegato in un contesto operativo realistico.

6.1 Definizione e Utilità di un Honeypot

6.1.1 Che cos'è un Honeypot

Un honeypot in informatica è un meccanismo di sicurezza progettato per funzionare come esca, con lo scopo di attirare i cybercriminali in modo da poterne osservare metodologie, tecniche e strumenti utilizzati durante un tentativo di intrusione [67].

Il termine "honeypot" (letteralmente "barattolo di miele") riflette efficacemente la sua funzione: attirare gli aggressori informatici come il miele attira gli insetti, per poi studiarli e sviluppare contromisure adeguate [68].

Si tratta di un sistema hardware o software che simula un ambiente vulnerabile, isolato dall'infrastruttura di produzione principale dell'organizzazione, progettato per essere percepito come un bersaglio legittimo e interessante dagli attaccanti [69], [70]. L'honeypot appare deliberatamente vulnerabile e allettante, imitando un obiettivo reale come un server, una rete o un'applicazione contenente dati apparentemente preziosi [67], [69].

6.1.2 Utilità nel Contesto di una Startup fintech

Nel contesto operativo e strategico di una startup fintech, l'implementazione di un sistema honeypot è stata motivata da una serie di considerazioni ponderate, incentrate sui benefici prospettici per la struttura di sicurezza dell'organizzazione. Tali benefici includono:

1. **Rilevamento Proattivo e Precoce delle Minacce Cybernetiche:** L'obiettivo primario assegnato all'honeypot era l'identificazione di tentativi di intrusione nelle loro fasi iniziali, antecedenti a un potenziale impatto sui sistemi critici che custodiscono dati finanziari sensibili. L'adozione di un honeypot consente, infatti, il tempestivo rilevamento di attività anomale o esplicitamente malevole, quali scansioni di porte di rete (port scanning) o tentativi di accesso non autorizzato ai sistemi [67].
2. **Acquisizione di Intelligence sulle Minacce (Threat Intelligence):** Si è postulato che il sistema honeypot potesse fungere da collettore di informazioni dettagliate concernenti le tattiche, le tecniche e le procedure (TTPs) impiegate da attori ostili, con un focus specifico sugli aggressori che mirano ai servizi del settore finanziario. Tale comprensione deriva dalla raccolta sistematica e dalla successiva analisi dei log generati dalle interazioni con l'honeypot [69].
3. **Mitigazione dell'Incidenza dei Falsi Positivi:** In antitesi con numerosi altri meccanismi di sicurezza, qualsiasi interazione registrata con un honeypot è, per definizione, intrinsecamente sospetta e con alta probabilità di natura malevola. Questa caratteristica intrinseca comporta una significativa riduzione del cosiddetto "alert fatigue" (affaticamento da allerta), offrendo un vantaggio tangibile rispetto ai sistemi di rilevamento delle intrusioni (IDS) tradizionali, i quali sono sovente caratterizzati da una soglia non trascurabile di falsi positivi che necessitano di onerosa gestione e approfondimento [70].

4. **Ottimizzazione Continua delle Contromisure Difensive:** L'impiego dell'honeypot facilita il perfezionamento iterativo dei sistemi di rilevamento delle intrusioni (IDS) e il potenziamento delle capacità di risposta agli incidenti (incident response), basando tali miglioramenti sull'analisi di attacchi reali o realisticamente simulati [69]. I dati empirici raccolti, concernenti le metodologie di attacco osservate, possono essere proficuamente traslati nella definizione di nuove e più efficaci regole per i firewall perimetrali e di rete, nonché per gli stessi IDS, incrementandone la specificità e l'efficacia [71].
5. **Supporto alla Conformità Normativa (Regulatory Compliance):** L'adozione di un honeypot contribuisce attivamente alla dimostrazione di un approccio proattivo e maturo alla gestione della sicurezza informatica. Tale proattività costituisce un elemento chiave per il soddisfacimento dei requisiti stringenti imposti da quadri normativi severi, quali la Direttiva sui Servizi di Pagamento (PSD2), il Regolamento Generale sulla Protezione dei Dati (GDPR) e altre regolamentazioni specifiche del settore fintech, che enfatizzano la necessità di meccanismi di difesa robusti e di monitoraggio continuo.

6.2 Tipologie di Honeypot e Selezione Progettuale

La selezione di una tipologia di honeypot appropriata costituisce una decisione strategica, condizionata dagli obiettivi specifici di sicurezza dell'organizzazione e dalla disponibilità di risorse allocabili all'implementazione e alla gestione. Per entità emergenti nel settore fintech, una comprensione approfondita delle opzioni disponibili è cruciale per orientarsi verso la soluzione più congrua ed efficace [70]. La letteratura scientifica distingue primariamente gli honeypot in base al livello di interazione che consentono con gli attori ostili e allo scopo precipuo per cui vengono dispiegati.

Il livello di interazione definisce la profondità con cui un attaccante può interagire con il sistema esca. Gli **honeypot a bassa interazione** emulano servizi di rete comuni, quali server web, FTP o database, offrendo però funzionalità limitate e controllate per l'interazione con l'attaccante [72]. Tali sistemi sono progettati primariamente per registrare le attività iniziali e spesso automatizzate degli aggressori, come scansioni di rete e tentativi di sfruttamento di vulnerabilità note, risultando particolarmente utili per la raccolta di dati su campagne di malware su larga scala e attacchi opportunistici [71]. Come evidenziato in diversi casi studio accademici, anche sistemi che rappresentano semplicemente attacchi comuni registrati possono

fornire insight significativi [73]. La loro implementazione e manutenzione richiedono risorse computazionali e umane contenute, e il rischio intrinseco di compromissione del sistema honeypot stesso, o di utilizzo come trampolino per ulteriori attacchi, è minimizzato. Di conseguenza, si dimostrano efficaci prevalentemente contro attacchi automatizzati e scansioni di massa, che costituiscono una porzione significativa del rumore di fondo nel panorama delle minacce cybernetiche attraverso attacchi come bot e worm [74].

Al contrario, gli **honeypot ad alta interazione** mirano a replicare sistemi operativi completi, applicazioni complesse o interi segmenti di rete, fornendo un ambiente altamente realistico e interattivo [72]. Questa elevata fedeltà permette di attrarre attacchi più sofisticati e mirati, consentendo agli attaccanti un'interazione estesa e meno vincolata. Tale caratteristica è fondamentale per osservare in dettaglio tecniche avanzate, strumenti personalizzati e le TTP (Tattiche, Tecniche e Procedure) di attori ostili più evoluti [75]. La capacità di raccogliere informazioni dettagliate sui metodi d'attacco avanzati è uno dei principali vantaggi, sebbene ciò comporti una richiesta significativamente maggiore in termini di risorse computazionali, competenze tecniche per l'implementazione, il monitoraggio continuo e la gestione dei rischi. Infatti, il rischio che un honeypot ad alta interazione venga compromesso e successivamente utilizzato come testa di ponte per lanciare attacchi verso altri sistemi (interni o esterni) è considerevolmente più elevato, necessitando quindi di robuste misure di isolamento e contenimento [73].

Parallelamente alla classificazione basata sull'interazione, gli honeypot possono essere distinti in base al loro scopo principale. Gli **honeypot di ricerca** sono prevalentemente impiegati da istituzioni accademiche, governative e centri di ricerca specializzati in cybersecurity. Il loro obiettivo primario è l'analisi approfondita e dettagliata degli attacchi, degli strumenti e delle motivazioni degli aggressori, al fine di comprendere l'evoluzione delle minacce, sviluppare nuove tecniche di difesa e contribuire alla conoscenza collettiva nel campo della sicurezza informatica [67]. Questi sistemi, spesso ad alta interazione, sono generalmente complessi, richiedono un monitoraggio continuo e intensivo e sono focalizzati sulla massimizzazione della raccolta di intelligence.

D'altra parte, gli **honeypot di produzione** trovano comune impiego in contesti aziendali e sono integrati come componenti attivi all'interno di una strategia di difesa più ampia, spesso affiancando sistemi di rilevamento delle intrusioni (IDS) [67]. Il loro scopo è più operativo: sono concepiti per identificare attacchi in corso diretti verso l'organizzazione, deviare gli aggressori dai sistemi di produzione reali, generare allerte tempestive per i team di sicurezza e raccogliere informazioni utili per rafforzare le difese esistenti. L'enfasi è sul rilevamento e sulla risposta, piuttosto che sull'analisi forense esaustiva.

Considerando il contesto specifico di una startup fintech, per il presente studio

è stata operata una scelta mirata, optando per l'implementazione di un honeypot classificabile come di **Produzione** con un livello di interazione **Medio**. Nello specifico, la scelta è ricaduta sulla suite T-Pot CE (Community Edition) versione 22.04. T-Pot rappresenta una soluzione aggregata di honeypot multi-servizio, la cui architettura è basata su container Docker. Questa suite integra e orchestra una varietà di sensori honeypot noti e consolidati, tra cui Cowrie (per l'emulazione di servizi SSH e Telnet), Dionaea (progettato per catturare malware), Honeytrap (un honeypot a bassa interazione per vari protocolli), Elasticpot (un honeypot per Elasticsearch), e Suricata (un IDS/IPS di rete). La gestione e la visualizzazione dei dati raccolti avvengono tramite un'interfaccia web e un backend ELK (Elasticsearch, Logstash, Kibana) integrato.

Tale decisione è stata suffragata da diverse motivazioni strategiche. In primo luogo, l'obiettivo primario era la **raccolta di dati significativi e dettagliati sulle TTP degli attaccanti**. Un sistema a media/alta interazione come T-Pot, con la sua moltitudine di sensori, permette di andare oltre la semplice registrazione di scansioni di porte, catturando informazioni quali credenziali tentate durante attacchi brute-force, comandi eseguiti all'interno delle shell emulate dai sensori come Cowrie, e campioni di malware depositati attraverso servizi come Dionaea. In secondo luogo, si è ricercato un **bilanciamento ottimale tra il realismo dell'ambiente esca e la complessità gestionale**. T-Pot offre un ambiente sufficientemente realistico da attrarre un'ampia gamma di attaccanti, inclusi quelli non esclusivamente automatizzati, pur rimanendo gestibile all'interno di un progetto con risorse definite. La sua architettura containerizzata e gli script di installazione forniti dalla comunità ne facilitano notevolmente il dispiegamento e la manutenzione iniziale, un aspetto cruciale soprattutto per startup con team di sicurezza potenzialmente limitati. Infine, la finalità era quella di **simulare un honeypot che una startup fintech potrebbe effettivamente dispiegare per la protezione proattiva dei propri asset**. Pertanto, un honeypot di produzione multi-sensore, capace di emulare una superficie d'attacco diversificata, è stata ritenuta la scelta più coerente e rappresentativa.

6.3 Valutazione Strategica degli Honeypot nel Contesto della Cybersecurity Contemporanea

L'impiego di architetture honeypot si colloca in una consolidata tradizione di ricerca e pratica operativa volta a potenziare la postura difensiva e l'intelligence sulle minacce (Threat Intelligence). Tali sistemi sono riconosciuti come strumenti

proattivi per la sicurezza, sebbene la loro adozione implichi una ponderata valutazione dei benefici intrinseci a fronte dei rischi e delle limitazioni operative [69], [73].

I **meriti principali** derivanti dall'adozione strategica di honeypot sono molteplici e significativi. In primo luogo, essi costituiscono una fonte primaria per la **raccolta di intelligence empirica sulle minacce**. L'atto di attirare l'attaccante in un ambiente controllato consente di osservare direttamente le Tattiche, Tecniche e Procedure (TTPs) impiegate, gli strumenti specifici (es. scanner di vulnerabilità, exploit kit, payload malevoli) e i vettori di attacco privilegiati. L'identificazione di "attacking patterns" attraverso l'analisi dei dati generati dagli honeypot è un obiettivo chiave per comprendere le strategie offensive [69], [73].

Inoltre, gli honeypot facilitano una **comprensione approfondita delle metodologie di attacco** contro sistemi e servizi specifici. Mimando sistemi legittimi, permettono di studiare come gli avversari interagiscono con i target e tentano di sfruttare le vulnerabilità [69]. Le interazioni registrate offrono un quadro realistico delle vulnerabilità simulate più frequentemente sondate e delle tecniche di exploitation tentate, informazioni cruciali per informare le strategie di difesa e per l'affinamento dei controlli di sicurezza.

Un vantaggio operativo distintivo degli honeypot risiede nella loro **intrinseca capacità di minimizzare i falsi allarmi**. Poiché un honeypot, per sua natura, non dovrebbe essere oggetto di traffico legittimo, qualsiasi interazione è, a priori, da considerarsi sospetta o malevola. Ciò semplifica drasticamente il processo di analisi degli alert e permette di concentrare le risorse investigative su eventi ad alta probabilità di rischio, migliorando l'efficienza operativa dei Security Operation Centers (SOC).

La **scalabilità e la costo-efficacia**, particolarmente evidenti nell'utilizzo di soluzioni containerizzate come Docker su piattaforme cloud, rappresentano ulteriori fattori incentivanti. L'architettura modulare di molti sistemi honeypot moderni consente di implementare complesse suite di deception con risorse computazionali contenute e con una gestione semplificata dell'aggregazione e analisi dei log [69].

Nonostante i chiari vantaggi, l'impiego di honeypot presenta **caveat e rischi non trascurabili**. Il principale tra questi è il **rischio di identificazione (fingerprinting)** da parte di attori ostili sofisticati. Qualora un attaccante riconosca il sistema come un'esca, potrebbe decidere di evadere il monitoraggio, fornire informazioni fuorvianti, o addirittura tentare di compromettere l'honeypot stesso per utilizzarlo come testa di ponte (foothold), ovvero un punto di appoggio consolidato all'interno dell'infrastruttura (o di un perimetro virtuale) da cui l'attaccante può espandere la propria influenza e capacità operativa. Sebbene esistano tecniche per mitigare il fingerprinting, la natura open-source di alcune soluzioni e la diffusione di firme note possono facilitarne il rilevamento.

Proprio il **rischio di compromissione e subseguente abuso** dell'honeypot rappresenta la preoccupazione più critica, specialmente per i sistemi a media e alta interazione che offrono maggiori superfici di attacco. Una volta compromesso, un honeypot potrebbe essere utilizzato per lanciare attacchi verso terze parti, esponendo l'organizzazione a responsabilità legali e danni reputazionali. Di conseguenza, rigorose politiche di isolamento di rete e restrizioni sul traffico in uscita sono essenziali per contenere l'impatto di un'eventuale breccia.

La **gestione della complessità e delle risorse analitiche** necessarie per estrarre valore dai dati raccolti costituisce un'altra sfida. L'analisi dei voluminosi log prodotti dagli honeypot presenta diverse complessità: dalla necessaria pulizia e organizzazione dei dati grezzi alla gestione di dataset di grandi dimensioni, richiedendo competenze specializzate, tempo adeguato e strumenti di analisi appropriati.

È fondamentale non cadere nel **falso senso di sicurezza**. Un honeypot rappresenta solo un tassello di una strategia di difesa a più livelli e non può sostituire i controlli di sicurezza essenziali come firewall, sistemi di prevenzione delle intrusioni, aggiornamenti tempestivi dei sistemi e una solida igiene informatica. La sua efficacia si esprime al meglio quando viene integrato in un ecosistema di sicurezza più ampio, come parte di una strategia di protezione completa e coordinata[69].

6.4 Selezione della Piattaforma Honeypot: T-Pot CE 22.04

La selezione della tecnologia è stata il risultato di un compromesso ponderato tra realismo, gestibilità e costi, in linea con le risorse di una startup. Si sono valutate diverse tipologie di honeypot. Le soluzioni a bassa interazione, che si limitano a emulare servizi con funzionalità controllate [72], sono state scartate in quanto considerate troppo semplicistiche; esse sono in grado di rilevare prevalentemente attacchi automatizzati e forniscono un'intelligence superficiale, inadeguata a comprendere TTPs complesse [73]. All'estremo opposto, gli honeypot ad alta interazione, che replicano sistemi operativi completi [72], sono stati esclusi a causa dell'onerosità gestionale e del rischio intrinseco. La gestione di un sistema reale esposto a Internet richiede un monitoraggio intensivo e comporta un significativo rischio di compromissione e di utilizzo come testa di ponte (*foothold*) per ulteriori attacchi, un'eventualità inaccettabile [75].

La scelta è quindi ricaduta su **T-Pot CE 22.04**, una suite di honeypot a **media interazione multi-sensore**. Questa soluzione rappresenta il bilanciamento ideale per il contesto di Finanz. La sua architettura basata su container

Docker integra una diversità di sensori consolidati, quali Cowrie per l'emulazione di SSH/Telnet, Dionaea per la cattura di malware e Suricata come IDS, simulando una superficie d'attacco eterogenea e massimizzando la raccolta dati. La containerizzazione, inoltre, semplifica il dispiegamento, la gestione e l'isolamento dei singoli componenti, permettendo al contempo una facile personalizzazione. Lo stack di analisi ELK (Elasticsearch, Logstash, Kibana) integrato fornisce dashboard pronte all'uso per l'analisi immediata degli eventi, riducendo significativamente il tempo di implementazione. Essendo la Community Edition un software open-source, i costi sono limitati al solo dispiegamento in cloud, rendendola una soluzione di produzione economicamente sostenibile [67].

6.5 Implementazione Sicura dell'Architettura su AWS

L'implementazione è stata eseguita seguendo rigorosamente i principi di *security by design* e *least privilege*. L'obiettivo primario era creare un ambiente efficace nella raccolta di dati, ma al contempo ermeticamente isolato per prevenire qualsiasi rischio per l'infrastruttura di produzione di Finanz.

6.5.1 Architettura di Rete Isolata

È stato creato un Virtual Private Cloud (VPC) dedicato, denominato **Finanz-Honeypot-VPC**, completamente separato dal VPC di produzione. Lo schema concettuale dell'architettura è illustrato di seguito.

VPC Isolato (Finanz-Honeypot-VPC) in eu-south-1

```
|
|-- Public Subnet (Finanz-Honeypot-PublicSubnet)
|   |-- EC2 Instance (Finanz-Tpot-EC2) [t3a.medium, Ubuntu 22.04]
|   |   |-- Elastic IP (IP Pubblico Statico)
|   |   |-- IAM Role (Permessi minimi per CloudWatch/Secrets Manager)
|   |   '-- Security Group (Finanz-Tpot-SG)
|   '-- Network ACL (Finanz-Honeypot-NACL)
|
|-- Internet Gateway (Finanz-Honeypot-IGW)
|
'-- Integrazione con Servizi AWS di Monitoraggio e Logging
    |-- AWS CloudWatch (Logs, Alarms)
    |-- AWS GuardDuty (Threat Detection)
    '-- AWS SNS (Notifiche) & EventBridge
```

6.5.2 Configurazione dei Controlli di Sicurezza di Rete

La sicurezza del perimetro è stata la priorità assoluta. Per il Security Group **Finanz-Tpot-SG**, le regole di ingresso sono state configurate per aprire le porte dei sensori honeypot (e.g., 21, 23, 80, 443, 3306) a qualsiasi indirizzo (0.0.0.0/0) al fine di massimizzare l'esposizione agli attacchi. È fondamentale sottolineare che la porta 22 esposta pubblicamente è gestita dal sensore Cowrie e non dal demone SSH reale dell'host. Le porte di gestione di T-Pot (64294-64297) e la porta SSH dell'host, configurata su una porta non standard (2222) per offuscamento, sono state rese accessibili esclusivamente dall'IP di un Bastion Host di gestione. Le regole di uscita sono state impostate per bloccare tutto il traffico di default, consentendo connessioni solo verso gli endpoint AWS necessari e per gli aggiornamenti di sistema.

Come seconda linea di difesa, le Network ACLs (NACLs) associate alla subnet sono state configurate con regole di ingresso permissive, per non filtrare traffico potenzialmente interessante per i sensori, ma con regole di uscita estremamente restrittive, bloccando esplicitamente qualsiasi traffico non essenziale.

6.5.3 Provisioning Automatizzato e Sicuro dell'Istanza EC2

L'istanza **Finanz-Tpot-EC2** è stata lanciata tramite uno script **user-data**, garantendo un'installazione automatizzata e ripetibile. Per evitare l'uso di credenziali in chiaro, pratica insicura e inaccettabile, la gestione delle password è stata affidata ad **AWS Secrets Manager**. Una password robusta e generata casualmente per l'utente di T-Pot è stata memorizzata in Secrets Manager; all'istanza EC2 è stato associato un IAM Role con permessi di sola lettura limitati a quello specifico segreto. Lo script di installazione, illustrato nel Listato 6.1, recupera la password al momento dell'avvio e la inserisce nel file di configurazione di T-Pot.

```

1 #!/bin/bash
2 # Script per l'installazione automatizzata e sicura di T-Pot
   CE su AWS
3
4 # Aggiornamento e installazione prerequisiti
5 sudo apt-get update -y
6 sudo apt-get install -y git awscli
7
8 # Clonazione del repository T-Pot
9 cd /opt
10 sudo git clone https://github.com/telekom-security/tpotce.
   git
11 cd tpotce
12
```

```

13 # Preparazione file di configurazione
14 sudo cp ./iso/installer/tpot.conf.dist ./iso/installer/tpot.
    conf
15
16 # --- GESTIONE SICURA DELLA PASSWORD ---
17 # Recupero della password dall'AWS Secrets Manager.
18 # L'IAM Role dell'istanza deve avere i permessi per questa
    azione.
19 WEB_USER_PASSWORD=$(aws secretsmanager get-secret-value \
20 --secret-id finanz/honeypot/tpot_web_password \
21 --region eu-south-1 --query SecretString --output text)
22
23 # Iniezione sicura della password nel file di configurazione
24 # utilizzando un delimitatore non standard per 'sed'.
25 sudo sed -i "s|#WEB_PW=.*|WEB_PW=$(printf '%s\n' "
    $WEB_USER_PASSWORD")|" ./iso/installer/tpot.conf
26 sudo sed -i "s|^WEB_PW=.*|WEB_PW=$(printf '%s\n' "
    $WEB_USER_PASSWORD")|" ./iso/installer/tpot.conf
27
28 # Avvio dell'installazione non interattiva
29 sudo ./install.sh --type=standard --conf=/opt/tpotce/iso/
    installer/tpot.conf
30
31 # Configurazione post-installazione dell'agente CloudWatch
    Logs
32 # (configurato per monitorare /data/tpot/log/* e inviare a /
    finanz/tpot/logs)

```

Listing 6.1: Script `tpot-install-script.sh` per l'installazione sicura

6.5.4 Sviluppo e Integrazione di un Sensore Fintech Personalizzato

Un honeypot generico raccoglie intelligence generica. Per aumentare il valore del sistema per Finanz, è stato sviluppato e integrato un **sensore personalizzato**. Si tratta di un'applicazione web minimale scritta in Python con il framework Flask, containerizzata con Docker, e progettata per emulare un endpoint API fittizio: `POST /api/v1/payments/authorize`. Il codice sorgente dell'applicazione (Listato 6.2) è stato progettato per la massima semplicità e per una registrazione dettagliata.

```

1 from flask import Flask, request, jsonify
2 import logging

```

```

3 import json
4
5 app = Flask(__name__)
6
7 # Configura il logging per scrivere su stdout, che verra'
  catturato da Docker
8 logging.basicConfig(level=logging.INFO, format='%(asctime)s
  - %(message)s')
9
10 @app.route('/api/v1/payments/authorize', methods=['POST'])
11 def authorize_payment():
12     # Estrae i dati rilevanti dalla richiesta
13     source_ip = request.remote_addr
14     headers = dict(request.headers)
15     try:
16         body = request.get_json()
17     except:
18         body = request.get_data(as_text=True)
19
20     # Crea un record di log strutturato in formato JSON
21     log_entry = {
22         "event_type": "api_interaction",
23         "source_ip": source_ip,
24         "endpoint": request.path,
25         "method": request.method,
26         "headers": headers,
27         "body": body
28     }
29
30     # Logga l'intera interazione
31     logging.info(json.dumps(log_entry))
32
33     # Restituisce sempre una risposta di errore generica
34     return jsonify({"status": "error", "message": "
  Authorization Failed"}), 401
35
36 if __name__ == '__main__':
37     app.run(host='0.0.0.0', port=5000)

```

Listing 6.2: Codice sorgente del sensore API personalizzato (api_sensor.py)

Questa applicazione è stata containerizzata tramite un semplice `Dockerfile`. L'immagine Docker risultante è stata quindi integrata nell'ecosistema T-Pot modificando il file `docker-compose.yml` per aggiungere un nuovo servizio, mappando la

porta interna del container (5000) a una porta dell'host, che a sua volta è esposta pubblicamente. Questa personalizzazione permette di catturare tentativi di interazione specifici, come l'iniezione di SQL o NoSQL nel corpo della richiesta JSON, l'enumerazione di parametri o l'uso di token di autenticazione rubati, fornendo un'intelligence di valore inestimabile.

6.6 Analisi dei Costi e Valutazione del Valore Strategico

La sostenibilità economica di qualsiasi iniziativa di sicurezza è un fattore critico, specialmente nel contesto di una startup come Finanz, dove ogni risorsa deve essere allocata in modo ottimale. Una valutazione superficiale potrebbe erroneamente classificare un honeypot come un mero centro di costo. Tuttavia, un'analisi approfondita del Total Cost of Ownership (TCO) e del ritorno strategico sull'investimento (ROI) rivela un quadro nettamente più complesso e favorevole. Le stime seguenti sono state aggiornate per riflettere i requisiti hardware minimi raccomandati per la suite T-Pot, i listini prezzi di AWS e, soprattutto, l'impatto dei modelli di remunerazione e incentivazione tipici dell'ecosistema startup.

6.6.1 Scomposizione Analitica dei Costi (TCO)

Il TCO di questa implementazione si articola in due macro-categorie: costi diretti di infrastruttura e costi operativi di personale, la cui natura in una startup merita un'analisi specifica.

Costi Diretti di Infrastruttura AWS

I costi infrastrutturali sono stati calcolati considerando i requisiti minimi ufficiali di T-Pot, i listini AWS vigenti e le tariffe specifiche per la regione `eu-south-1` (Milano).

- **Istanza EC2 (t3a.large):** È stata selezionata un'istanza di questa classe (2 vCPU, 8 GB RAM) per garantire che lo stack ELK potesse gestire picchi di log durante attacchi intensi senza perdita di dati. Il costo on-demand è di \$0,0752/h, per un totale di circa **\$55/mese**.
- **Storage EBS (gp3, 128 GB):** Volume di storage necessario per il sistema operativo, i container di T-Pot e una ritenzione locale dei log di 30 giorni. Il costo è di circa **\$10/mese**.

- **CloudWatch Logs & GuardDuty:** Stimando un'ingestione di 20 GB di log al mese, il costo aggregato per logging e threat detection si attesta intorno a **\$31/mese**.
- **Indirizzo IP Elastico:** Un indirizzo IPv4 pubblico associato a un'istanza ha un costo orario, per un totale di circa **\$4/mese**.

Sulla base di queste voci, il TCO mensile per la sola infrastruttura è calcolabile come segue:

$$\text{TCO Infrastruttura Mensile} \approx \$55 + \$10 + \$31 + \$4 = \mathbf{\$100}$$

Costi Operativi e di Personale: Un'Analisi Contestualizzata

L'analisi dei costi operativi in una startup in fase iniziale non può limitarsi a un calcolo convenzionale basato su tariffe orarie di mercato. La realtà operativa impone di considerare modelli alternativi che minimizzano l'esborso di cassa.

Scenario 1: Risorsa Interna (Fondatore/CTO). Nelle prime fasi di vita di una startup, è comune che i compiti tecnici, anche quelli specialistici, siano svolti direttamente dai fondatori (e.g., il Chief Technology Officer). In questo scenario, il costo non è monetario, ma un **costo-opportunità**: le 4-6 ore settimanali dedicate all'analisi dell'honeypot sono ore sottratte allo sviluppo del prodotto, alla ricerca di clienti o ad altre attività core business. Pur non essendo un'uscita di cassa, è un fattore strategico di primaria importanza.

Scenario 2: Mentor Esterno o Advisor Tecnico. Per compiti altamente specialistici come la cybersecurity, è prassi comune per le startup affidarsi a mentor o advisor esterni. In questi casi, la remunerazione raramente avviene tramite pagamento di una tariffa oraria. È invece molto più frequente che avvenga tramite l'assegnazione di **quote aziendali (equity) o stock option**. Questo modello allinea gli interessi del mentor con il successo a lungo termine dell'azienda.

La Quantificazione del Costo Non-Monetario. L'assegnazione di equity non è "gratuita". Essa rappresenta un costo economico reale per l'azienda e i suoi fondatori, che si manifesta come **diluizione del capitale** (ownership dilution). Sebbene questa forma di compenso non si traduca in un'uscita di cassa diretta (*cash outflow*) e quindi non impatti il budget operativo a breve termine, è una spesa non-monetaria che deve essere contabilizzata e che influenza la struttura proprietaria e le future valutazioni dell'azienda.

In sintesi, il modello di TCO per il personale è notevolmente flessibile. In entrambi gli scenari più probabili per una startup come Finanz, **l'impatto sul flusso di cassa mensile per la gestione dell'honeypot è minimo o nullo**. Questa caratteristica rende il progetto estremamente attraente dal punto di vista della gestione del budget.

6.6.2 Impatto dei Programmi di Incentivazione AWS per Startup

Un ulteriore elemento che altera radicalmente l'equazione economica è il programma **AWS Activate**, che offre crediti cloud alle startup. Anche il tier base del programma (*Founders Tier*), che assegna \$1.000 di crediti, è sufficiente a coprire quasi interamente il costo vivo dell'infrastruttura per il primo anno (\$1.200). Con i tier superiori (*Portfolio Tier*), che possono arrivare a \$100.000 di crediti, il costo dell'infrastruttura diventa **effettivamente nullo** per diversi anni. Ne consegue che, per tutta la durata dei crediti, il TCO finanziario del progetto coincide quasi esclusivamente con il costo del personale, che come analizzato, non grava sul budget di cassa.

6.6.3 Valutazione del Ritorno sull'Investimento (ROI) Strategico

Alla luce di un TCO con un impatto minimo sulla cassa aziendale, il ROI del progetto diventa straordinariamente elevato. Secondo studi di settore (e.g., IBM Cost of a Data Breach Report), il costo medio di una violazione nel settore finanziario si attesta intorno ai \$6 milioni. Se l'intelligence fornita dall'honeypot contribuisce a ridurre la probabilità annuale di una violazione anche solo dello 0,5%, il beneficio atteso (la perdita evitata) è:

$$\Delta \text{ALE} = \$6.000.000 \times 0,005 = \mathbf{\$30.000}$$

Questo valore, che rappresenta una stima conservativa, surclassa di ordini di grandezza il TCO del sistema. Il rapporto costi/benefici è quindi eccezionalmente favorevole, rendendo l'adozione dell'honeypot non solo una scelta tecnicamente saggia, ma una decisione di business strategica e finanziariamente astuta.

6.7 Validazione Empirica e Analisi dei Risultati

Per trascendere una semplice verifica funzionale e valutare l'efficacia dell'architettura in uno scenario realistico, è stata condotta una simulazione di attacco

controllata, metodica e multi-fase, ispirata ai cicli di vita degli attacchi reali e alle tecniche di penetration testing in ambiente cloud.

6.7.1 Metodologia della Simulazione di Attacco Multi-Fase

L'esperimento è stato condotto da un'istanza EC2 esterna ("attacking machine") e ha seguito un approccio a tre fasi, che emula la progressione di un attaccante da una ricognizione opportunistica a un tentativo di post-sfruttamento mirato.

Fase 1: Ricognizione Esterna e Attacco Opportunistico

Questa fase ha simulato le azioni di un attaccante a bassa sofisticazione o di una botnet, il cui scopo è mappare la superficie d'attacco e trovare vulnerabilità "low-hanging fruit".

- **Scansione di Rete e Servizi:** Utilizzo di `Nmap` con opzioni aggressive (`-A -sV`) per identificare porte aperte, servizi in esecuzione e loro versioni, e per tentare un fingerprinting del sistema operativo.
- **Attacco a Forza Bruta:** Utilizzo di `Hydra` contro i servizi SSH, Telnet e MySQL emulati, impiegando dizionari di credenziali comuni (e.g., `rockyou.txt`) per simulare tentativi di compromissione di password deboli.
- **Enumerazione di Risorse Web:** Utilizzo di strumenti come `gobuster` per sondare l'esistenza di percorsi, file e directory comuni sul server web esposto.

Fase 2: Sfruttamento Simulato e Accesso Iniziale

Questa fase ha simulato il successo della Fase 1. Si è ipotizzato che l'attacco di forza bruta avesse avuto successo, identificando una credenziale debole (e.g., `root:123456`) per il servizio SSH emulato da Cowrie. L'azione conseguente è stata la connessione tramite SSH al servizio Cowrie utilizzando le credenziali "compromesse" per accedere alla shell interattiva fittizia.

Fase 3: Post-Sfruttamento, Escalation e Movimento Laterale

Questa è la fase più critica, che simula le azioni di un attaccante che, dopo aver ottenuto un punto d'appoggio (*foothold*), tenta di comprendere l'ambiente, scalare i privilegi e accedere a dati sensibili. I comandi eseguiti all'interno della shell Cowrie sono stati ispirati dalle tecniche di post-exploitation descritte nella letteratura specifica per AWS.

```

1 # 1. Enumerazione dell'ambiente e identità'
2 whoami
3 uname -a
4 # Tentativo di scoprire se si è in un ambiente AWS e rubare
   le credenziali
5 curl http://169.254.169.254/latest/meta-data/iam/security-
   credentials/
6
7 # 2. Tentativo di Escalation dei Privilegi (con credenziali
   fittizie)
8 # Si ipotizza che il ruolo ottenuto abbia la permission iam:
   AttachRolePolicy.
9 # L'attaccante tenta di assegnare privilegi di
   amministratore al proprio ruolo.
10 # (Nota: questo comando fallirebbe, ma l'honeypot ne
   registrerebbe il tentativo)
11 aws iam attach-role-policy --role-name Finanz-Tpot-Role --
   policy-arn arn:aws:iam::aws:policy/AdministratorAccess
12
13 # 3. Tentativo di Movimento Laterale e Esfiltrazione Dati
14 # L'attaccante cerca bucket S3 accessibili
15 aws s3 ls
16 # Tenta di scaricare un file da un bucket che sembra
   interessante
17 aws s3 cp s3://finanz-documents-prod/quarterly-report.pdf .
18
19 # 4. Tentativo di creare una backdoor per la persistenza
20 # L'attaccante prova a creare un nuovo utente IAM con le
   proprie chiavi
21 aws iam create-user --user-name backdoor_user
22 aws iam create-access-key --user-name backdoor_user

```

Listing 6.3: Comandi simulati nella fase di post-sfruttamento

6.7.2 Correlazione e Analisi Approfondita degli Eventi Rilevati

L'esperimento ha prodotto un volume ricco e stratificato di dati, confermando l'efficacia dell'architettura a ogni livello.

- **Rilevamenti di Fase 1:** Le dashboard di Kibana si sono immediatamente popolate. Suricata ha generato migliaia di alert, classificando correttamente le scansioni Nmap. Cowrie ha registrato meticolosamente ogni singola

credenziale tentata da Hydra. In parallelo, AWS GuardDuty ha generato autonomamente findings di severità Alta e Media, tra cui `Recon:EC2/Portscan` e `UnauthorizedAccess:EC2/SSHBruteForce`.

- **Rilevamenti di Fase 2 e 3:** Qui risiede il valore aggiunto principale. Il log di Cowrie ha registrato integralmente la sessione di post-sfruttamento (Listato 6.3). Ha catturato i tentativi di interrogare il metadata service di EC2, rivelando l'intento dell'attaccante di rubare credenziali IAM. Ha registrato il tentativo di *privilege escalation* tramite `aws iam attach-role-policy`, fornendo una firma comportamentale di un'attività estremamente pericolosa. Ha documentato i tentativi di movimento laterale verso S3. Sebbene questi comandi AWS fallirebbero nell'ambiente emulato, il loro tentativo registrato è di per sé un'informazione di intelligence di prim'ordine.
- **Rilevamenti del Sensore Custom:** Le interazioni mirate con l'endpoint `/api/v1/payments/authorize` sono state catturate dal sensore Flask. I log strutturati in JSON hanno permesso di analizzare non solo i payload (e.g., tentativi di SQLi), ma anche gli header HTTP (e.g., User-Agent), che possono essere utilizzati per creare firme specifiche per il WAF di produzione.

In sintesi, la validazione ha dimostrato che l'architettura non solo rileva le minacce, ma fornisce un contesto profondo sulle **intenzioni** e le **metodologie** degli attaccanti, un risultato che va ben oltre le capacità di un sistema di sicurezza tradizionale.

6.8 Conclusioni e Sviluppi Futuri

L'implementazione dell'honeypot T-Pot su AWS si è dimostrata un successo strategico per Finanz. A fronte di un costo operativo contenuto e di un impegno gestionale definito, il sistema fornisce un flusso costante di threat intelligence ad alta fedeltà, direttamente pertinente al contesto operativo della startup. Questo progetto ha dimostrato che anche una startup con risorse limitate può e deve adottare misure di sicurezza proattive, trasformando gli attacchi da minacce a preziose opportunità di apprendimento.

Il lavoro svolto pone inoltre le basi per una serie di sviluppi futuri di cruciale importanza. Tra questi si annoverano l'integrazione dei dati raccolti con un sistema SIEM (Security Information and Event Management) centrale per una correlazione avanzata degli eventi; l'automazione delle risposte (SOAR) per bloccare proattivamente gli indirizzi IP malevoli sul WAF di produzione; e l'evoluzione dei sensori custom per emulare flussi di autenticazione o di onboarding cliente, al fine di studiare TTPs ancora più sofisticate. In conclusione, l'honeypot si conferma

come un componente di difesa attivo e fondamentale nella strategia di sicurezza di un'organizzazione moderna.

Capitolo 7

Conclusioni

Bibliografia

- [1] Gartner, *Rapporto sugli investimenti globali fintech*, Investimenti globali in fintech: crescita e trend, 2018.
- [2] *Tecnofinanza - Wikipedia*, Accesso: 2023-10-01. indirizzo: <https://it.wikipedia.org/wiki/Tecnofinanza>.
- [3] *fintech: quali sono le startup e i numeri in Italia*, Accesso: 2023-10-01. indirizzo: https://blog.osservatori.net/it_it/fintech-in-italia.
- [4] W. Zhang, Y. Wu e M. Liu, «Digital payment systems: Risks and mitigation,» *Journal of Financial Innovation*, vol. 6, n. 2, 2020.
- [5] A. Milne e P. Parboteeah, «The business models and economics of peer-to-peer lending,» *European Credit Research Institute*, 2016.
- [6] L. Hornuf e A. Schwienbacher, «Internet-based entrepreneurial finance: Lessons from German crowdfunding markets,» *Journal of Corporate Finance*, vol. 50, pp. 556–591, 2018.
- [7] M. Eling e M. Lehmann, «The impact of digitalization on the insurance value chain and the regulatory response,» *The Geneva Papers on Risk and Insurance*, vol. 43, pp. 359–396, 2018.
- [8] C. Catalini e J. Gans, «Some simple economics of the blockchain,» *NBER Working Paper No. 22952*, 2016.
- [9] Anonimo, *Le sfide della cybersecurity nelle startup fintech*, Rapporto sul rischio e le vulnerabilità in ambito fintech, 2023.
- [10] Anonimo, *Differenze tra Cybersecurity Bancaria e fintech*, Analisi comparativa degli approcci di sicurezza, 2023.
- [11] T. Puschmann, «fintech,» *Business & Information Systems Engineering*, vol. 59, n. 1, pp. 69–76, feb. 2017, ISSN: 1867-0202. DOI: 10.1007/s12599-017-0464-6. indirizzo: <https://doi.org/10.1007/s12599-017-0464-6>.

- [12] G. Kaur, Z. Habibi Lashkari e A. Habibi Lashkari, «Introduction to Cybersecurity,» in *Understanding Cybersecurity Management in fintech: Challenges, Strategies, and Trends*. Cham: Springer International Publishing, 2021, pp. 17–34, ISBN: 978-3-030-79915-1. DOI: 10.1007/978-3-030-79915-1_2. indirizzo: https://doi.org/10.1007/978-3-030-79915-1_2.
- [13] 2024. indirizzo: https://www.theseus.fi/bitstream/handle/10024/853540/Kauppinen_Teppo.pdf?sequence=2&isAllowed=y.
- [14] Anonimo, *Minacce informatiche nel settore fintech*, Analisi delle tipologie di attacchi e delle vulnerabilità in ambito fintech, 2023.
- [15] J. Cawthra, M. Ekstrom, L. Lusty, J. Sexton, J. Sweetnam e A. Townsend, «Data Integrity: Detecting and Responding to Ransomware and Other Destructive Events,» National Institute of Standards e Technology, rapp. tecn. 1800-26A, dic. 2020. DOI: <https://doi.org/10.6028/NIST.SP.1800-26>.
- [16] National Institute of Standards and Technology (NIST), *Security and Privacy Controls for Information Systems and Organizations*, SP 800-53 Rev. 5, NIST, 2020. indirizzo: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>.
- [17] N. I. of Standards e Technology, «Enhanced Security Requirements for Protecting Controlled Unclassified Information,» National Institute of Standards e Technology, rapp. tecn. 800-172, 2021.
- [18] N. I. of Standards e Technology, *Least Privilege - Glossary*, NIST Computer Security Resource Center, 2015.
- [19] N. I. of Standards e Technology, *OSCAL Content with Separation of Duties*, NIST, 2021.
- [20] S. Rose, O. Borchert, S. Mitchell e S. Connelly, «Zero Trust Architecture,» National Institute of Standards e Technology, rapp. tecn. 800-207, ago. 2020.
- [21] J. H. Saltzer e M. Schroeder, «The Protection of Information in Computer Systems,» *Proceedings of the IEEE*, vol. 63, n. 9, pp. 1278–1308, 1975. DOI: 10.1109/PROC.1975.9939. indirizzo: <https://ieeexplore.ieee.org/document/1451869>.
- [22] R. Smith, «A Contemporary Look at Saltzer and Schroeder’s 1975 Design Principles,» *IEEE Security & Privacy*, vol. 10, n. 6, pp. 20–25, 2012. DOI: 10.1109/MSP.2012.85.

- [23] G. Stoneburner, C. Hayden e A. Feringa, «Engineering Principles for Information Technology Security (A Baseline for Achieving Security), Revision A,» National Institute of Standards e Technology, NIST Special Publication 800–27rA, giu. 2004. DOI: 10.6028/NIST.SP.800–27rA. indirizzo: <https://doi.org/10.6028/NIST.SP.800–27rA>.
- [24] R. Ross, V. Pillitteri, R. Graubart, D. Bodeau e R. McQuaid, «Developing Cyber-Resilient Systems: A Systems Security Engineering Approach, Volume 2,» National Institute of Standards e Technology, NIST Special Publication 800–160 Vol. 2, nov. 2019. DOI: 10.6028/NIST.SP.800–160v2. indirizzo: <https://doi.org/10.6028/NIST.SP.800–160v2>.
- [25] J. Feigenbaum, A. Jaggard e R. Wright, «Accountability in Computing: Concepts and Mechanisms,» *Foundations and Trends® in Privacy and Security*, vol. 2, n. 4, pp. 247–399, 2020. DOI: 10.1561/33000000002.
- [26] National Institute of Standards and Technology. «NIST Computer Security Resource Center Glossary: *Non-repudiation*.» Accessed 27 May 2025. indirizzo: https://csrc.nist.gov/glossary/term/non_repudiation.
- [27] *open banking europe: security & identification standards for apis & communications open banking europe -providing collaborative services to support psd2 xs2a, in partnership with the financial industry*. indirizzo: <https://www.openbankingeurope.eu/media/1943/oasis-obe-api-identification-and-security-standards-for-apis-and-communications.pdf>.
- [28] A. Cavoukian, «Privacy by Design: The 7 Foundational Principles,» Information e Privacy Commissioner of Ontario, rapp. tecn., 2009. indirizzo: [https://www.ipc.on.ca/wp-content/uploads/resources/7foundationalprinciples](https://www.ipc.on.ca/wp-content/uploads/resources/7foundationalprinciples.pdf) pdf.
- [29] P. Mell e T. Grance, *The NIST Definition of Cloud Computing*, 2011. indirizzo: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>.
- [30] M. Vats, S. Gambhir e M. Gomathy, «A Systematic Review of the Impact of Cloud Computing on fintech Industry,» *BCIIT eJournal*, vol. 1, n. 1, pp. 37–44, 2024. indirizzo: https://bciit.ac.in/pdf/eJournal/Issue1/6.Mansi_A%20Systematic%20Review%20of%20the%20Impact%20of%20Cloud%20Computing%20on%20fintech%20Industry_37-44.pdf.
- [31] Homan, Beránek et al., «Cloud Computing Adoption in SMEs: Exploring IaaS, PaaS, and SaaS through a Bibliometric Study,» *HRMARS*, 2025. indirizzo: https://hrmars.com/papers_submitted/24452/cloud-computing-adoption-in-smes-exploring-iaas-paas-and-saas-through-a-bibliometric-study.pdf.

- [32] Anonymous, «Overview of Service and Deployment Models Offered by Cloud Computing, based on International Standard ISO/IEC 17788,» *International Journal of Advanced Computer Science and Applications*, 2018. indirizzo: https://www.academia.edu/91325535/Overview_of_Service_and_Deployment_Models_Offered_by_Cloud_Computing_based_on_International_Standard_ISO_IEC_17788.
- [33] Red Hat, *Significato di IaaS, PaaS e SaaS - Cloud computing*, <https://www.redhat.com/it/topics/cloud-computing/iaas-vs-paas-vs-saas>, Accessed 2025-01-21, 2025.
- [34] Maticmind, «Modelli di distribuzione tipici del Cloud Computing,» 2024, Accessed 2025.
- [35] Anonymous, «Security Considerations for Hybrid Cloud Deployments in fintech Using Blockchain,» *Journal of Artificial Intelligence, Machine Learning and Data Science*, 2024. indirizzo: <https://urfjournals.org/open-access/security-considerations-for-hybrid-cloud-deployments-in-fintech-using-blockchain.pdf>.
- [36] «Above the Clouds: A Brief Survey,» *arXiv preprint*, ago. 2019, Survey exploring cloud computing concepts and applications, with focus on low-cost and dynamic scaling benefits for developing world companies. indirizzo: <https://arxiv.org/abs/1908.02124>.
- [37] A. W. Services, *Global Infrastructure*, 2024. indirizzo: <https://aws.amazon.com/about-aws/global-infrastructure/>.
- [38] A. W. Services, *Amazon CloudFront*, 2024. indirizzo: <https://aws.amazon.com/cloudfront/>.
- [39] A. W. Services, *AWS Networking*, 2024. indirizzo: <https://aws.amazon.com/products/networking/>.
- [40] A. W. Services, *AWS Nitro Hypervisor*, 2024. indirizzo: <https://aws.amazon.com/ec2/nitro/>.
- [41] IBM, *What Is Infrastructure as a Service (IaaS)?* <https://www.ibm.com/topics/iaas>, Accessed: [Inserisci data accesso], 2024.
- [42] A. W. Services, *AWS Well-Architected Framework*, 2024. indirizzo: <https://aws.amazon.com/architecture/well-architected/>.
- [43] *What is Amazon EC2 Auto Scaling?* Accessed: 2025-03-10, 2025. indirizzo: <https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html>.

- [44] A. W. Services, *Amazon RDS Multi-AZ Deployments*, <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>, Accessed: [Inserisci data accesso se necessario], 2024.
- [45] *AWS Well-Architected Framework*, Accessed: 2025-03-10, 2024. indirizzo: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>.
- [46] A. W. Services, *AWS Resilience Hub*, 2024. indirizzo: <https://aws.amazon.com/resilience-hub/>.
- [47] A. W. Services, *AWS Shared Responsibility Model*, 2024. indirizzo: <https://aws.amazon.com/compliance/shared-responsibility-model/>.
- [48] CB Insights, «Why Startups Fail: Top 12 Reasons,» 2023, Accessed: 2025-05-03. indirizzo: <https://www.cbinsights.com/research/report/startup-failure-reasons-top/>.
- [49] Verizon, «2023 Data Breach Investigations Report,» 2023, Accessed: 2025-05-03. indirizzo: <https://www.verizon.com/business/resources/reports/dbir/>.
- [50] Ponemon Institute, «2023 State of Cybersecurity Report,» 2023, Accessed: 2025-05-03. indirizzo: <https://www.ponemon.org/research/>.
- [51] Amazon Web Services. «AWS Foundational Security Best Practices standard,» Amazon Web Services, visitato il giorno 27 ott. 2023. indirizzo: <https://docs.aws.amazon.com/securityhub/latest/userguide/fsbp-standard.html>.
- [52] A. Saraswat. «Break Glass - When All Else Fails: Secure Emergency Access in AWS.» [Online; accesso 15-10-2024].
- [53] Cloud Defense, *Ensure hardware MFA is enabled for the root user account [AWS Foundational Security Best Practices IAM.6]*, [https://www.clouddefense.ai/compliance-rules/aws-fs-practices/iam/foundational-](https://www.clouddefense.ai/compliance-rules/aws-fs-practices/iam/foundational-security-iam-6) security-iam-6, [Online; accesso 15-10-2024], 2023.
- [54] S. Rose, O. Borchert, S. Mitchell e S. Connelly, «Zero Trust Architecture,» National Institute of Standards e Technology, rapp. tecn. SP 800-207, 2020. indirizzo: <https://doi.org/10.6028/NIST.SP.800-207>.
- [55] P. Grassi, «Digital Identity Guidelines,» National Institute of Standards e Technology, rapp. tecn. SP 800-63-3, 2023. indirizzo: <https://doi.org/10.6028/NIST.SP.800-63-3>.
- [56] PCI Security Standards Council, *PCI DSS v3.2.1 Quick Reference Guide*, https://www.pcisecuritystandards.org/pdfs/pci_ssc_quick_guide.pdf, accessed 04-May-2025, 2024.

- [57] D. S. Labs, *Best practices for creating least-privilege AWS IAM policies*, <https://www.datadoghq.com/blog/iam-least-privilege/>, accessed 04-May-2025, 2024.
- [58] Amazon Web Services, *Identity-based policies for Amazon EC2*, <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/iam-policies-for-amazon-ec2.html>, accessed 04-May-2025, 2025.
- [59] Amazon Web Services, *Elastic Beanstalk service role*, <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/concepts-roles-service.html>, accessed 04-May-2025, 2024.
- [60] Amazon Web Services, *Security best practices for Amazon S3*, <https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html>, accessed 04-May-2025, 2024.
- [61] Amazon Web Services, *Actions, resources and condition keys for AWS Elastic Load Balancing*, https://docs.aws.amazon.com/service-authorization/latest/reference/list_awselasticloadbalancing.html, accessed 04-May-2025, 2025.
- [62] Amazon Web Services, *Identity and access management for Amazon RDS*, <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/UsingWithRDSIAM.html>, accessed 04-May-2025, 2024.
- [63] Amazon Web Services, *Security best practices in IAM*, <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>, accessed 04-May-2025, 2024.
- [64] Amazon Web Services, *Permissions boundaries for IAM entities*, https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_boundaries.html, accessed 04-May-2025, 2025.
- [65] Amazon Web Services, *Temporary security credentials in IAM*, https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html, accessed 04-May-2025, 2024.
- [66] AWS Builders - dev.to community, *What are AWS Service Control Policies (SCPs)? [2022 Guide]*, <https://dev.to/aws-builders/what-are-aws-service-control-policies-scps-2022-guide-4f67>, [Online; accesso 15-10-2024], 2022.
- [67] Proofpoint. «Honeypot, cos'è? – Significato e vantaggi.» Accessed: 2024-05-06. indirizzo: <https://www.proofpoint.com/it/threat-reference/honeypot>.
- [68] UniverseIT. «Cos'è e come viene utilizzato un Honeypot.» Accessed: 2024-05-06. indirizzo: <https://universeit.blog/honeypot/>.

- [69] A. Mairh, D. Barik, K. Verma e D. Jena, «Honeypot in network security: a survey,» in *Proceedings of the 2011 International Conference on Communication, Computing & Security*, ser. ICCCS '11, Rourkela, Odisha, India: Association for Computing Machinery, 2011, pp. 600–605, ISBN: 9781450304641. DOI: 10.1145/1947940.1948065. indirizzo: <https://doi.org/10.1145/1947940.1948065>.
- [70] G. Perego, *Cos'è un honeypot*, mag. 2023. indirizzo: <https://www.insic.it/senza-categoria/cose-un-honeypot-come-funziona-e-quali-sono-le-tipologie/>.
- [71] Fortinet. «Che cos'è un Honeypot? Significato, tipi, vantaggi e altro.» Accessed: 2024-05-06. indirizzo: <https://www.fortinet.com/it/resources/cyberglossary/what-is-honeypot>.
- [72] V. Lavecchia. «Definizione di honeypot in informatica.» Accessed: 2024-05-06. indirizzo: <https://vitolavecchia.altervista.org/definizione-di-honeypot-informatica/>.
- [73] K. Gadapa, S. Sai, J. Devineni, Y. Reddy Bommu e V. Busi, *AWS Honeypot Attack Data Analysis*. indirizzo: <https://mason.gmu.edu/~sdevine3/Project%20Milestone%202.pdf>.
- [74] NordVPN. «Honeypot: Cos'è e come funziona. »indirizzo: <https://nordvpn.com/it/blog/honeypot-cose/>.
- [75] E. Tsang, *Research Honeypot on AWS*, feb. 2022. indirizzo: <https://blog.devgenius.io/creating-a-research-honeypot-on-aws-b0ded134729a>.